

网络同步在游戏历史中的发展变化

作者：Jerish

前言：

网络同步属于游戏开发中比较重要且复杂的一部分，然而这么多年来却一直没有一个相对完整的文章或书籍来详细的讲解这块内容。再加上网上资料杂乱无章，技术水平参差不齐，导致很多朋友对一些概念和原理理解有误。**作者Jerish**耗费近一年的时间，参考了大量的论文与资料（近200篇），从网络同步的基本概念讲起，循序渐进地深入到服务器架构与同步算法的实现细节，相对系统的梳理了网络同步技术的发展历史与应用场景。从内容上看该文章属于一篇论文综述，会在每一个技术后面给出对应的参考资料链接。

该内容首发在**网易雷火的知乎账号**上，经作者后续整理与修正通过微信公众号《**游戏开发那些事**》发布**免费**的PDF版本供大家参考学习。

主要内容如下：

- 1) 网络同步的基本概念和理解
- 2) 游戏服务器架构的发展历程
- 3) Lockstep（帧同步）的发展历史与技术细节
- 4) 状态同步的发展历史与技术细节
- 5) 物理同步的原理与手段
- 6) 常见的同步优化手段与分析

如果对文章有什么建议或想法，可以通过以下方式联系作者（也欢迎大家加群进行技术交流）
知乎[Jerish](#) / 微信公众号 / QQ技术交流群

微信公众号：

游戏开发那些事



游戏开发那些事(学…

QQ 群号：875867499



一、网络同步概念理解

二、网络架构与传统同步

1.网络游戏架构的发展

2.传统同步面临的问题

三、锁步同步 lockstep（帧同步）

1.早期的Lockstep

2.Bucket Synchronization

3.锁步同步协议 Lockstep protocol

4.RTS中的Lockstep

5.Pipelined Lockstep protocol

6.TimeWarp

7.Lockstep与"帧"同步

8.Lockstep小结

四、状态同步 State Synchronization

1.雷神之锤与快照同步（Quake and Snapshot）

2.《星际围城：部落》引擎中的网络架构（The TRIBES Engine Networking Model）

3.客户端预测与回滚（Client-side prediction and Rollback）

4.事件锁定与时钟同步(Event Locking and Clock Synchronization)

5.插值技术（Interpolation and Extrapolation）

6.延迟补偿（Lag Compensation）

7.Trailing state synchronization

8.状态同步框架的演变

9.守望先锋与ECS架构

10.状态同步小结

五、物理同步

1.概念与理解

2.面临的问题与解决方案

a.物理引擎的确定性问题

b.同步误差如何被物理引擎放大

c.《看门狗2》的载具同步

d.《火箭联盟》的物理同步

六、TCP VS UDP

七、常见同步优化技术

1.表现优化（弱化玩家对延迟的感受）：

a.插值优化

b.客户端预先执行+回滚

2.延迟对抗（弱化玩家对延迟的感受）：

a.延迟补偿

b.命令缓冲区

c.从具体实现的技巧上对抗延迟

3.丢包对抗（弱化玩家对延迟的感受）：

a.使用TCP而不是UDP

b.冗余UDP数据包

4.带宽优化（减小延迟）：

a.同步对象裁剪

b.分区，分房间

c.数据压缩与裁剪

d.减少遍历以及更细力度的优化

5.帧率优化（减小延迟）：

a.提升帧率

b.保持帧率稳定与匹配

c.计算压力分担

八、总结

一、网络同步概念理解

网络游戏是电子游戏中一个极为重要的品类，自网络诞生以来，游戏行业就没有停止过对多人游戏玩法的探索。但是随着技术的发展，我们发现游戏服务器的架构发展却与通常的WEB服务器架构渐行渐远。在我看来，导致二者差异化的一个重要原因就是**现代游戏服务器**（更准确的说是游戏逻辑服务器）对网络同步有着很高的即时性要求。

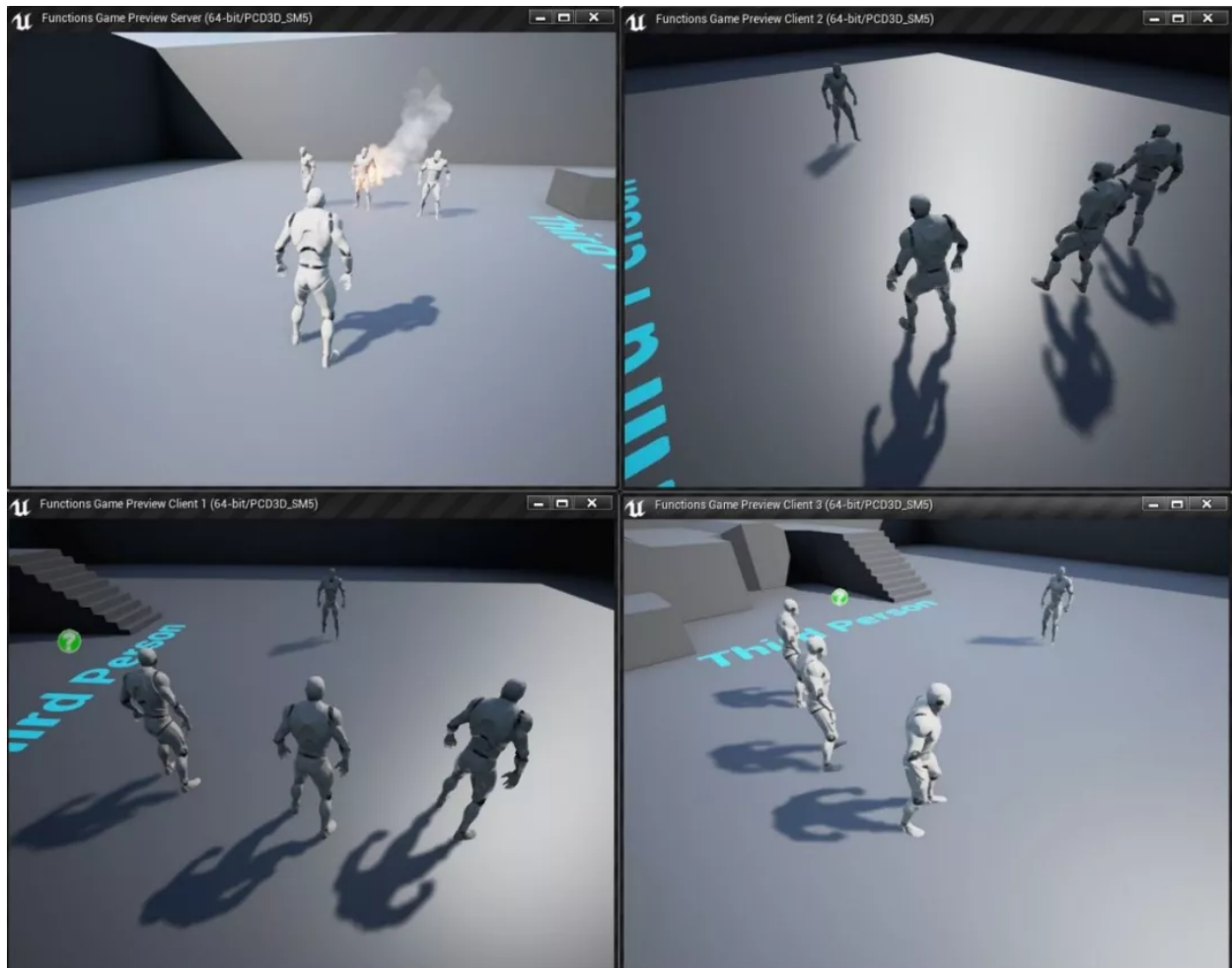


图1-1 游戏中的网络同步效果

那什么是网络同步呢？虽说是游戏中常用的概念，但其实任何与互联网有关的技术都需要网络同步，我这里会尝试用通俗易懂的方式循序渐进的给大家讲解其原理和概念。在其他领域，我们也许可以称他为“数据同步”，比如说用户A用他的手机点击按钮注册了一个QQ账号，那么他的手机号等个人信息就会被存储到服务器上面，这就是一个数据同步的过程。在这个过程中，数据由A的手机流向了QQ的官方服务器。对于游戏来说，其实原理也是一样，不过由于游戏中玩家关注的是游戏的视觉效果，所以我们不仅要同步数据，还要同步表现。可以简单认为，网络同步 = 数据同步 + 表现同步，数据同步是后端操作，而表现同步就是让前端对后端同步过来的数据进行进一步的处理从而达到表现上的一致。所以这样看来，不仅是游戏，任何拥有前端的产品都是这样的，比如浏览器、各种手机App，当你点击登录的时候，服务器就会要同步的个人信息发给你，并显示在界面上，你用任何一款手机显示的内容都是一样的。



图1-2 一般应用中的数据同步

不过一般Web服务器只是单纯的从服务器向客户端进行数据同步，不会把其他客户端的数据都发给你。而在游戏里面，你需要让N个客户端的显示看起来一模一样（由于网络延迟，同一时刻不可能完全一样），所以需要把其他玩家的一些数据也发给你，不能说A玩家跳了一下，B玩家看到A却趴下了，那样游戏就没法玩了。对于大部分的互联网产品，针对的都是个人用户的，并不需要两个用户之间进行交互，所以也就不需要不同用户的客户端界面保持一致。也行讲到这里，你会拍案而起，“浏览器不久是这样的么？我们所有人看一个网页都是一样的”。没错，最早期的一批网络游戏就是在BBS上进行的，那时候以文字交流的模式进行多人交互，效果本质上和很多页游是非常相似的。假如XXX知乎大佬的页面是一个棋盘的样子，每个人都可以任其首页上面下一枚围棋，那么这不就是一个可以全民参与的围棋游戏么？页游大概就是这么一个原理。

然而，前面我们还忽略了一个游戏中非常重要的需求（尤其是在MMO、FPS这种类型的网游中）——那就是实时性。你可以容忍微信点进去一篇文章要花2秒钟，但是你不可能接受你的子弹要2秒后才打到敌人。实际上，在各种电子竞技里面，0.1秒的延迟就足以让整个游

戏的局势发生逆转。像浏览器这种页面显示都吞吞吐吐的应用，如何用他流畅的玩FPS和MOBA呢？（关于云游戏这里先不谈）

到这里，我们再次梳理一下网络同步的概念。**可以认为 网络同步 = 实时的多端数据同步 + 实时的多端表现同步。从计算机的角度来描述，网络同步是一个网络IO与CPU计算同样密集的游戏功能，他对主机CPU的性能要求较高，且单核的性能比并发更为重要。**当然，游戏同步还有一些其他的特点，比如数据一般不要求存储、允许一定程度的丢失等，这里我们就不再进一步讨论。

了解以上网络同步的特点后，我们就可以进一步展开细节来说，比如传输的数据内容有哪些？数据的传递流向是什么样子的？我们平时口中常谈的“帧同步”，“状态同步”如何理解？为了能以一个更全面更权威的视角来讲解网络同步，笔者阅读了大量的论文和资料，对同步技术的发展和细节有了更为清晰的认识（很多内容与网上参差不齐的博客描述并不相同）。由于同步本身与服务器架构无法剥离，文章还会涉及到一些游戏网络架构的发展历史。

二、网络架构与传统同步

1. 网络游戏架构的发展

1973年夏天，高中暑期实习生在美国加利福尼亚州NASA的研究中心首次撰写了游戏《迷宫战争》[1]。通过使用[串行电缆](#)连接两台算机，增加了两人游戏功能。由于涉及两台对等的计算机，可以使用相同的格式化协议包相互发送信息，因此可以认为这是第一个**P2P架构**的电子游戏。在那个时代，并没有多人在线游戏，互联网也没有诞生，网络同步一词更是无人知晓。不过当两台计算机上的数据进行传递时，最最最简单的同步模型就已经悄无声息的出现了，A把操作信息通过电缆发给B，B收到数据后处理，在把自己的操作数据通过电缆发送给A。比较有意思的一点是，计算机技术的发展或多或少都与电子游戏有着紧密的联系，甚至很多技术的诞生就是源于对游戏交互方式的探索。

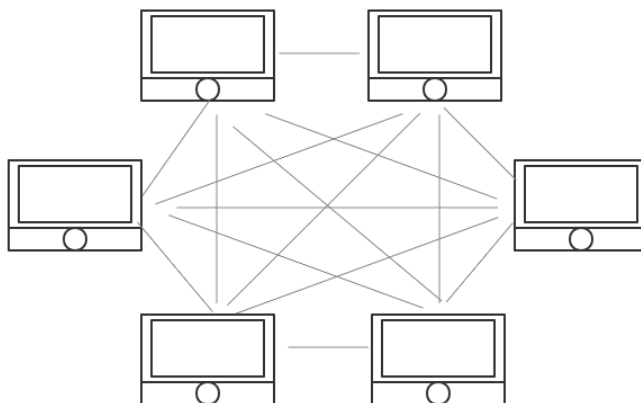


图2-1 P2P架构

1978年，Roy Trubshaw编写了世界上第一个MUD程序《MUD1》，后来又在此基础上诞生了开源的 MudOS（1991），成为众多网游的鼻祖。MUDOS使用单线程无阻塞套接字来服务所有玩家，所有玩家的请求都发到同一个线程去处理，主线程每隔1秒钟更新一次所有对象。这时候所谓的同步，就是把玩家控制台的指令发送到专有的服务器，服务器按顺序处理后再发送给其他所有玩家（几乎没有什么验证逻辑），这是最早的**CS架构**。当时PC图形化还不成熟，MUD早期的系统只有着粗糙的纯文字界面，由于也没有物理引擎等游戏技术，所以对网络延迟、反馈表现要求并不高。

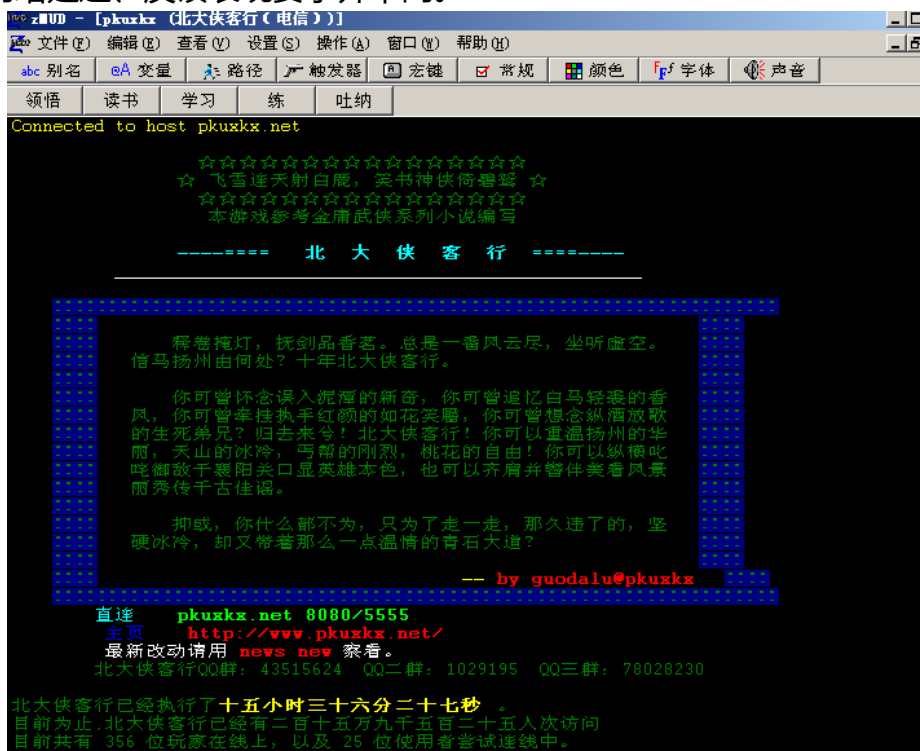


图2-2 国产MUD游戏

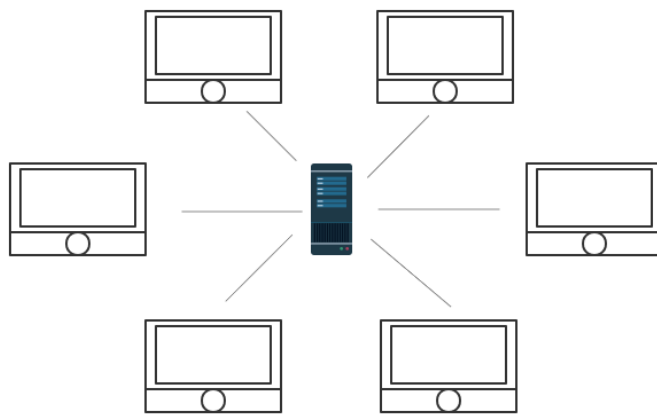


图2-3 早期CS架构

大概在上世纪90年代，在P2P架构的基础上，很自然地诞生了以某个客户端为Host主机（或叫做ListenServer）的CS架构，这样的架构不需要单独都维护一个服务器，任何一个客户端都可以是Sever，能够比较方便的支持局域网内对战，也能节省服务器的运行与开发成本。不过，虽说也是CS架构，如果Host主机不做任何server端的校验逻辑，那么其本质上还是P2P模型，只不过所有的客户端可以把消息统一发送到一个IP，Host再进行转发，这种方式我们称其为Packet Server。

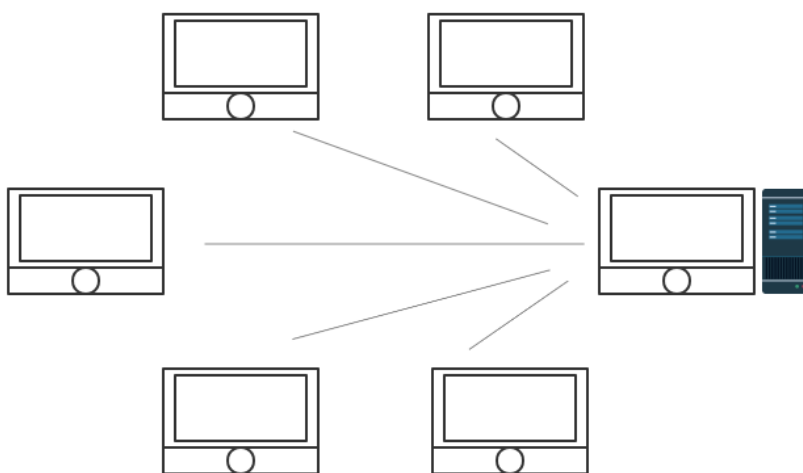


图2-4 带有ListenServer的CS架构

后来一些游戏团队（比如id software）又对CS架构做了进一步调整，先是将大部分的逻辑处理移到服务器上（服务器可能是一个独立的无窗口的后台程序），客户端只负责渲染。随后为了对抗网络延迟提升客户端的流畅性，又把一部分逻辑交回给客户端本地预执行，最终成为很多经典游戏和引擎的架构方式。

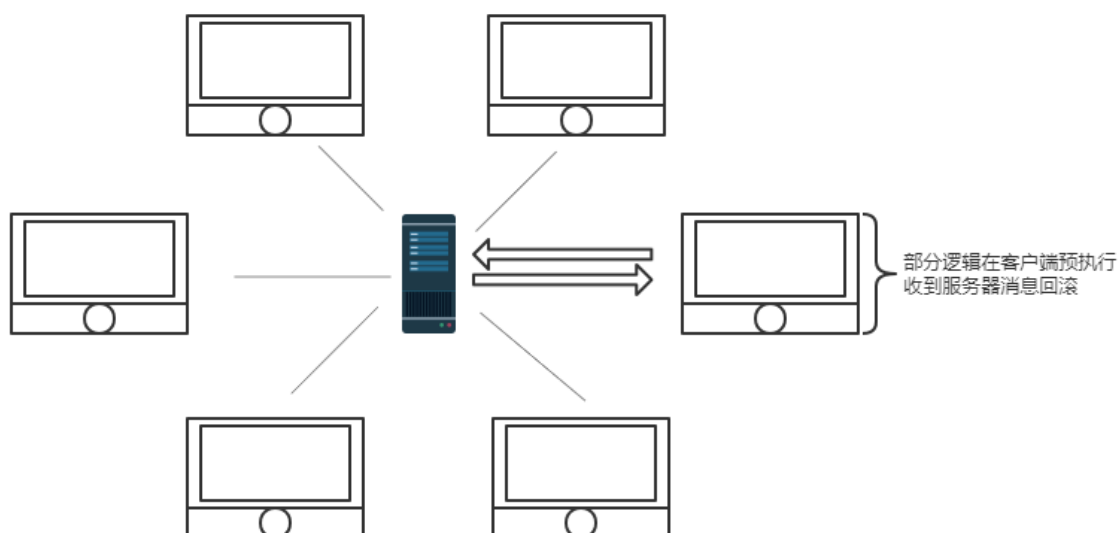


图2-5 客户端与服务器都执行逻辑的CS架构

在2000年后，Eric Cronin的团队在传统多服务器的架构上[2]提出镜像服务器模型[3]。这种模型提供了多个服务器的拷贝，避免单点崩溃影响到所有玩家的问题。类似CDN，玩家还可以选择就近的服务器进行通信，降低了通信延迟。不过，这种方式增加了服务器的租用和维护成本，在后续的游戏网络架构中并没有被大量使用，倒是WEB服务器广泛采用这种模型并不断将其发扬光大。

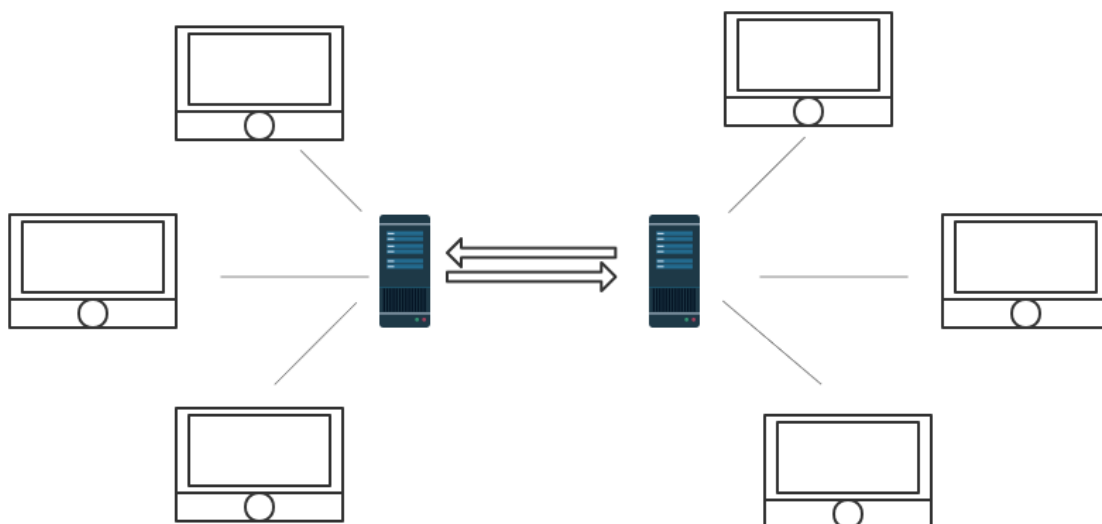


图2-6 镜像服务器架构

再后来，游戏服务器架构不断发展。游戏存储负载和网络连接负载随后从逻辑服上拆分出来，形成独立的服务；玩家数量增多后，又将游戏拆分成多个平行世界，出现了分服和跨服；游戏逻辑进一步复杂后，又开始按照功能去划分成网关服务器、场景服务器、非场景服

务器等。我们今天讨论的网络同步几乎都是在逻辑服务器（基本上无法拆分）上进行的，所以后续的这些架构方式与网络同步的关系并不是很大，这里就不再赘述。

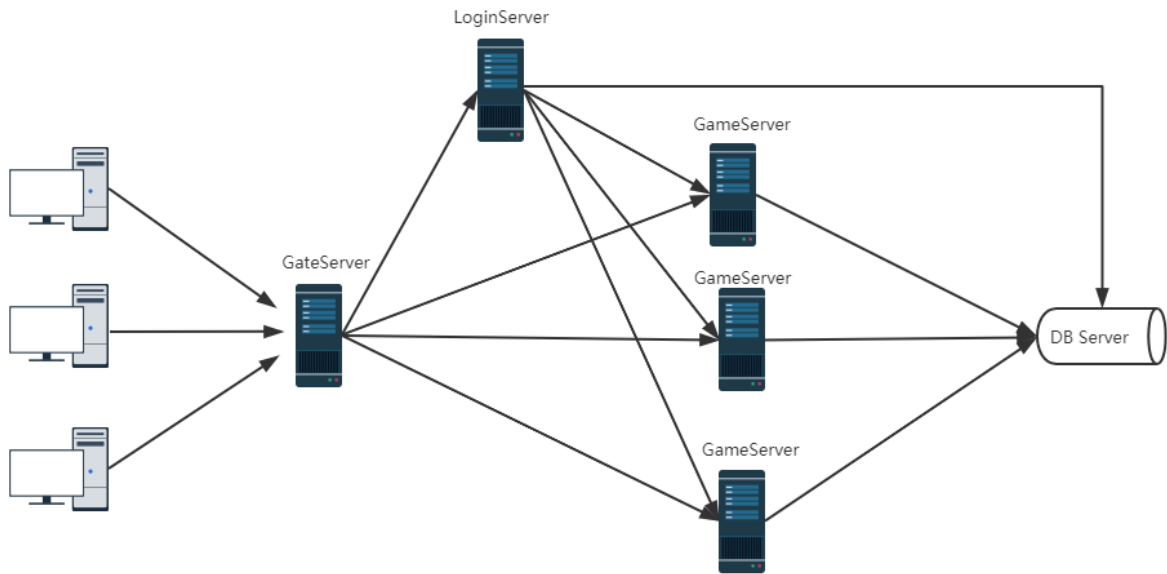


图2-7 分服网络架构

2.传统同步面临的问题

大部分的网络同步技术最早并不是诞生在游戏中，而是出现在各种计算机模拟仿真系统中。在70，80年代就有很多实验室针对Computer Simulation研究各种复杂且有效的同步手段（interactive simulation），我们后面提到的很多游戏同步算法[11]都是由这些早期的研究发展过来的。

网络游戏刚出现的时候，大部分还属于弱交互游戏，可以将其简单理解为一种回合制游戏。在每个回合开始时，所有玩家一同思考并把相关操作指令信息发送给其他玩家，其他玩家收到了别人的消息后就会在本地处理然后结束当前回合，如果没有收到就会进入无限期的等待。由于每个回合有比较长的思考和操作时间，所以网络延迟可以忽略不计，只要保证在回合结束的时候，所有玩家的状态的数据保存一致即可。这种游戏采用的同步方式与计算机网络中的**停等协议**（stop-and-wait-type）非常相似，是一种很自然也很简单的同步模型。不过由于当时网络同步并没有形成体系，所以这种同步方式也没有名字。在局域网盛行以及玩家数量较少的条件下，这种同步方式与架构都是可行的。

然而，随着游戏的种类和玩法复杂性的提升，传统的同步方法面对的问题也接踵而来。

1. 在CS架构下逻辑在客户端执行还是在服务器执行？如果逻辑都在服务器执行，那么客户端的操作都会被发送到服务器运算，服务器计算出结果后通知客户端，客户端拿到结果后再做表现，这样的好处是所有的逻辑由服务器处理和验证，客户端无法作

弊，但坏处是会造成客户端的资源被浪费，服务器运算压力过大。如果逻辑在各个客户端执行，那么玩家可以在本地计算后再把本地得到的结果告知服务器，服务器只进行简单的转发，这样的好处是玩家的本地表现很流畅，但坏处是很容易在本地进行作弊。而对于P2P架构，反作弊更是一个严重的问题，我连一个权威服务器都没有，根本无法验证其他客户端消息的真伪，怎么知道其他玩家有没有作弊？

2. 我们要发送什么数据来进行同步？如果发送每个对象当前的状态，那么如果一个游戏里面有大量的角色，就会大规模的占用网络带宽，造成数据拥塞、丢包等等问题。如果发送玩家指令，那这个指令是要服务器执行还是服务器转发？而且对于大型多人在线游戏又没必要处理所有不相关的玩家信息，同样浪费网络资源。

3. 面对日益成熟的计算机网络协议，我们选择哪种来进行同步？TCP、UDP还是Http？

这时，游戏开发者们需要面对“发什么数据”，“在哪计算”，“发给谁”等细节问题，他们开始考虑引入更多的其他相关领域的技术（比如计算机模拟仿真）来解决游戏中的同步问题，网络同步概念初见端倪。

三、锁步同步 lockstep（帧同步）

Lockstep就是我们口中常说的“帧同步”。但严格来说，Lockstep并不应该翻译成帧同步，而是——锁步同步算法。（LockStep由军事语境引入，用来表示齐步行军，队伍中的所有人都执行一致的动作步伐）首次引入计算机领域[4]，应该是用于计算机容错系统，即“使用相同的、冗余的硬件组件在同一时间内处理相同的指令，从而保持多个CPU、内存精确的同步”，所以一开始与游戏并没有任何关系。

1.早期的Lockstep

不过，早在1994年，FPS鼻祖Doom就已经采用了类似Lockstep的方式进行网络同步[5]。Doom采用P2P架构，每个客户端本地运行着一个独立的系统，该系统每0.02秒钟对玩家的动作（鼠标操作和键盘操作，包括前后移动、使用道具、开火等）采样一次得到一个 tick command 并发送给其他所有玩家，每个玩家都缓存来自其他所有玩家的 tick commands，当某个玩家收到所有其他玩家的 tick commands 后，他的本地游戏状态会推进到下一帧。在这里，tick command的采集与游戏的推进是相互独立的。

其实当时并没有Lockstep这个说法，doom的整篇论文里面也没有出现过这个词。不过后面概念逐渐清晰后我们会发现，Doom采用的同步方式就是我们常说的原始版本的Lockstep——“确定性锁步同步（Deterministic Lockstep）”。

2. Bucket Synchronization

1999年，Christophe Diot和Laurent Gautier的团队开发了一款基于互联网的页游——MiMaze，基于传统的Time Bucket Synchronization[6]他们在发布的论文里面提出了改进后的Bucket Synchronization同步方法[7]。Bucket Synchronization把时间按固定时长划分为多个Bucket，所有的指令都在Bucket里面执行。考虑到网络延迟的情况，每个玩家在本地的命令不会立刻执行而是会推迟一个时延（该时延的长度约等于网络延迟），用来等待其他玩家的Bucket的到来。如果超过延迟没有到达，既可以选择放弃处理，也可以保存起来用于外插值（Extrapolation）或者使用前面的指令重新播放。在这种方式下，每个玩家不需要按照Lockstep的方式严格等待其他玩家的命令在处理，可以根据网络情况顺延到后面的bucket再执行。Bucket Synchronization可以认为是我们常说的“乐观帧锁定”算法。

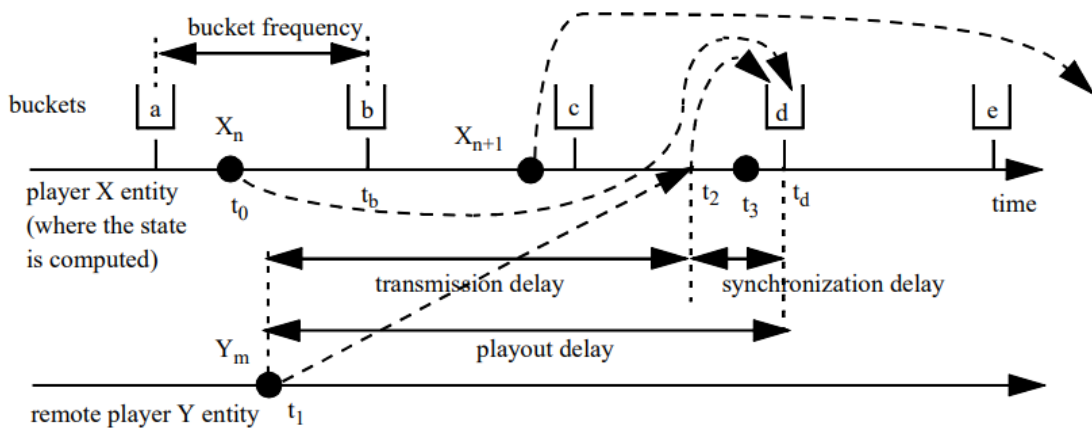


图3-1 Bucket Synchronization同步示意图

3. 锁步同步协议 Lockstep protocol

前面提到的Deterministic Lockstep虽然简单，但是问题却很多，包括浮点数跨平台的同步问题、玩家数量增长带来的带宽问题以及显而易见的作弊问题（在P2P架构下几乎没有任何反作弊能力）。说到作弊这里不妨先简单谈一下游戏外挂，外挂这个东西原本是指为增加程序的额外功能而追加的内容，但随着网络游戏的诞生就开始与游戏绑定起来。针对不同的游戏类型有着各式各样的外挂工具，包括游戏加速、透视、自动瞄准、数据修改等。从技术上来讲，有些外挂是无法做到完全避免的。在CS架构下，因为大部分核心逻辑都是在服务器上面计算，很多作弊手段无法生效。但是P2P架构下作弊却变得异常简单，甚至不需要很复杂的修改工具。比如客户端A使用了外挂工具，每次都将自己的操作信息推迟发送，等到看到了别人的决策后再决定执行什么，这种外挂称为lookahead cheats。（或者假装网络信号不好丢弃第K步的操作，第K+1步再发送）

因此，在2001年，Nathaniel Baughman和Brian Neil Levine在IEEE上发表了论文，提出**锁步同步协议 Lockstep protocol** [8]来对抗lookahead cheat类型的外挂。这可能是第一次“Lockstep”一词被正式的用于游戏网络同步的描述中。不过要注意的是，**这里的 Lockstep protocol并不是我们前面提到的Deterministic Lockstep**，相比之前的在第K步（第K个Tick Command间隔）就直接发送第K+1步的明文操作信息，**Lockstep protocol 每一步都分两次发送信息**。大概的流程如下：

- 先针对要发送的明文信息进行加密，生成“预提交单向哈希（secure one-way commitment hash）”并发送给其他客户端。
- 待本地客户端收到所有其他客户端的第K步预提交哈希值之后，再发送自己第K步的明文信息
- 等到收到所有其他客户端的第K步明文信息后，本地客户端会为所有明文信息逐个生成明文哈希并和预提交的哈希值对比，如果发现XXX客户端的明文哈希值和预提交哈希值不相等，则可以判定该客户端是外挂。反之，游戏正常向前推进。

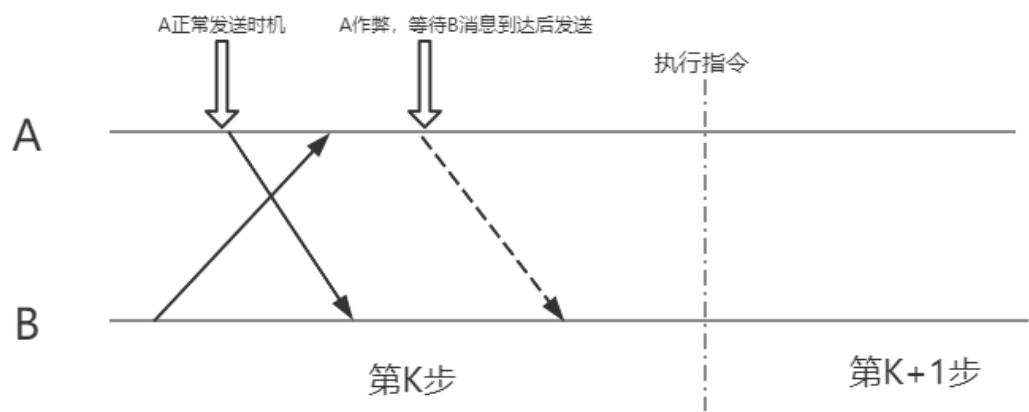


图3-2 Lookahead cheat作弊示意图

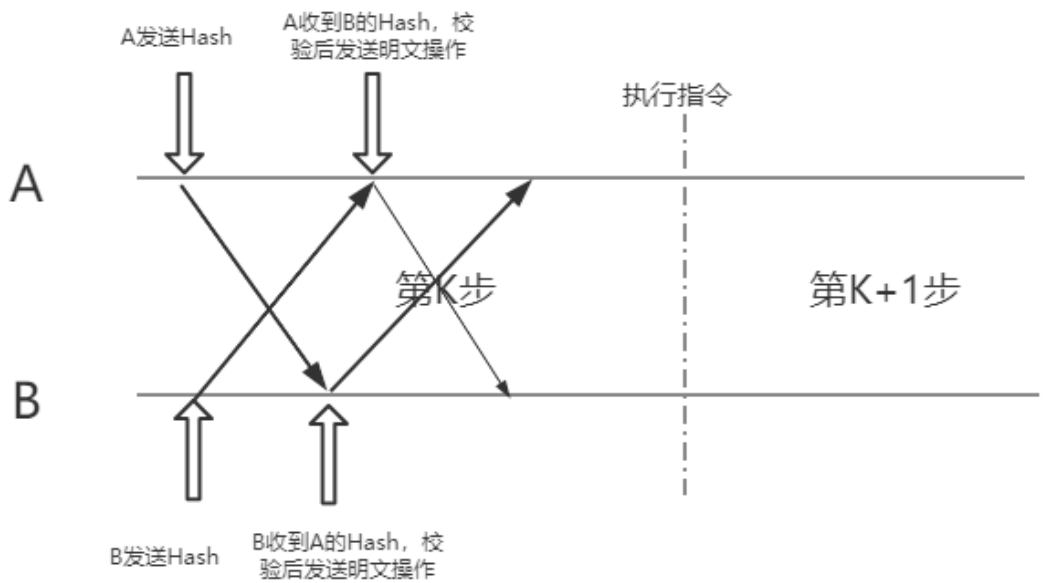


图3-3 Lockstep protocol防作弊流程

这种协议的虽然可以对抗外挂，但是很明显带来了带宽以及性能浪费，而且网络条件好的客户端会时刻受到网络差的客户端的影响。所以他们又在此基础上提出异步的 Lockstep (asynchronous Synchronization lockstep)。大体的思路是利用玩家角色的 SOI (Spheres of Influence, 和AOI概念差不多)，两个玩家如果相距很远互不影响，就采用本地时钟向前推进(非Lockstep方式同步)，如果互相靠近并可能影响到对方就变回到严格的LockStep同步，这里并不保证他们的帧序列是完全一致的。

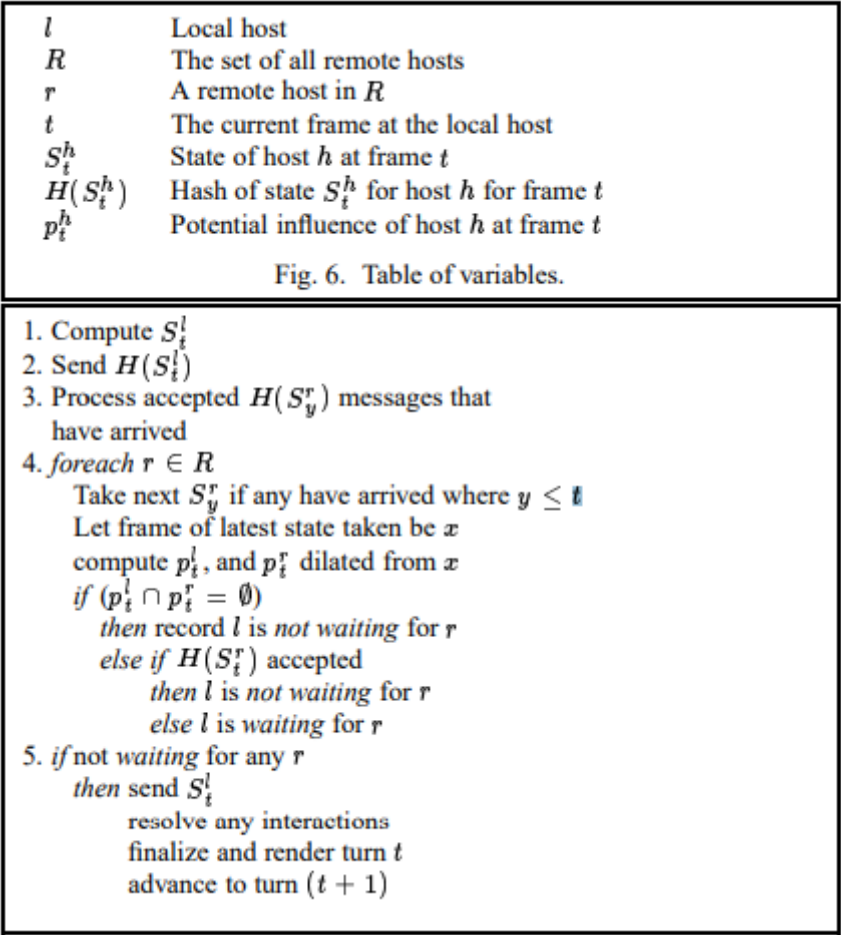


图3-3 Asynchronous Synchronization lockstep流程伪代码

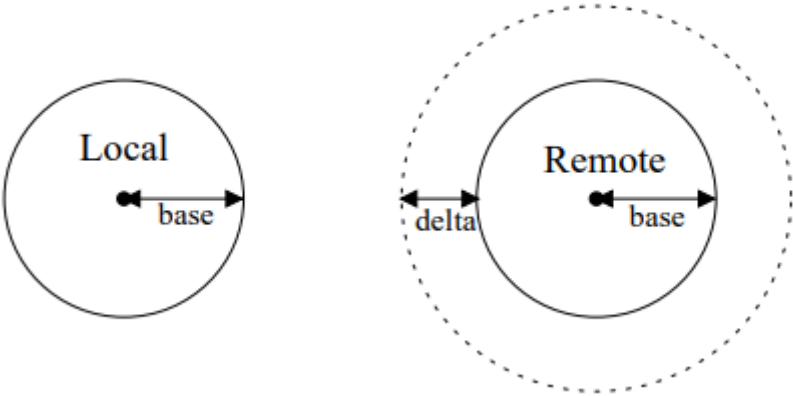


图3-4 SOI示意图

4.RTS中的Lockstep

同一年，2001的GDC大会上，“帝国时代”的开发者Mark Terrano和Paul Bettner针对RTS游戏提出了优化版的锁步协议[9]。早在1996年他们就开始着手于帝国时代1的开发，不过很快就发现在RTS游戏中的网络同步要比其他类型游戏（比如FPS）复杂的多，一是游戏中可能发生位置变化的角色非常多，必须要合理的减少网络同步带宽，二是玩家对同步频率极为敏感，每一秒的疏忽都可能影响局势。所以，他们在传统 Lockstep（当时仍然没有Deterministic Lockstep这个概念）的基础上做了优化，首先保持每一步只同步玩家的操作数据，然后对当前的所有命令延迟两帧执行的方法来对抗延迟。具体来说，就是第K步开始检测到本地命令后会推迟到第K+2步进行发送和执行，K+1步收集到的其他客户端命令会推迟到K+3步去执行，每K步执行前会去判断本地是否有前两步的命令，如果有就继续推进。（关于具体的推进策略，论文里面写的不是很清楚，这里加入了作者自己的判断）

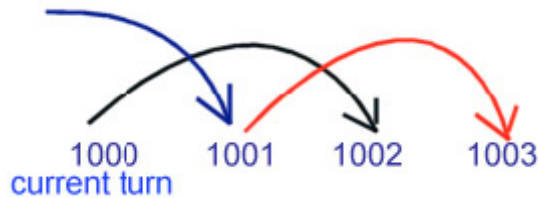
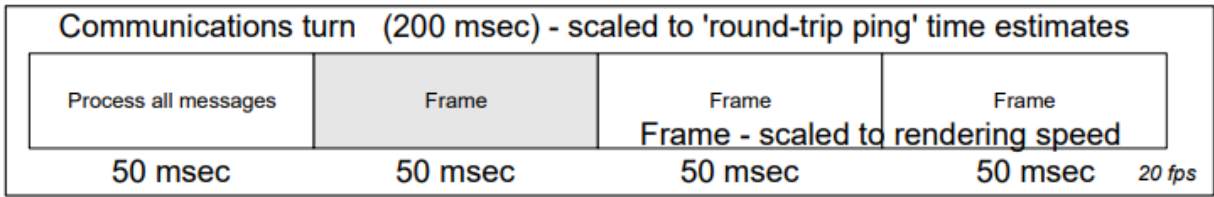


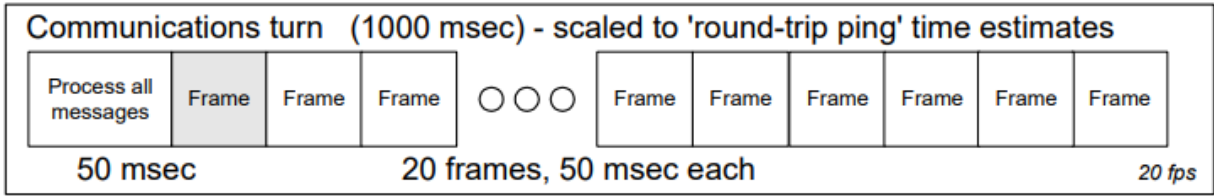
图3-5 延迟执行的Lockstep

此外，为了避免高性能机器受低性能机器的影响而变“卡”，“帝国时代”里面每一步（称为一个turn）的长度是可以调整的，并且完全与渲染分开处理。每个客户端会根据自身的机器性能与网络延迟情况来动态调整步长时间，如果性能优良但是延迟高就会拉长每个turn的时间（由于是非严格的帧锁定，多出的时间用于正常进行多个帧的渲染以及Gameplay的处理，虽然可能有误差），如果性能差但是网络正常就会把大部分的时间用于每个turn的渲染，在这种条件下每个客户端相同的turn执行的本地时间虽然不同，但是执行的内容是完全一致的。

A Single Communication Turn



High Internet Latency with normal machine performance



Poor machine performance with normal latency

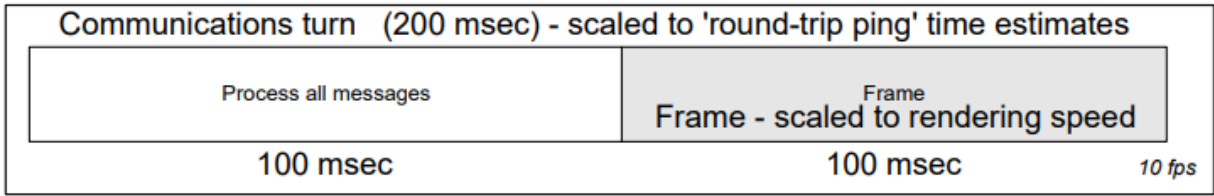


图3-6 根据客户端环境调整的Turn

5.Pipelined Lockstep protocol

流水线操作是一种高效的数据传输模型，在计算机技术里面随处可见（比如计算机存储系统中主存与Cache的交互）。2003年，Ho Lee、Eric Kozlowski等人对Bucket synchronization、Lockstep protocol等协议进一步分析并针对存在的缺点进行优化，提出了Pipelined Lockstep protocol[10]。他们发现只有当前玩家的指令行为不与其他人产生冲突，就可以连续的发送的自己的指令而不需要等待其他人的消息。举个例子，假如一个游戏只有7个格子，玩家A和B分别站在左右两边，每次的指令只能向前移动一格。那么A和B至少可以连续发送三个指令信息而不需要等待对面玩家的数据到来。

Pipelined Lockstep protocol基于Lockstep protocol，为了防止cheatahead外挂同样需要提前发送hash，这种操作同步、不等待超时玩家的确定性锁步的特性逐渐成为“Lockstep”的标准，被广泛应用于网络同步中。

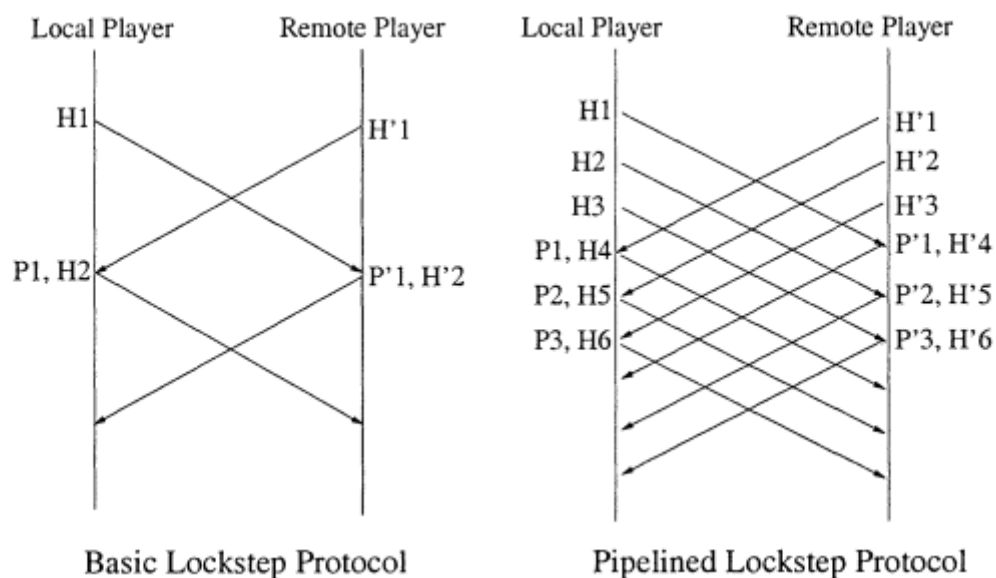


图3-7 流水线Lockstep示意图

6.TimeWarp

TimeWarp原本是指科幻小说中的时间扭曲，其实早在1982年就被D Jefferson等人引入计算机仿真领域[11]，后续又被Jeff S. Steinrnan进行优化和调整[6][12]。TimeWrap算法基本思路是多个物体同时进行模拟，当一个物体收到了一个过去某个时刻应该执行的事件时，他应该回滚到那个时刻的状态，并且回滚前面所有的行为与状态。

前面提到的Pipelined Lockstep protocol可以流畅的处理玩家互相不影响的情况，但是却并没有很好的解决状态冲突与突发的高延迟问题。参考TimeWarp这种思路，我们可以将本地执行过的所有操作指令进行保存同时把对应时刻的游戏世界状态存储成一个快照

(Snapshot)，本地按照Pipelined Lockstep protocol的规则进行推进，如果后期收到了产生冲突的指令，我们可以根据快照回滚到冲突指令的上一个状态，然后把冲突后续执行过的事件全部取消并重新将执行正确的指令。这样如果所有玩家之间没有指令冲突，他们就可以持续且互不影响的向前推进，如果发生冲突则可以按照回退到发生冲突前的状态并重新模拟，保持各个端的状态一致。

7.Lockstep与"帧"同步

前面提到了那么多lockstep的算法，但好像没有一个算法使用到“帧”这个概念。其实“帧同步”可能属于一个翻译上的失误，宽泛一点来讲“帧同步”是指包含各种变形算法的Lockstep，严格来讲就是指最基本的Deterministic Lockstep。我猜测国内在引入这个概念的时候已经是2000年以后（具体时间没有考证），lockstep算法已经有很多变形，时间帧的概念也早已诞生，所以相关译者可能就把“lockstep”翻译成了“帧同步”。当然也可能是引入的时候翻译成了“按帧锁定同步”，后来被大家以简化的方式（帧同步）传递开

来。但不管怎么说，“帧”在实际应用中更多的是指画面渲染的频率，lockstep里面的“step”概念要更宽泛一些才是。

了解游戏的朋友，都知道游戏是有帧率（FPS）的，每一帧都会运行相当复杂的运算（包括逻辑和渲染），如果运算规模达到一定程度就会拉长这一帧的时间，帧率也就会随之下降。所有影视作品的画面都是由一张张图构成的，每一个画面就是一帧，一秒能放多少个画面，就是有多少帧。在游戏里面，渲染器会不停的向渲染目标输出画面，并在帧与帧之间游戏处理各种逻辑，逻辑会改变游戏世界对象的行为，这些行为又会影响游戏画面的变化，这就是游戏的核心框架。早期的lockstep里面渲染和逻辑都是放在一个帧里面去处理的，这样一旦命令受到网络延迟的影响，玩家本地就会卡在一个画面直到消息的到来。为了解决这个问题，有一些游戏（比如帝国时代、王者荣耀）会将逻辑和渲染分开处理（分为逻辑帧和渲染帧），逻辑帧每隔固定的时间（如66ms）去处理所有逻辑事件，而渲染帧可以每15ms执行一次。在不是严格锁帧的情况下，你本地即使没有收到网络数据也可以继续维持高频率的渲染，同时渲染帧执行时可以在不影响玩法的前提下对渲染对象进行插值处理（正在移动的对象不会由于短暂的延迟而静止不动）。这里面的逻辑帧就是lockstep里面的“Step”，也可以叫做“turn”，“bucket”或者“步”。

补充：在2020年中旬，[@知乎韦易笑老师](#)在他的文章《关于“帧同步”说法的由来》中，阐述了有关“帧同步”一词在国内的发展历史也解释清了该名词被乱用的原因。总的来说，国内“帧同步”与国外“Lockstep”的核心思想是一致的。这篇文章中的“帧同步”与“LockStep”在某些场景下可以互相取代。

8.Lockstep小结

了解了Lockstep的发展历程，最后我们再总结梳理一下。网络同步，其目标就是时刻保证多台机器的游戏表现完全一致。由于网络延迟的存在，时刻保持相同是不可能的，所以我们要尽可能在一个回合（turn）内保持一致，如果有人由于某些原因（如网络延迟）没有同步推进，那么我们就需要等他——这就是Lockstep（或者说一种停等协议）[19][20]。

LockStep其实是很朴素的一种思想，很早就被用于计算机仿真模拟、计算机数据同步等领域。后来网络游戏发展起来后，也很自然的被开发者们拿到了系统设计中。

早期lockstep被广泛用于局域网游戏内（延迟基本可以保持在50ms以内），所以这种策略是很有效的。lockstep每个回合的触发，并不是由收到网络包驱动，也不是由渲染帧数驱动（当然渲染帧率稳定的话也可以以帧为单位，每N帧一个回合），而是采用客户端内在的时钟稳定按一定间隔（比如100ms）的心跳前进。游戏的一开始，玩家在本地进行操作，操作指令会被缓存起来。在回合结束前（网络延迟在50ms以内），我们会收到所有其他客户端的指令数据，然后和前面缓存的指令一同执行并推进到下一个回合。如果玩家在一个回合开始到50ms（网络延迟的一半）都没有任何操作，我们可以认为玩家发出了一个Idle指令，打包发给其他客户端[13][14]。

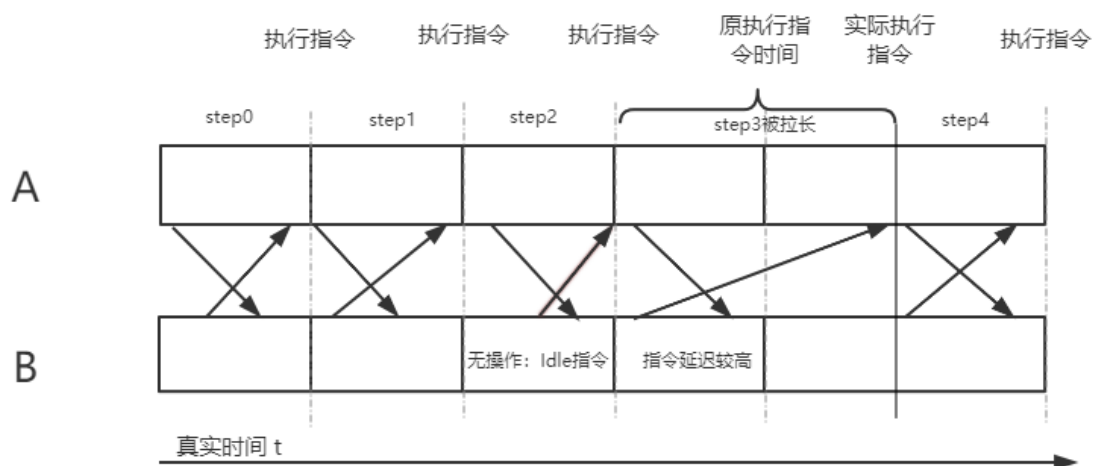


图3-8 传统的Lockstep

换个角度来看，假如一场游戏持续了20分钟，不考虑延迟的情况下整场游戏就是12000个回合（所有客户端都是如此）。现在我们反过去给每个回合添加指令，确保每个回合都收集到所有玩家的指令，那么就可以严格保证所有客户端每个回合的表现都是一样的。假如我们再把这指令都存储起来，那么就推演出整场比赛，这也是为什么lockstep为什么做回放系统很容易。

至于lockstep为什么要发送指令而不是状态，其实是与当时的网络带宽有关。很多游戏都有明确的人数限制（一般不超过10个人），玩家在每个回合能做的操作也有限甚至是不操作，这样的条件下所有玩家的指令一共也占用不了多少带宽。如果换成同步每个角色的状态数据，那么数据量可能会膨胀10倍不止。从原则上说，锁步数据既可以是游戏角色的状态信息也可以是玩家的操作指令，只不过由于各种原因没有采取状态数据罢了。在下一章，我还会对状态同步做进一步的讲解，他与lockstep的发展是相辅相成的，也不是网上常说的那种对立关系。

```

1  //lockstep操作指令的结构体
2  struct Input
3  {
4      bool up;
5      bool down;
6      bool left;
7      bool right;
8      bool space;
9      bool Z;
10 };

```

仔细分析一下lockstep，其实就能发现不少缺点。首当其冲的就是网络条件较差的客户端很容易影响其他玩家的游戏体验。为了尽可能保证所有玩家的游戏体验，开发者们不断的对游戏进行更深一步的分析，先后提出了各种优化手段和算法，包括使用乐观帧锁定，把渲染与同步逻辑拆开，客户端预执行，将指令流水线化，操作回滚等。关于回滚还有很多细节[16][18]，我会在下个章节里面进行更详细的阐述。

其次，lockstep的另一个问题就是很难保证命令一致的情况下，所有客户端的计算结果完全一致，只要任何一个回出现了一点点的误差就可能像蝴蝶效应一样导致两个客户端后面的结果截然不同。这个问题听起来容易，实际上执行起来却有很多容易被忽略的细节，比如RPC的时序，浮点数计算的偏差，容器排序的不确定性，随机数值计算不统一等等。浮点数的计算在不同硬件（跨平台更是如此）上很难保持一致的，可以考虑转换为定点数，随机计算保持各个端的随机种子一定也可以解决，但是具体实现起来可能还有很多问题，需要踩坑之后才能真正解决。

四、状态同步 State Synchronization

上一篇我们曾提到，国内对“Lockstep”的翻译经常会引起一些初学者的误解。但好在网上的参考文献和资料比较丰富的，查证起来还比较容易。反倒是当我开始仔细研究大家都耳熟能详的“状态同步”时[21]，我竟然一时半会难以追溯其发展来源。可能是因为早期的技术领域里面好像并没有这个概念，笔者能找到最早的关于状态同步的资料源于上世界90年代的防火墙产品——FireWalls-1[22][23][24]。这款产品出自公司Check Point，是第一个使用状态检测（stateful inspection）的商业防火墙软件。其中的stateful inspection表示让多个防火墙共享各自状态表中包含的信息，这种信息传递的方式与我们游戏中的状态同步非常相似。由此可见，状态同步与Lockstep一样，也是经历了一个相对漫长的时间才发展成如今我们熟知的模样。



图4-1 checkpoint防火墙软件

在二十年前，相比于使用帧同步（为了方便描述，后续的文章中以帧同步代替Lockstep）还是状态同步，开发者们更关心的是网络架构的实现方式（P2P/CS）。换句话讲，在当时业内看来，P2P架构的同步模型虽然减少了延迟，但由于作弊、跨平台、难以维护大型网络游戏等问题，人们更希望用CS架构来取代P2P。同时，开发者们虽然可以继续CS架构下使用逻辑比较简洁的帧同步，但有不少开发者都认为刚刚诞生的状态同步貌似更符合CS架构的同步理念。

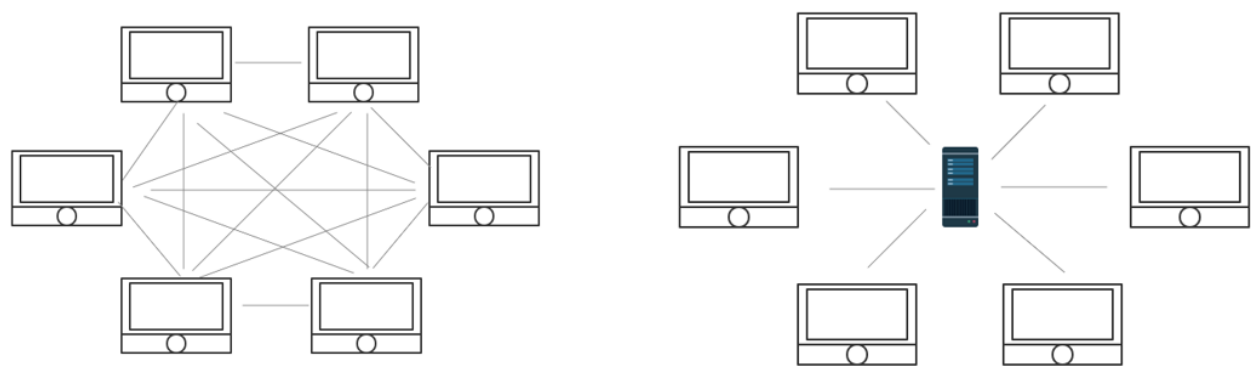


图4-2 P2P架构与CS架构对比图

需要强调的是，帧同步与状态同步并不是一个简单的对立概念，其中的差异包括“数据格式与内容”，“逻辑的计算位置”，“是否有权权威服务器”等。随着时间的推进，两种算法互相借鉴互相发展，早已不是当年的样子。网上存在很多概念模糊的文章，包括一些大佬对同步的概念理解也有偏差，这些都很容易对新手产生误导。所以笔者建议，如果你想真正的了解或者学习网络同步，不妨跟着这篇文章去了解二者的发展历史，相信看过后的你一定更能深刻的理解到帧同步与状态同步的异同。（文末同上篇一样贴出了大量的文献内容）

1.雷神之锤与快照同步（Quake and Snapshot）

快照是一个通用的行业术语，即在任何给定时刻记录设备的状态并在设备出现故障时进行还原。快照技术常用于计算机的各种存储系统，例如逻辑卷管理、数据库、文件系统等。在游戏领域中，快照的含义更像是照片一样，将当前场景所有的信息保存起来。严格来说，快照同步应该属于状态同步的前身，虽然思想相似但是具体实现却有不小的差异。

1996年，在doom发行不久后，Id software就公开了新作——雷神之锤（Quake）。在Quake里他们舍弃了之前的P2P而改用CS架构，同时也舍弃了lockstep的同步方式。新的架构下，客户端就是一个纯粹的渲染器(称为Dumb Client)，每一帧玩家所有的操作信息都会被收集并发送到服务器，然后服务器将计算后的结果压缩后发给客户端来告知他们有哪些角色可以显示，显示在什么位置上。



图4-3 游戏Quake

上述的这个过程就是我们所说的快照同步，即服务器每帧接受客户端的输入来计算整个世界的状态，然后将结果快照发送给所有客户端。Quake这里所谓的快照，就是把整个游戏世界里面所有对象的状态做一次临时保存（他更强调的是对象的可视化状态，比如位置和旋转等）。通过这个快照，我们可以还原出这一刻世界的状态应该是什么样子的。

Quake运行时，逻辑帧率与渲染帧率是保持一致的。由于所有的核心逻辑都是在服务器进行，所以也不需要通过锁步来避免客户端不同步的问题，只要在收到服务器消息后执行渲染就好了。当然，对于性能以及网络环境较差的玩家来说，游戏体验仍然很糟糕。因为你按下一个按钮后，可能很长时间都没有反应，当收到服务器的快照消息后，你可能已经被网络好的玩家击杀了。

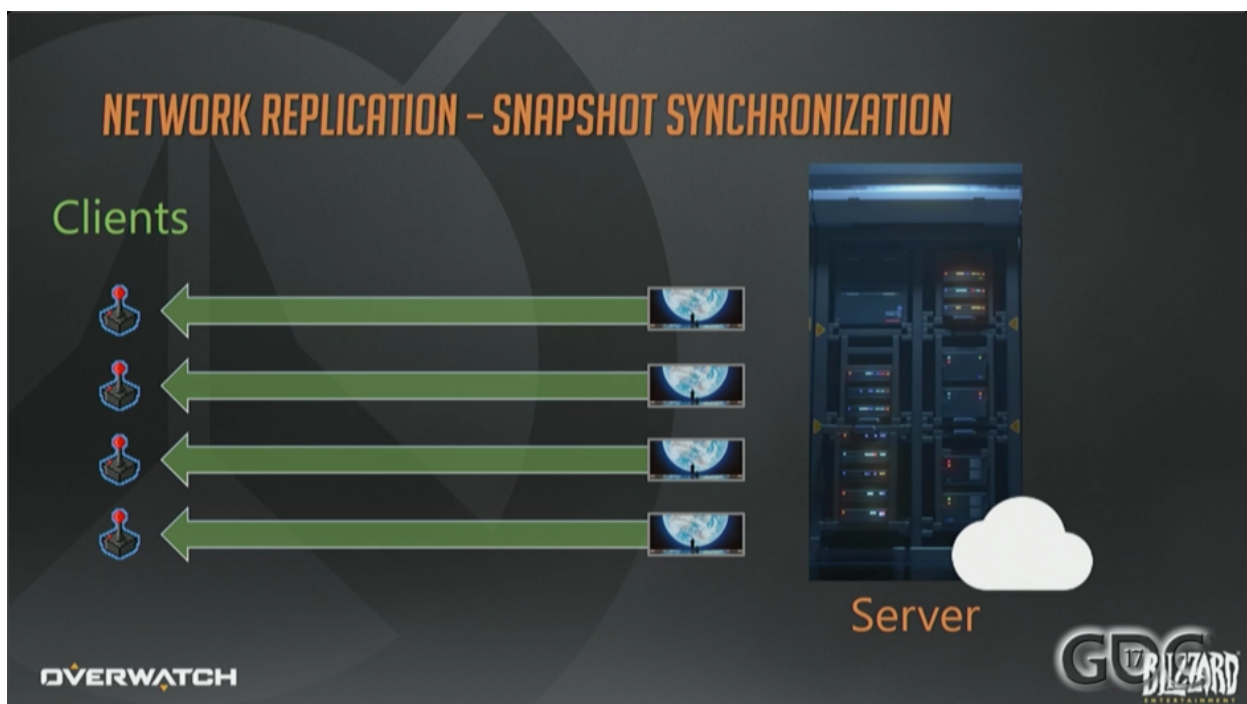


图4-4 守望先锋中对快照同步的解释

2. 《星际围城：部落》引擎中的网络架构

(The TRIBES Engine Networking Model)

IdSoftware自2012年以来已经陆续把Quake以及Doom相关的源码上传到了GitHub上面[25]。如果你看过其中Quake的源码，会发现整个网络的架构还是比较简单清晰的，博主FABIEN SANGULARD就在网上分享了关于Quake源码的剖析[26]（还有很多其他项目的源码剖析）。

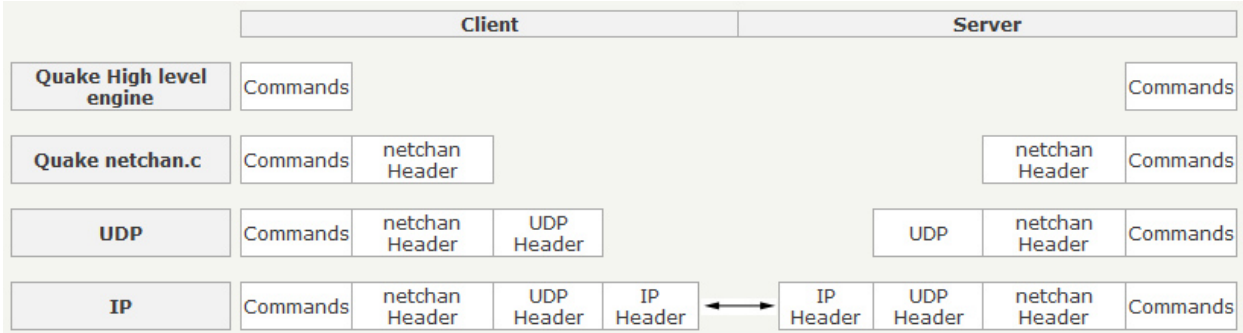


图4-5 Quake的网络通信数据结构

```
1 //client
2 WinMain
3 {
4     while (1)
5     {
6         newtime = Sys_DoubleTime ();
7         time = newtime - oldtime;
8         Host_Frame (time)
9         {
10             setjmp
11             Sys_SendKeyEvents
12             IN_Commands
13             Cbuf_Execute
14             /* Network */
15             CL_ReadPackets
16             CL_SendCmd
17
18             /* Prediction//Collision */
19             CL_SetUpPlayerPrediction(false)
20             CL_PredictMove
21             CL_SetUpPlayerPrediction(true)
22             CL_EmitEntities
23
24             /* Rendition */
```

```

25  SCR_UpdateScreen
26  }
27  oldtime = newtime;
28  }
29  }

```

但Quake里面由于客户端只是一个简单的渲染器，同步过程中会出现很多明显的问题，比如延迟过大，客户端性能浪费，服务器压力大等。而其中最明显的问题就是对带宽的浪费，对于一个物体和角色比较少的游戏，可以使用快照将整个世界的状态都存储并发送，但是一旦物体数量多了起来，带宽占用就会直线上升。所以，我们希望不要每帧都把整个世界的的数据都发过去，而是只发送那些产生变化的对象数据（可以称为增量快照同步）。更进一步的，我们还希望将数据拆分的更细一些，并根据客户端的特点来定制发送不同的数据。基于这种思想，《星际部落：围攻》团队的开发者们开始对网络架构进行抽象和分层，构造出来一套比较完善的"状态同步"系统并以此开发出了Tribe游戏系列。

The TRIBES Engine可以认为是第一个实现状态同步的游戏引擎，《星际部落：围攻》也可以认为是第一个比较完美的实现了状态同步的游戏。

下图是该引擎的网络架构[27]：

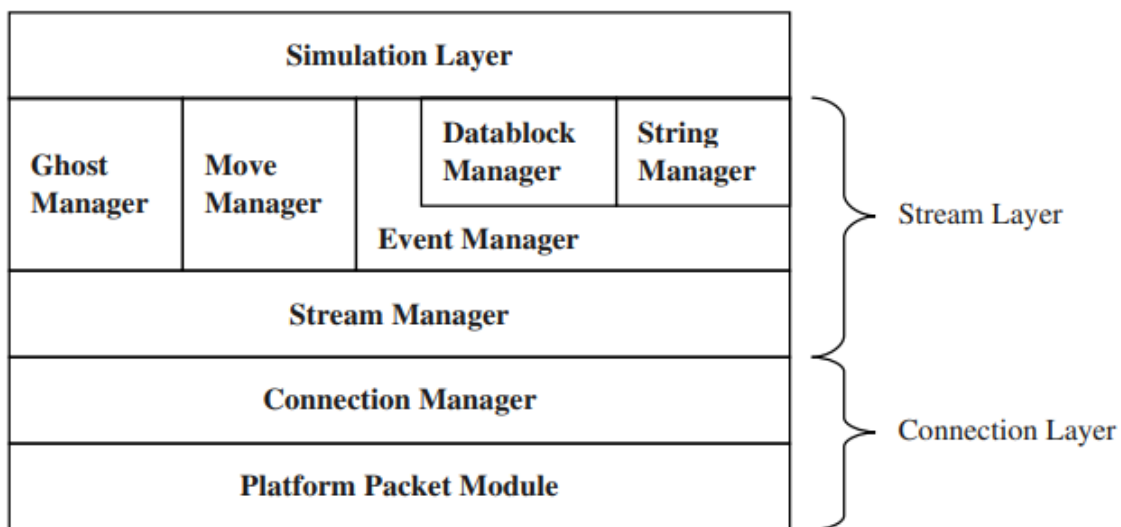


图4-6 Tribe网络架构抽象图

平台数据包模块（Platform Packet Module）可以理解成被封装的Socket模块，连接管理器（Connection Manager）处理多个客户端与服务器的连接，流管理器（Stream Manager）负责将具体的数据分发到上面的三个高级管理器。

- Ghost管理器：负责向客户端发送需要同步对象的状态信息，类似属性同步。
- 事件管理器：维护事件队列，每个事件相当于一个的RPC。

- 移动管理器：本质上与事件管理器相同，但是由于移动数据的需要高频的捕捉和发送，所以单独封装成一个特殊的管理器。

3.客户端预测与回滚 (Client-side prediction and Rollback)

《毁灭公爵》是上世纪90年代一个经典的FPS游戏系列，首部作品的发布时间与Doom几乎相同，网络架构也极为相似。在1996年发布的《毁灭公爵3D》里面，为了提高客户端的表现与响应速度，他放弃了“Dumb客户端”的方案并首次采用客户端预测来进行优化（这里主要指移动预测）[28]。即在服务器确认输入并更新游戏状态之前，让客户端立即对用户输入进行本地响应。由于这种方式可以大大的降低网络延迟所带来的困扰，很快的Quake也开始参考对网络架构进行的大刀阔斧的修改。在1997年发布的更新版本QuakeWorld里面[29][30]，Quake添加了对互联网对战的支持以及**客户端预测**等新的内容。

关于预测，其实就是本地先执行，所以并不需要什么特别的算法，反倒是预测后的客户端与服务器的同步处理有很多值得优化的地方。由于玩家的行为是没办法完全预测的，所以你不知道玩家会在什么时候突然停下或者转弯，所以经常会发生预测失败的情况。

如果玩家本地的预测结果与服务器几乎一致，那么我们认为预测是成功且有效的，玩家不会受到任何影响，可以继续操作。反之，如果客户端结果与服务器不一致，我们应该如何处理呢？这里分为两种情况。

一.在没有时间戳的条件下，收到了一条过时的服务器位置数据。你在本地的行为相比服务器是超前的，假如你在time=10ms的和time=50ms时候分别发送了一条指令。由于网络延迟的存在，当你已经执行完第二个指令的时候才收到服务器对第一条指令的位置同步。很明显，我们不应该让过时的服务器数据来纠正你当前的逻辑。解决方法就是在每个指令发出的时候带上他的时间戳，这样客户端收到服务器反馈的时候就知道他处理的是哪条指令信息。

二.假如我们在指令里面添加了时间戳的信息，并收到了一条过时的服务器位置数据。在上一篇文章里我们提到了TimeWarp算法，即当一个对象收到了一个过去某个时刻应该执行的事件时，他应该回滚到那个时刻的状态，并且回滚前面所有的行为与状态（包括取消之前行为所产生的事件）。这个时候我们可以用类似的方法在本地进行纠正，大体的方案就是把玩家本地预执行的指令都记录好时间戳并存放到一个MOVE BUFFER列表里（类似一个滑动窗口）。如果服务器的计算结果与你本地预测相同，可以回复你一个ACKMOVE。如果服务器发现你的某个移动位置有问题时，会把该指令的时间戳以及正确的位置打包发给你。当你收到ACKMOVE的时候，你可以把MOVE BUFFER里面的数据从表里面移除，而当你收到错误纠正信息时就需要本地回滚到服务器指定的位置同时把错误时刻后面MOVE BUFFER里面的指令重新执行一遍。这里读者可能会产生一个疑问——为什么不直接拉回？因为这时候他

想纠正的是之前的错误而不是现在的错误，如果简单的拉回就会让你觉得被莫名其妙的拉回到以前的一个位置。同时，考虑到已经在路上的指令以及后续你要发送的预测指令，会让服务器后续的校验与纠正变得复杂且奇怪，具体流程细节可以参考下图。另外，Gabriel Gambetta博主在他的文章中，也对这种情况进行了简单的分析[31]。

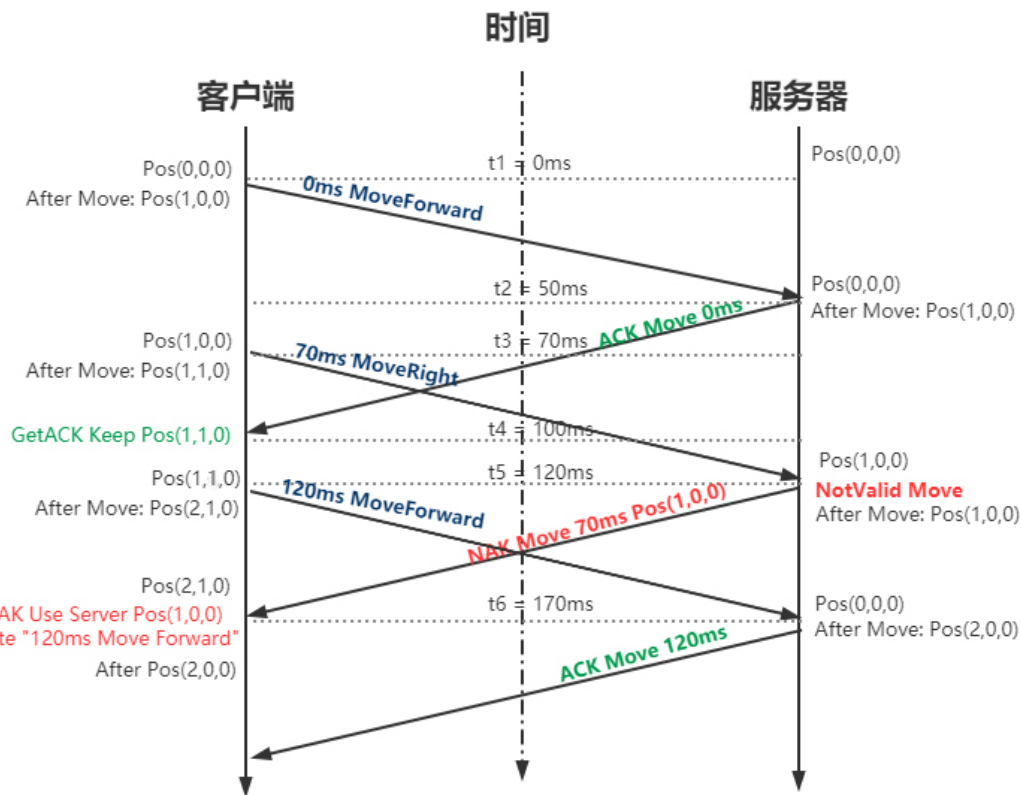


图4-7 预测回滚示例

关于TimeWarp算法的补充：Timewarp技术最早出现于仿真模拟中[32]，我们可以认为这些仿真程序中采用的是“以事件驱动的帧同步”。也就是说，给出一个指令，他就会产生并触发多个事件，这些事件可能进而触发更多的事件来驱动程序，同理取消一个过去发生的事件也需要产生一个新的取消事件才行。这样造成的问题就是回滚前面的N个操作，就需要产生N个新的对抗事件，而且这N个事件还需要发送到所有其他的客户端执行。如果这N个事件又产生了新的事件，那么整个回滚的操作就显得复杂了很多。换成前面移动的例子来解释一下，就是客户端收到服务器的纠正后，他会立刻发送回滚命令告诉（P2P架构下）所有其他客户端，我要取消前面的操作，然后其他客户端在本地也执行回滚。而在如今的CS架构状态同步的方式下，服务器可能早就拒绝了客户端的不合法行为，所以并不需要处理回滚（同理，其他客户端也是）。所以严格来说，TimeWarp技术以及优化后的BreathTimeWarp技术[33]都是针对“以事件驱动的帧同步”，并不能与预测回滚这套方案完全等价。当然，随着时间的推移，很多概念也变的逐渐宽泛一些，我们平时提到的时间回溯TimeWarp技术大体上与快照回滚是一个意思的。

4.事件锁定与时钟同步(Event Locking and Clock Synchronization)

1997年，Jim Greer与Zack Booth Simpson在开发出了他们第一款基于CS架构的RTS游戏——“NetStorm: Island at war”。随后在发布的文章中又提出了“事件锁定”这一概念[34]，相比帧同步会受到其他客户端延迟的影响，事件锁定是基于事件队列严格按序执行的，客户端只管发消息然后等待服务器的响应即可，其他时候本地正常模拟，不需要等待。在目前常见的游戏中，我们很少会听说到事件锁定这种同步方式，因为事件锁定的本质就是通过RPC产生事件从而进行同步（也就是排除属性同步的状态同步）。事件锁定在CS架构上是非常自然的，相比帧同步，可以定义并发送更灵活的信息，也不必再担心作弊的问题。

不过，由于事件中经常会含有时间相关的信息（比如在X秒进行开火）以及服务器需要对客户端的不合法操作进行纠正，所以我们需要尽可能的保持客户端与服务器的时钟同步。实现时钟同步最常见且广泛的方式就是网络时间协议（Network Time Protocol，简称NTP）[35]，NTP属于应用层协议下层采用UDP实现，1979年诞生以来至今仍被应用在多个计算机领域里，包括嵌入式系统时间、通信计费、Windows时间服务以及部分游戏等。NTP使用了一种树状、半分层的网络结构来部署时钟服务器，每个UDP数据包内包含多个时间戳以及一些标记信息用来多次校验与分析，

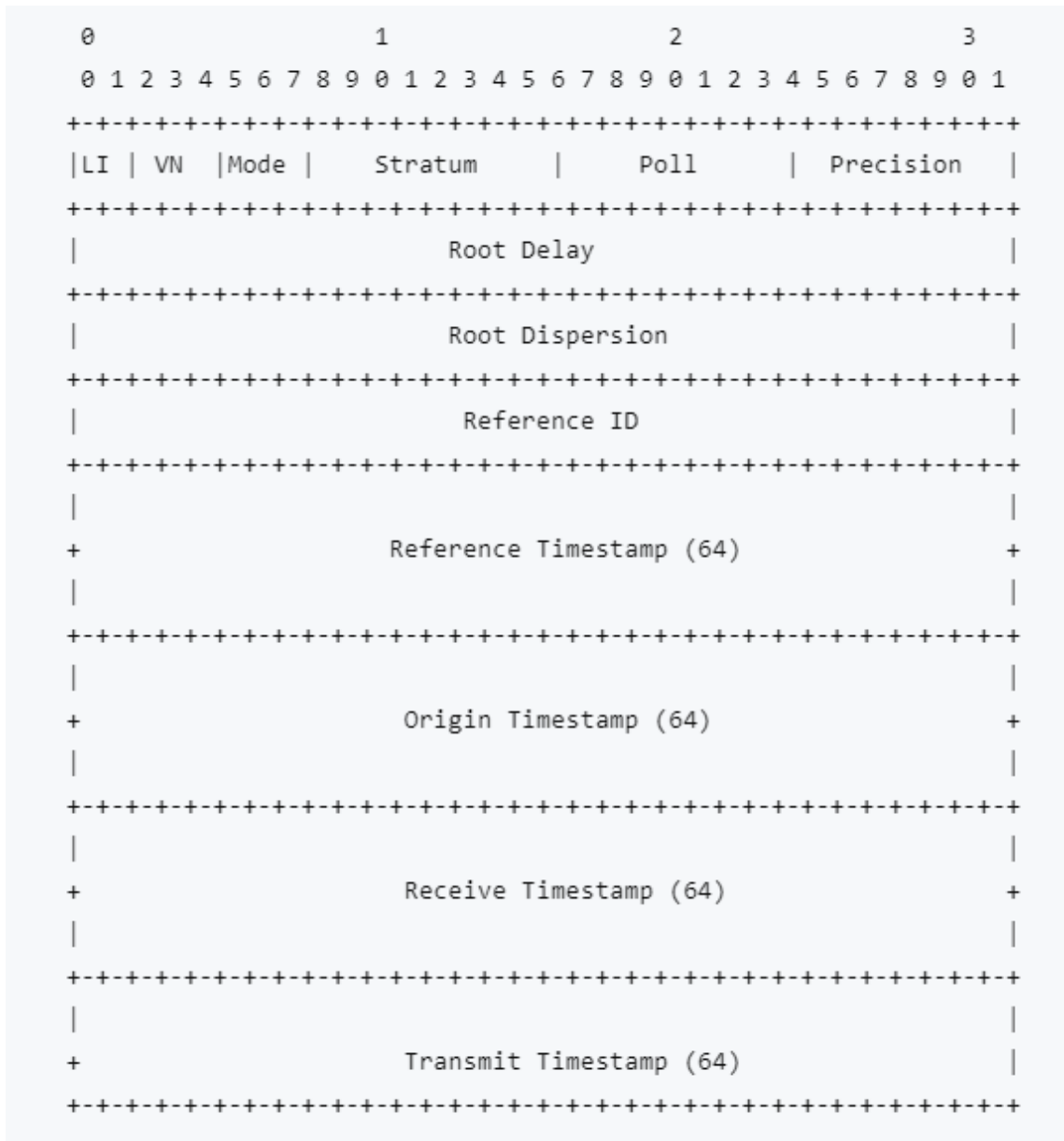


图4-8 NTP协议数据结构

整个时钟同步的具体算法涉及到非常多的细节，我们这里只考虑他的时钟同步算法（其他的内容请参考历年的RFC）：

假如一个服务器与客户端通信，客户端在 t_0 向服务器发送数据，服务器在 t_1 收到数据， t_2 响应并回包给客户端，最后客户端在 t_3 时间收到了服务器的数据。

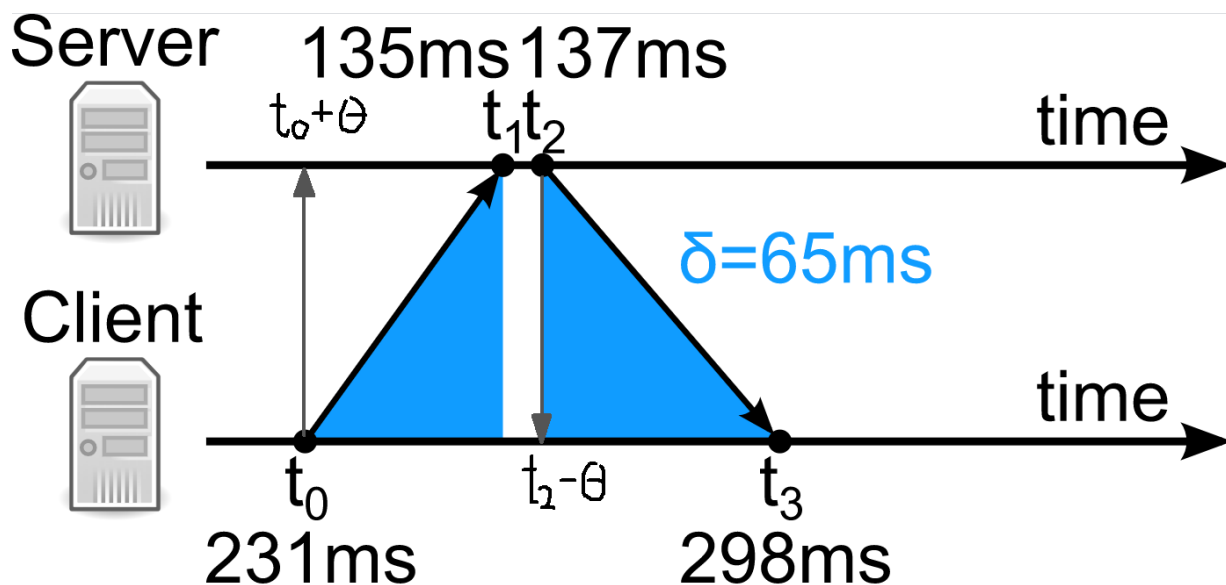


图4-9 时钟同步算法核心逻辑图

二者的时间差为 “ θ ”，假如往返延迟相同，则有

$$t_1 - (t_0 + \theta) = t_3 - (t_2 - \theta)$$

所以可以将 “ θ ” 定义为

$$\theta = \frac{(t_1 - t_0) + (t_2 - t_3)}{2}$$

将往返延迟相加，那么可以得到一个RTT延迟

$$\delta = (t_3 - t_0) - (t_2 - t_1)$$

当然，该操作不会只执行一次，客户端会同时请求多个服务器，然后对结果进行统计分析、过滤，并从最好的三个剩余候选中估算时间差，然后调整时钟频率来逐渐减小偏移。如果我们的系统对精度要求不是非常高，我们还可以使用简化版的SNTP(Simple Network Time Protocol)，时钟同步算法与NTP是相同的，不过简化了一些流程。

不过无论是NTP还是SNTP，对于游戏来说都过于复杂（而且只能用UDP实现）。因此Jim Greer等人提出了“消除高阶的流式时间同步”，流程如下：

1. 客户端把当前本地时间附在一个时间请求数据包上，然后发送给服务器
2. 服务器收到以后，服务器附上服务器时间戳然后发回给客户端

3. 客户端收到之后，用当前时间减去发送时间除以2得到延迟。再用当前时间减去服务器时间得到客户端和服务端时间差，再加上半个延迟得到正确的时钟差异 $\text{delta} = (\text{Currenttime} - \text{senttime})/2$
4. 第一个结果应该立刻被用于更新时钟，可以保证本地时间和服务器时间大致一致
5. 客户端重复步骤1至3多次，每次间隔几秒钟。期间可以继续发送其他数据包的，但是为了结果精确应该尽量少发
6. 每个包的时间差存储起来并排序，然后取中位数作为中间值
7. 丢弃和中间值偏差过大（超出一个标准偏差，或者 超过中间值1.5倍）的样例，然后对剩余样例取算术平均

上述算法精髓在于**丢弃和中间值偏差超过一个标准偏差的数值**。其目的是为了去除TCP中重传的数据包。举例来说，如果通过TCP发送了10个数据包，而且没有重传。这时延迟数据将集中在延迟的中位数附近。假如另一个测试中，如果其中第10个数据包被重传了，重传将导致这次的采样在延迟柱状图中极右端，处于延迟中位数两倍的位置。通过直接去掉超出中位数一个标准偏差的样例，可以过滤掉因重传导致的不准确样例。（排除网络很差重传频繁发生的情况）

5.插值技术 (Interpolation and Extrapolation)

插值技术在早期的帧同步就被应用到游戏里面了。或者说更早的时候就被应用到军事模拟，路径导航等场景中。插值分为内插值[36]（ interpolation ）以及外插值[37]

（extrapolation，或者叫外推法）两种。内插值是一种通过已知的、离散的数据点，在范围内推求新数据点的方法（重建连续的数据信息），常见于各种信号处理和图像处理。在这篇文章中，我们指**根据已知的离散点在一定时间内按照一定算法去模拟在点间的移动路径**。内插值具体的实现方法有很多，如

- 片段插值 (Piecewise constant interpolation)
- 线性插值 (Linear interpolation)
- 多项式插值 (Polynomial interpolation)
- 样条曲线插值 (Spline interpolation)
- 三角内插法 (trigonometric interpolation)
- 有理内插 (rational interpolation)
- 小波内插 (wavelets interpolation)

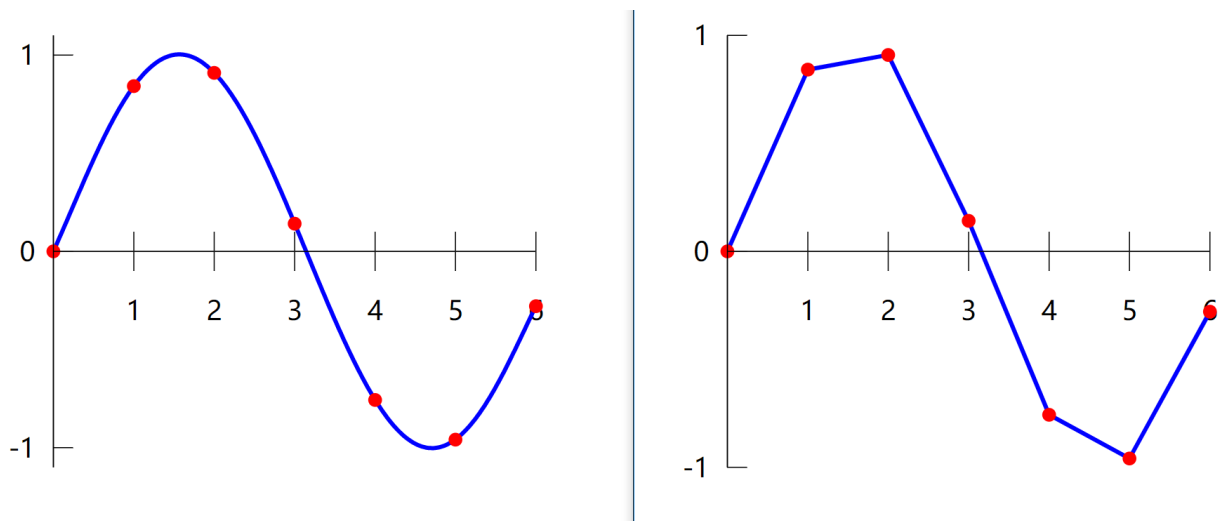


图4-10 多项式插值与线性插值对比

外插值，指从已知数据的离散集合中构建超出原始范围的新数据的方法，也可以指根据过去和现在的发展趋势来推断未来，属于统计学上的概念。与外插值还有一个相似的概念称为DeadReckoning（简称DR），即导航推测。DR是一种利用现在物体位置及速度推定未来位置方向的航海技术，属于应用技术方向的概念。DR的概念更贴近游戏领域，即给定一个点以及当前的方向等信息，推测其之后的移动路径，外推的算法也有很多种，

- 线性外推（Linear extrapolation）
- 多项式外推（Polynomial extrapolation）
- 锥形外推（Conic extrapolation）
- 云形外推（French curve extrapolation）

在游戏中，一般按照线性外推或匀变速直线运动推测即可。不过，对于比较复杂的游戏类型，我们也可以采用三次贝塞尔曲线、向心Catmull-Rom曲线等模拟预测。

$$P_t = P_0 + V_0 T + \frac{1}{2} A_0 T^2$$

总之，无论是内插值还是外插值，考虑到运算的复杂度以及表现要求，游戏中以线性插值、简单的多项式插值为主。

插值技术的应用：

早期的lockstep算法中，在一个客户端在收到下一帧信息前，为了避免本地其他角色静止卡顿，会采用外插值来推断其接下来一小段时间的移动路径[38][39]。普通DR存在一个问题（参考下图），t0时刻其他客户端收到了主机的同步信息预测向虚线的方向移动，不过主机客户端却开始向红色路径方向移动，等其他客户端在t1时刻收到同步信息后会被突然拉倒

t_1' 的位置，这造成了玩家不好的游戏体验。为了解决从预测位置拉扯到真实位置造成的视觉突变，我们会增加一些相应的算法来将预测对象平滑地移动到真实位置。

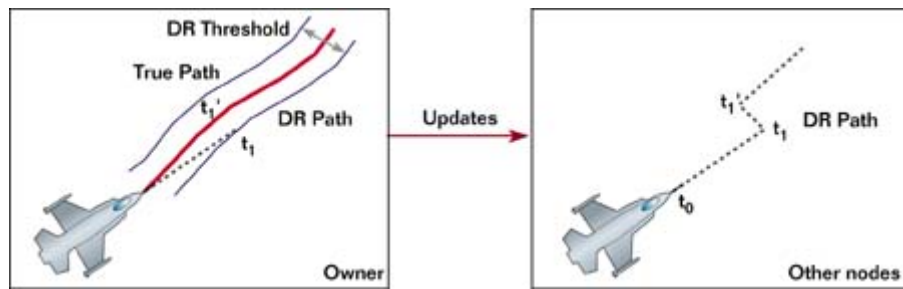


图4-11 DR的问题

在状态同步中，由于客户端每次收到的是其他的角色的位置信息，为了避免位置突变，本地会采用内插值来从A点过度到B点。插值的目的很简单，就是为了保证在同步数据到来之前让本地的角色能有流畅的表现。

6.延迟补偿 (Lag Compensation)

2001年，Valve的新作《半条命》发布，打破了传统FPS游戏玩法。不久之后，其Mod《反恐精英》更是火遍了全球并作为独立游戏发布出去。

由于半条命是基于“Quakell引擎修改的GoldSrc引擎”开发，所以游戏同样采用了CS架构以及状态同步。不过，为了能达到他们心中理想的效果，半条命在网络同步上做出了不小的改动。首先，半条命也采用了客户端预测逻辑来保证本地玩家能够有流畅的手感，同时为了让客户端提高预测准确率（保证客户端与服务器上的代码逻辑一致），所以半条命里面他们让客户端与服务器执行的是同一套代码。其次，考虑到本地玩家的时间总是领先服务器，玩家开枪的时间到服务器执行时就一定会被延迟，所以为了尽量减小延迟所带来的问题，他们提出了一种名为延迟补偿的技术。

所谓延迟补偿[40]，就是弥补客户端到服务器同步延迟的一项技术，该技术的核心是**服务器**在指定时刻对玩家角色进行位置的回滚与计算处理。假如客户端到服务器的延迟为 Xms 。当客户端玩家开枪时，这个操作同步会在 Xms 后到达服务器，服务器这时候计算命中就已经出现了延迟。为了得到更准确的结果，服务器会在定时记录所有玩家的位置，当收到一个客户端开枪事件后，他会立刻把所有玩家回退到 Xms 前的位置并计算是否命中（**注意：计算后服务器立刻还原其位置**），从而抵消延迟带来的问题。

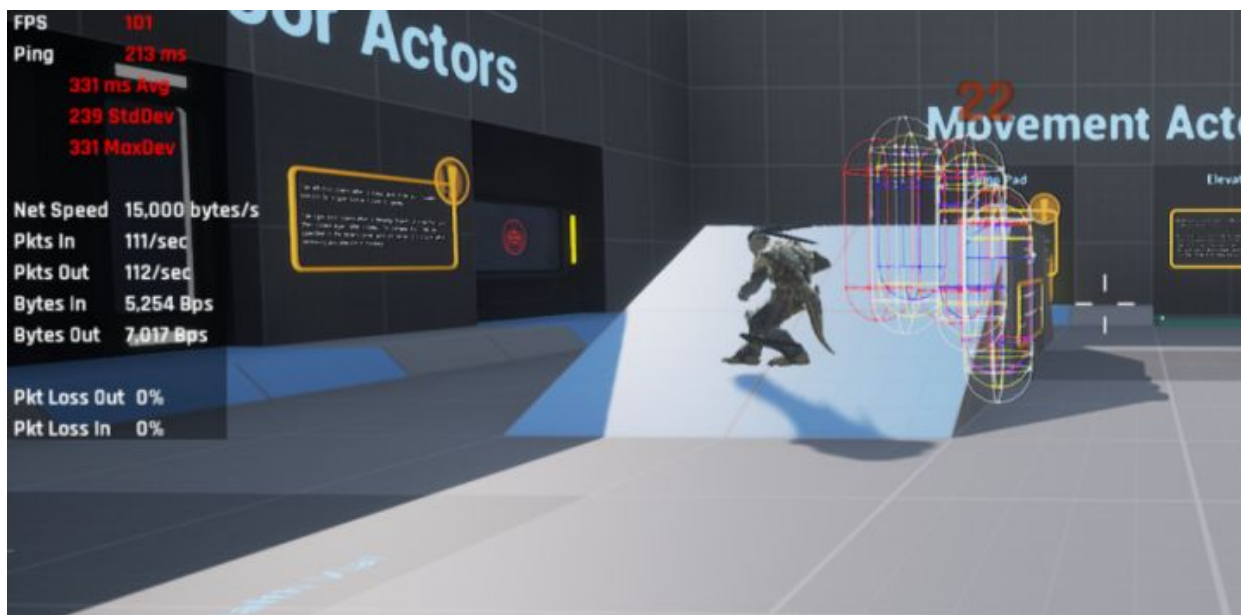


图4-12 红色是当前端的具体位置，黄色是回滚预测的位置

不过，延迟补偿并不是一个万能的优化方式，采用与否应该由游戏的类型与设计决定。考虑一个ACT类型的网游，玩家A延迟比较低、玩家B延迟比较高。在A的客户端上，玩家A在T1时间靠近B，而后立刻执行了一个**后滚**操作，发送到服务器。在B的客户端上，同样在T1时间发起进攻，然后发送命令到服务器。由于A的延迟低，服务器先收到了A的指令，A开始后滚操作，这时候A已经脱离了B的攻击范围。然后当B的指令到达服务器的时候，如果采用延迟补偿，就需要把A回滚到之前的位置结果就是A收到了B的攻击，这对A来说显然是不公平的。如果该情况发生在FPS里面，就不会有很大的问题，因为A根本不知道B什么时候瞄准的A。

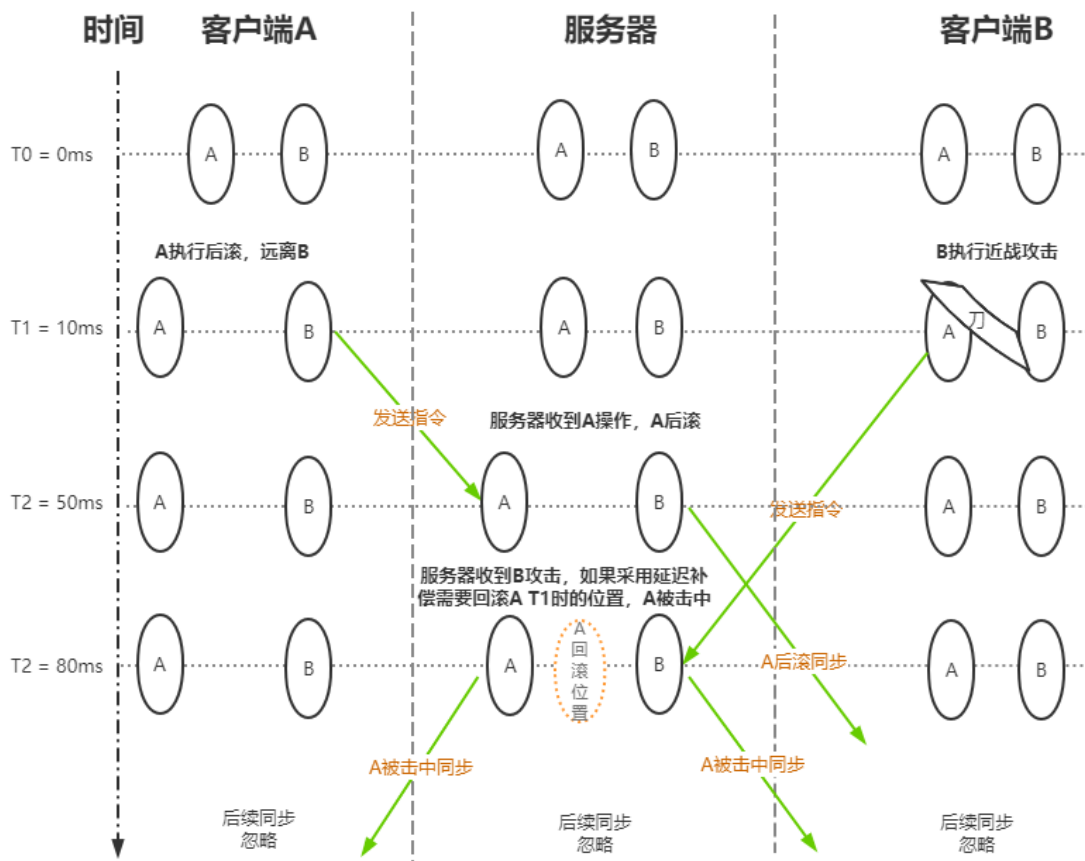


图4-13 ACT中采用延迟补偿会影响玩家体验

7.Trailing state synchronization

2004年, Eric Cronin等人在传统的Timewrap的回滚方式上提出了Trailing state synchronization算法[41] (TSS)。在他们看来, TimeWarp需要频繁的生成游戏快照进而占用大量内存(每次发送命令前都要生成一份), 而且每次遇到过期信息就立刻回滚并可能产生大量的对冲事件(anti-message)。这种同步方式是不适合Quake这种类型的FPS游戏的。

在TSS算法中, 游戏的快照不是随每个命令产生, 而是以某种延迟(比如100ms)间隔为单位对游戏做快照。他事先保存了N个完整的游戏状态(快照)以及命令链表, 让这N个状态以不同的延迟去模拟推进。游戏中延迟最低且被采用的状态称为Leading State, 其他的称为Trailing State, 每个状态都记录着一个命令链表(执行的以及未执行的), 各个状态的延迟间隔由开发者设定。Leading State向前推进的时候会不断的收到其他端的指令并添加到PendingCommands里面, 如果某个命令的执行时间小于当前已经推进到的时间(比如图4-14 CommandB指令在时间225ms才被Leading State执行), 就会放在表的最前面立刻执行, 这时候其实我们已经知道这个命令已经由于延迟错过正常执行时间, 可能要进行回滚操作了。但是对于后续的Trailing State, 这些过期Commands是可以被放到正确的位置的。当Trailing State执行到某个命令且发现Leading State在对应的位置没有这个命令的

话，他就会触发回滚（如果该命令对当前游戏无影响，其实也可以不回滚），将当前 Trailing State的状态信息拷贝到Leading State里面，然后设置错误命令时间至当前本地执行时间的所有命令为pending状态，触发这些状态的重新执行。

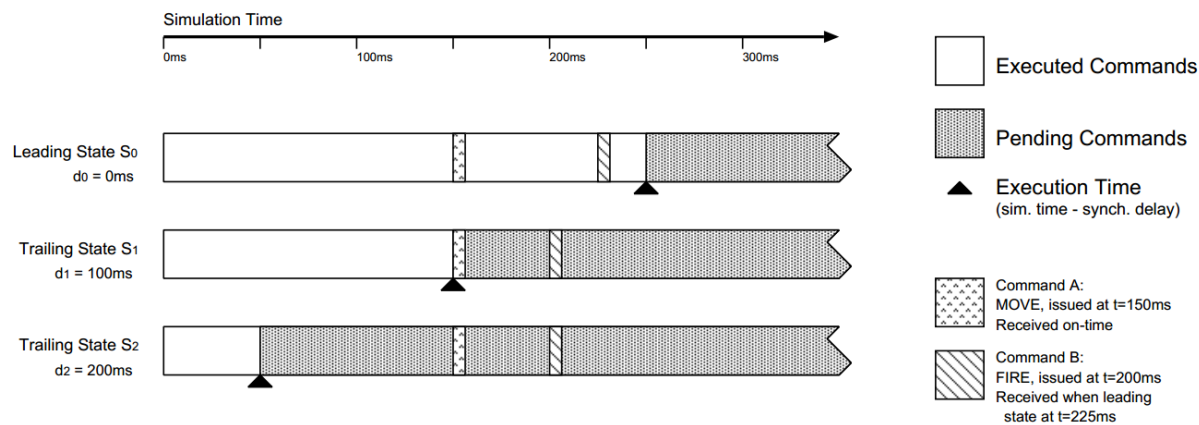


图4-14

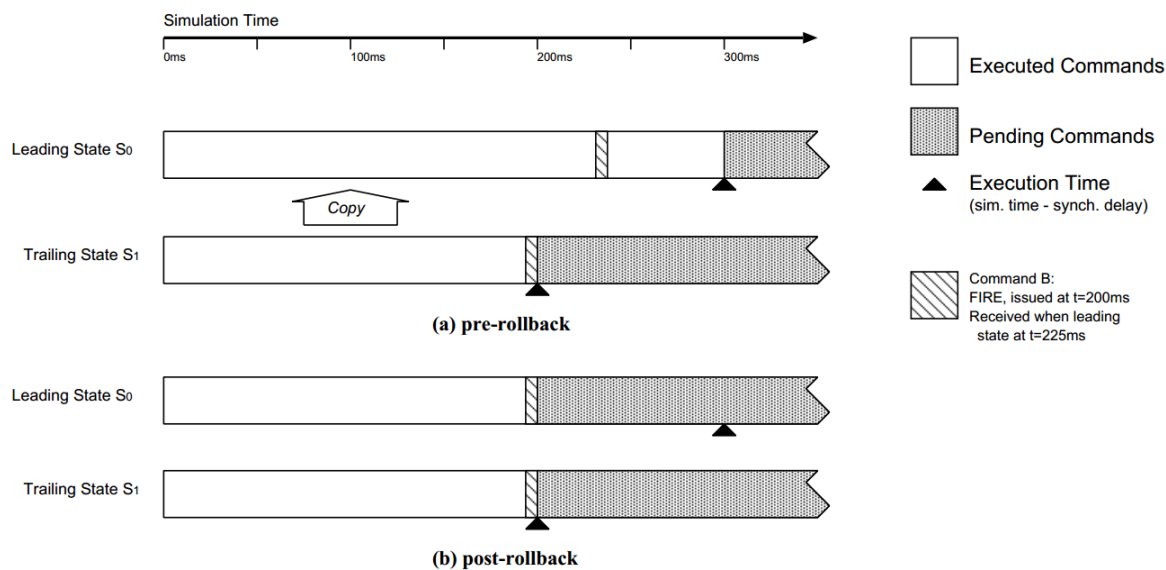


图4-15 Trailing State S1检测冲突并触发回滚

TSS相比TimeWarp，最大的优势就是大大降低了快照的记录频率（由原来的按事件记录改为按延迟时间分开记录），同时他可以避免由于网络延迟造成的连续多次指令错误而不断回滚的问题（Leading State不负责触发回滚，Trailing State检测并触发）。

不过TSS同时维护了多个游戏世界的快照，也无形中增加了逻辑的复杂度，在最近几年的网络游戏中也并没有看到哪个游戏使用了这种同步算法。在我看来，其实我们不必将整个世界的快照都记录，只要处理好移动的快照同时使用服务器状态同步就可以满足大部分情况了。

8.状态同步框架的演变

在2011年的GDC上，光环（Halo）项目的网络技术负责人David Aldridge就其网络同步框架发表了一次演讲。通过视频[42]，可以看到David同样借鉴了TribeEngine的网络架构并在此基础上进行更多细节的调整。

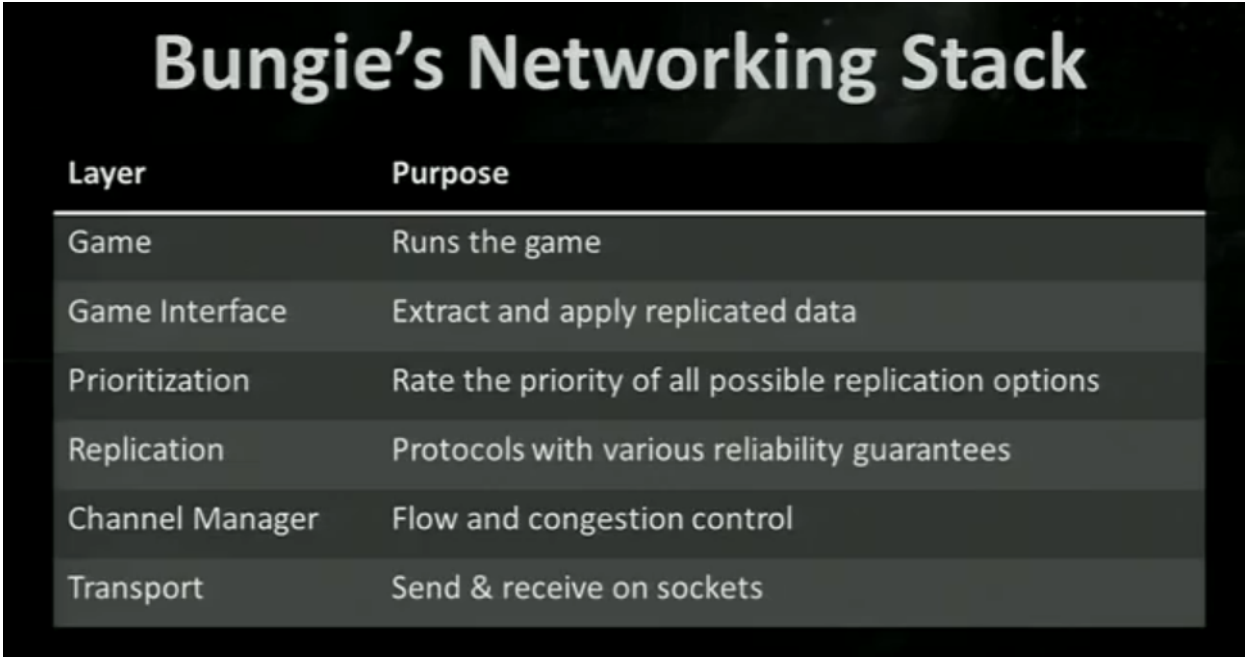


图4-16 光环的网络架构

光环项目的网络架构同样被分层，但相比Tribe却更加简洁和精炼。上图的Replication层是Gameplay开发中比较重视的，他决定了我们逻辑上层可用的同步手段。Halo里面有三种基本协议，State Data、Event、ControlData，分别是指“基于对象的属性同步”、“通过调用产生的事件同步”以及“玩家的输入信息同步”，其中移动同步归类于ControlData协议。

2015年，游戏业内著名的商业引擎——Unreal Engine正式开源，其中内置了一套非常完善的网络同步架构[43]。

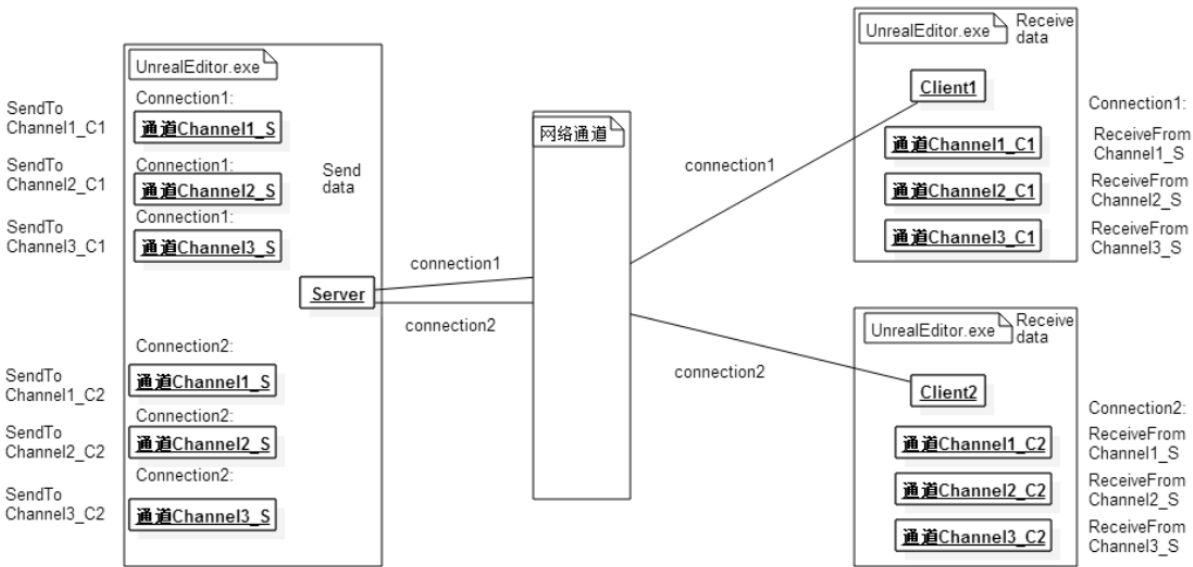


图4-17 UE4的网络通信架构

虚幻引擎的前身是FPS游戏——“虚幻竞技场”。该游戏早在1998年就发布，当时与Quake属于同类型的竞品。虚幻本身也是基于CS架构的状态同步，不过由于无法查找到当时的资料，笔者认为一开始可能也是与Quake非常相似的同步架构。后来在参考Tribe引擎的基础上，进行调整和优化，形成了如今的Netdriver/Connection/Channel/Uobject的模型，以及RPC和属性同步两种同步方式，这已经是网络同步发展至今非常典型且完善的状态同步方案了（后面要提到的OverWatch与其有很多相似之处）。作为一款游戏引擎，虚幻并没有将所有常见的同步手段都集成到引擎里面，只是将移动相关的优化方案（包括预测回滚、插值等）集成到了移动组件里面。其他的诸如延迟补偿，客户端预测等，他们放到了特定的Demo以及插件（GameplayAbility）当中。有兴趣的朋友可以去阅读一些Unreal的源码，看看最近几年其网络架构的发展变化。更多的细节也可以参考我的文章：[使用虚幻引擎4年，我想再谈谈他的网络架构](#)[44]

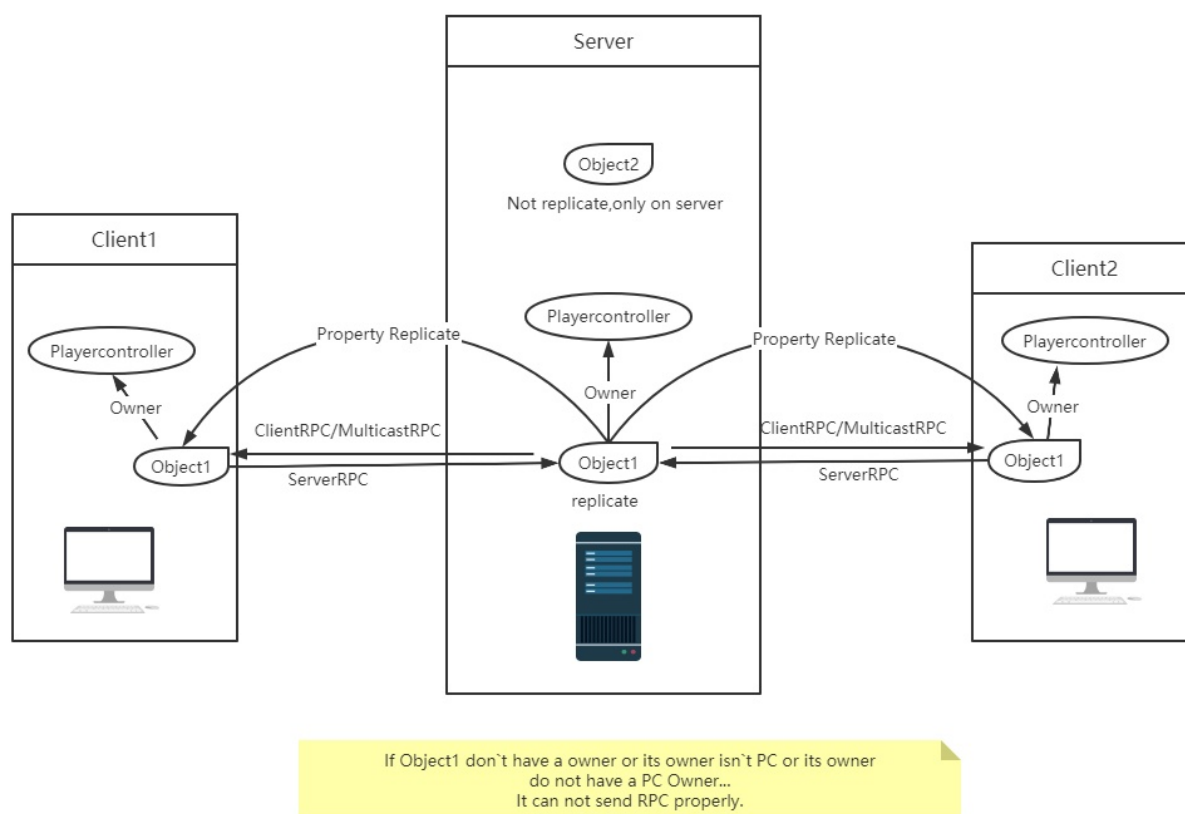


图4-18 UE4的通信方式展示

9.守望先锋与ECS架构

守望先锋可以说是近年来将网络同步优化到极致的FPS游戏，其中涵盖了我们可以用到的大部分同步优化技术。在2018年的GDC上，来自守望先锋的Gameplay程序TimFord分享了整个游戏的架构以及网络同步的实现方式[45]。

虽然OverWatch基于CS架构，但是却同时用到了帧同步（逻辑帧概念）以及状态同步包含的多种技术手段。为了实现确定性，他们固定了更新周期为16毫秒（电竞比赛时7毫秒），每个周期称为一个“命令帧”（等同于Lockstep中的“Turn”、“Bucket”）。在所有与客户端预表现和玩家行为有关的操作不会放在Update而是放在固定周期的UpdateFixed里更新，方便客户端预测与回滚。不过，整个游戏同步的核心还是状态同步，玩家也并不需要等待其他客户端的行为。

先用一句话来简单概括，守望先锋采用的是基于ECS架构的带有预测回滚的增量状态同步。

我们先从Gameplay层面去分析一下。在守望里面，网络同步要解决的问题被分为三部分，分别是**玩家移动，技能行为以及命中检测**。

移动模块，客户端本地会不断读取输入并立刻进行角色移动的模拟，他会在客户端记录一个缓冲区来保存历史的运动轨迹（即运动快照），用于后续与服务器纠正数据进行对比以及回滚。

技能行为模块，客户端添加了一个buffer来存储玩家的输入操作（带有命令帧的序号），同时保留历史的技能快照。一旦服务器发现客户端预测执行失败，就会让客户端先通过快照回滚到错误时刻（包括移动和技能），然后把错误时刻到当前时间的所有输入都重执行一遍。

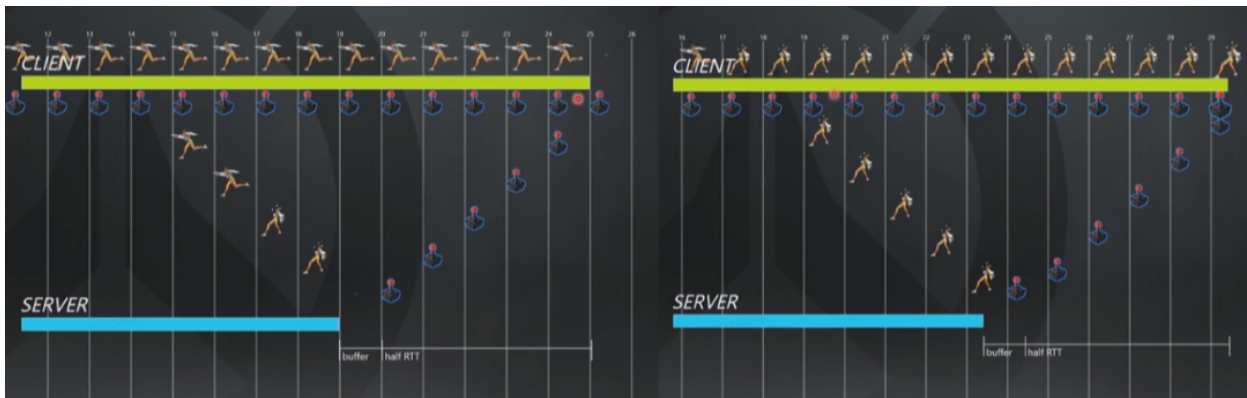


图4-19 左边是服务器通知客户端被眩晕，右边是客户端收到后进行回滚

命中模块，伤害计算在服务器，但是命中判定是在客户端处理（所以可能存在一些误差）。延迟补偿技术也被采用，但是不是在服务器回滚所有玩家的位置，而是检测当前玩家的准星与附近敌人的逻辑边界（bounding volumes）是否有交集，没有的话不需要回滚。

为了增强玩家的游戏体验，游戏还对不同ping的玩家进行了逻辑的调整。ping值会影响本地的预测行为，一旦PING值超过220毫秒，我们就会延后一些命中效果，也不会再去预测了，直接等服务器回包确认。PING为0的时候，对弹道碰撞做了预测，而击中点和血条没有预测，要等服务器回包才渲染。

当PING达到300毫秒的时候，碰撞都不预测了，因为射击目标正在做快读的外插，他实际上根本没在这里，这里也用到了前面提到的DR（Dead Reckoning）外推算法。



图4-20 守望先锋的命中检测边界展示

谈完Gameplay，我们可以再考虑一下他的状态同步是如何实现的。同样在2018年的GDC上，来自Overwatch服务器团队的开发工程师Phil Orwig分享了有关回放与同步的相关技术细节[46]。

客户端玩家操作后，这些指令会立刻发给服务器，同时本地开始执行预测。随后，服务器会将这一帧收到的所有玩家的输入进行处理和计算。在服务器上，每个对象产生的变化都会被记做一个Delta，并且会持续累积所有对象状态的变化并保存到一个临时的“每帧脏数据集合”（per frame dirty set）里。同时，服务器会对给每个客户端（每个Connection）也会维护一个对应的“脏数据集合”，这个集合可能保存一些之前没有发送出去的信息（如下图的C1是到客户端1的，C2是到客户端2的）。每帧结束时，所有客户端对应的“脏数据集合”会与当前脏集合F合并，随后当前脏集合F会被清空。



图4-21 守望先锋中的数据发送逻辑

同一个Tick的后期，这些对应不同客户端连接的脏集合（C1、C2等）会被序列化并发送给对应的客户端，同时从脏集合中移除。这里的序列化并不是完全使用原生的状态数据，而是维护了一个经客户端确认收到的状态数据的历史记录（比如我们服务器上已经记录了玩家的大部分信息，每次位置变化只序列化位置信息就可以了），这样我们就可以使用“增量编码”来改善带宽模型，即减少带宽的占用。

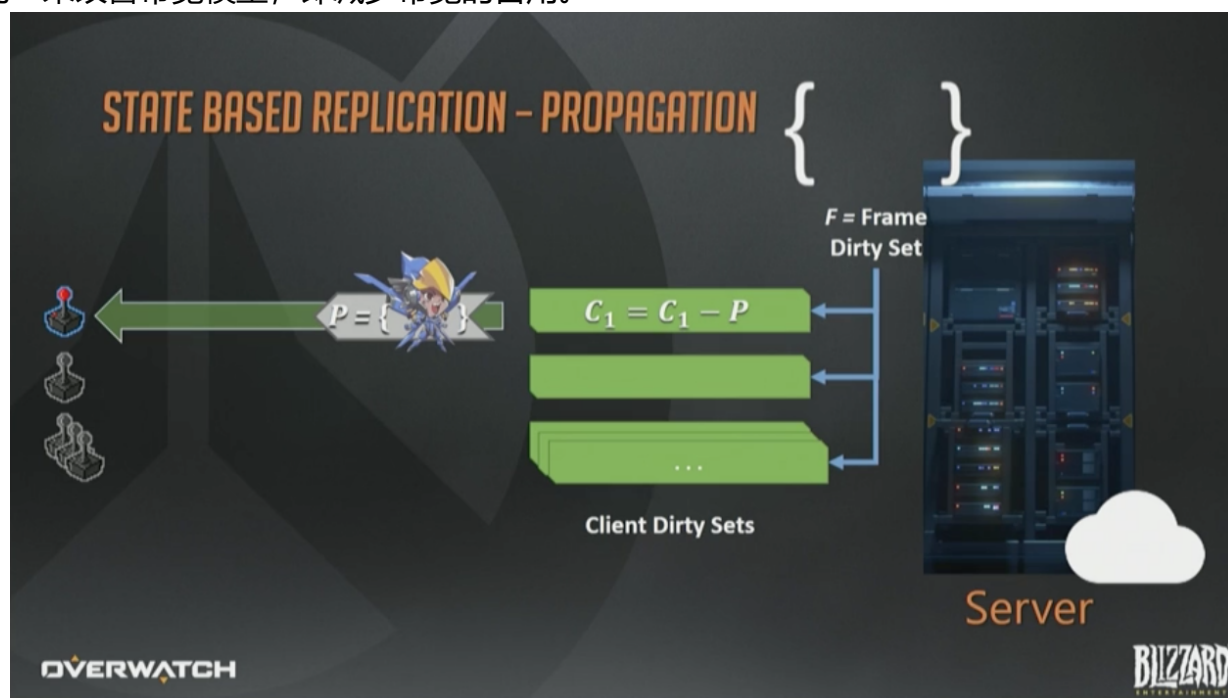


图4-22 增量发送同步数据

通过前面的分析，我们可以了解到整个网络同步的逻辑是很复杂的，细节也非常多。所以，我们也需要考虑是否能从底层和框架上做一些调整和优化。在守望先锋里面，他们并没有采

用常见的面向对象模型（OOP），而是使用了数据与操作行为分离的ECS架构[47]。Entity代表一个空的实体、Component代表一个只包含数据的组件、System代表一个处理数据的系统。在这个架构下，我们将面向对象编程转为面向数据编程，游戏的不同模块可以划分成不同的系统，每个模块只关心自己需要的数据（Component），这种模式下可以方便我们处理快照与回滚的逻辑。ECS系统看起来有着缓存友好、逻辑解耦等优点，但是操作起来问题也不少，其中最难处理的一个问题**就是如何控制System运作的次序。**

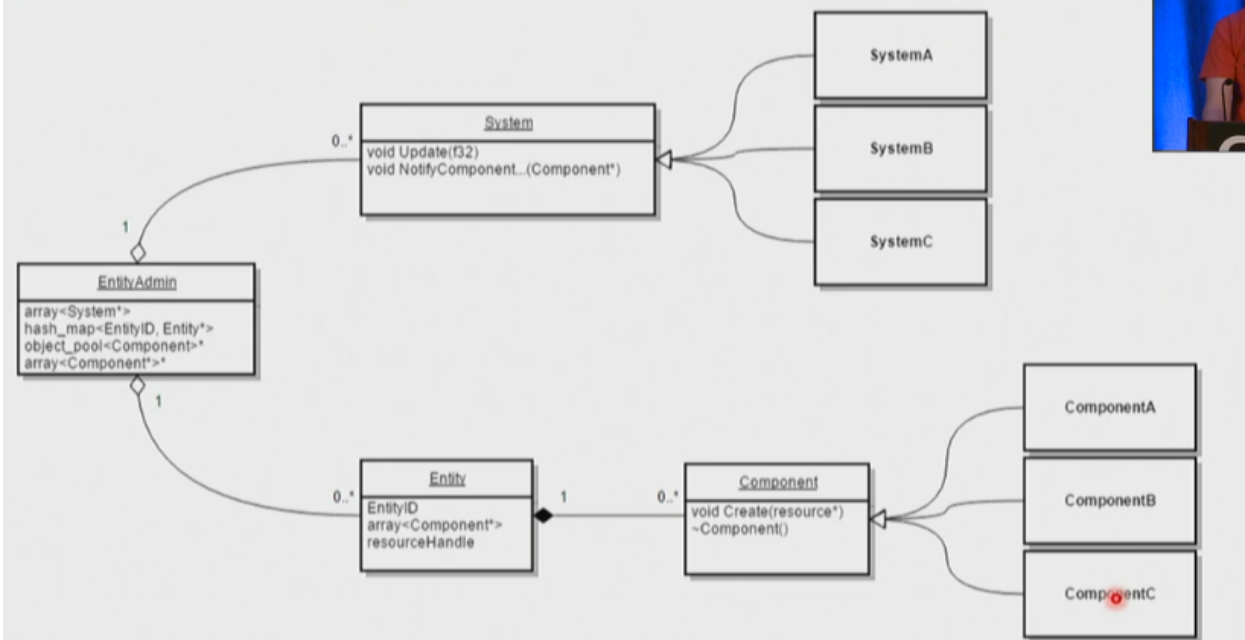


图4-23 守望先锋的ECS结构

最后，简单说一下底层的一些优化。为了提高通信效率，守望也采用定制的可靠UDP，因此会有不可避免的丢包情况。为了对抗丢包，每一帧的数据包包含的是最近N帧的数据，即使某一个数据包丢了也没什么影响。除此之外，他们还在服务器添加了一个缓冲区，记录玩家的输入信息。缓冲区越大，就能容忍越多的丢包，但也意味着同步延迟越大。所以，在网络条件良好的情况下，他们会尽力减小这个缓冲区的大小，而一旦客户端丢包，那么就可以提高客户端发送数据频率，进而服务器收到更多的包，缓存更多的数据用于抵消丢包。

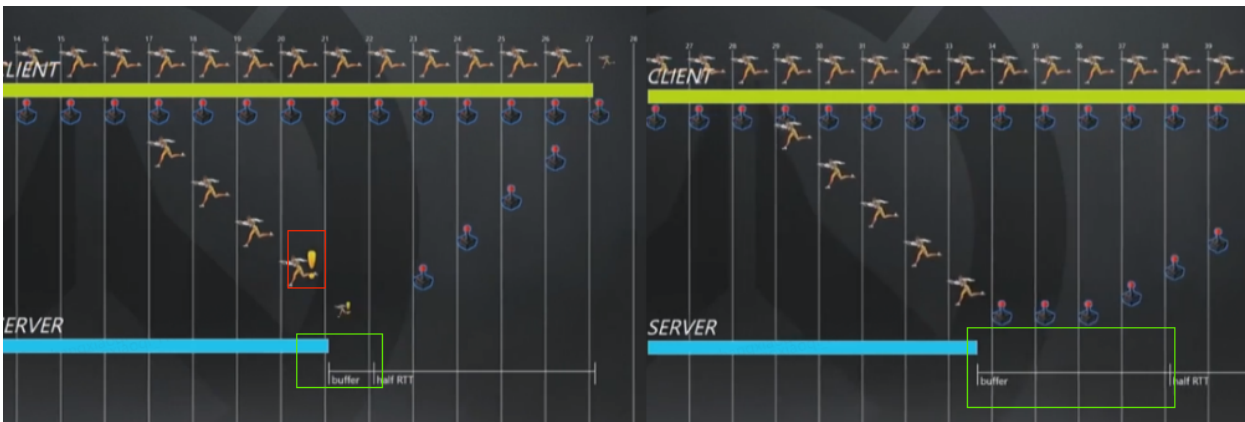


图4-24 动态Buffer

10.状态同步小结

状态同步大概在上世纪末就已经诞生（相比帧同步要晚一些），然而至今却没有一个完整的定义。不过单从名字上看，我们也能猜到“状态同步”同步的是对象的状态信息，如角色的位置、生命值等。

在Quake诞生前，其实也存在直接传输游戏对象状态的游戏，但是那时候游戏都比较简单，相关的概念也并不清晰。当时的架构模型以P2P为主，考虑搭配带宽限制等原因，军事模拟、FPS等游戏都采用了“Lockstep”的方式进行同步。

不过由于作弊问题日益严重、确定性实现困难重重等因素，CS架构逐渐代替P2P走向主流。我们也发现似乎所有的游戏状态信息都可以保存在服务器上，客户端只需要接受服务器同步过来的状态并渲染就可以了。按照这种思路，Quake诞生了，他抛弃了Doom的架构并带着状态同步的方式进入我们的视野。这时候的状态同步还只是简单的快照同步，每次同步前服务器都需要把整个游戏世界的状态信息打包发送给客户端。

然而，快照同步太浪费带宽了，不同的玩家在一段时间内只能在很小的范围内活动，根本没有必要知道整个世界的状态。同时，每次发送的快照都与之前的快照有相当多重重复的内容，确实过于奢侈。因此，星际围城：部落的开发团队构建出了一个比较完善的状态同步系统，用于对同步信息进行分类和过滤。

后来，光环、虚幻竞技场、守望先锋、Doom等游戏都在Tribe Engine的基础上不断完善状态同步，形成了如今的架构模型。

如今距离状态同步的诞生已经20余年，当我们现在再讨论状态同步时，到底是指什么呢？
我认为，如今的状态同步是指包含增量状态同步、RPC（事件同步）两种同步手段，并且可以在各个端传递任何游戏信息（包括输入）的一种同步方式。

目前的状态同步多用于CS架构，客户端通过RPC向服务器发送指令信息，服务器通过属性同步（增量状态同步）向客户端发送各个对象的状态信息。我们可以采用预测回滚、延迟补偿、插值等优化方式，甚至也可以采用“命令帧”的方式对同步做限制。不过在这个过程中，传递的内容以状态信息（即计算后的结果）为主，收到信息的另一端只需要和解同步过来的状态即可，不需要在本地通过处理其他端的Input信息来进行持续的模拟。

最后，再次拿出虚幻引擎的网络同步模型来展示当今的状态同步。

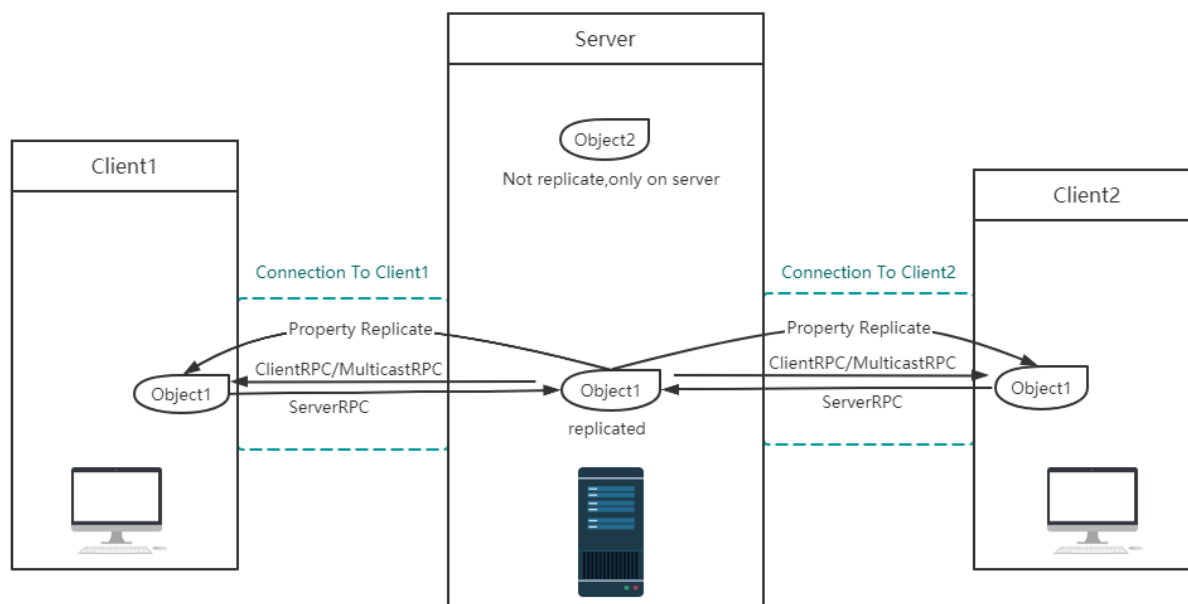


图4-25 虚幻引擎同步模型

五、物理同步

1.概念与理解

所谓“物理同步”，字面上讲就是“带有物理状态对象的网络同步”，严格上来说它并不是一个标准的技术名词，而是大家约定俗成的一个概念。按照我的个人理解，可以进一步解释为“在较为复杂的物理模拟环境或有物理引擎参与计算的游戏里，如何对持有物理状态信息的对象做网络同步”。在英文中，我们可以使用Replicate physics-simulated objects 或者Networked physics来表示类似的概念。



图5-1 火箭联盟中的物理玩法（用车踢球）

不过，考虑到并不是所有物理现象都交给物理引擎处理，而且有物理引擎参与的网游也并不一定需要对同步做任何处理，所以我们常说的物理同步更多的是指“**在网络游戏中，如果玩家的位置或者与玩家交互对象的位置需要经过物理引擎的模拟处理来得到结果，那么其中涉及到网络同步技术就可以称为物理同步**”。（这里的物理模拟一般指整个对象完全交给物理引擎去计算碰撞、位置、约束等，很多情况下可以等价为对**Ragdoll**的模拟）

备注：物理一词涉及的范围非常广，在游戏里面应用的场景也很多，但是并不一定需要进行网络同步，比如简单的抛物线运动，射线检测，与玩法无关的场景破碎等。

早在上世纪70年代，就诞生了许多围绕物理特性产生玩法的游戏，不过由于当时计算机系统算力有限，涉及到的物理计算都非常简单（比如乒乓球游戏中小球的移动模拟[48]）。随着计算机性能的飞速提升，开发者们考虑将环境中的所有对象都交由统一的物理模块驱动，由此慢慢的催生出了通用的物理引擎[49]。很快的，各个游戏开发商逐渐将物理引擎集成进来，将更多更复杂的物理模拟过程应用到游戏中，制作出了诸如极品飞车、FIFA、NBA、愤怒的小鸟等围绕物理特性进行玩法设计的游戏。另一方面，随着计算机网络的发展，游戏中的网络同步技术愈加成熟，网络游戏的品质也不断向单机游戏靠拢，我们也得以将传统的单机游戏拓展成多人游戏。物理模拟作为提升游戏趣味性的一大技术也自然逐渐被纳入其中，物理同步变得重要起来。

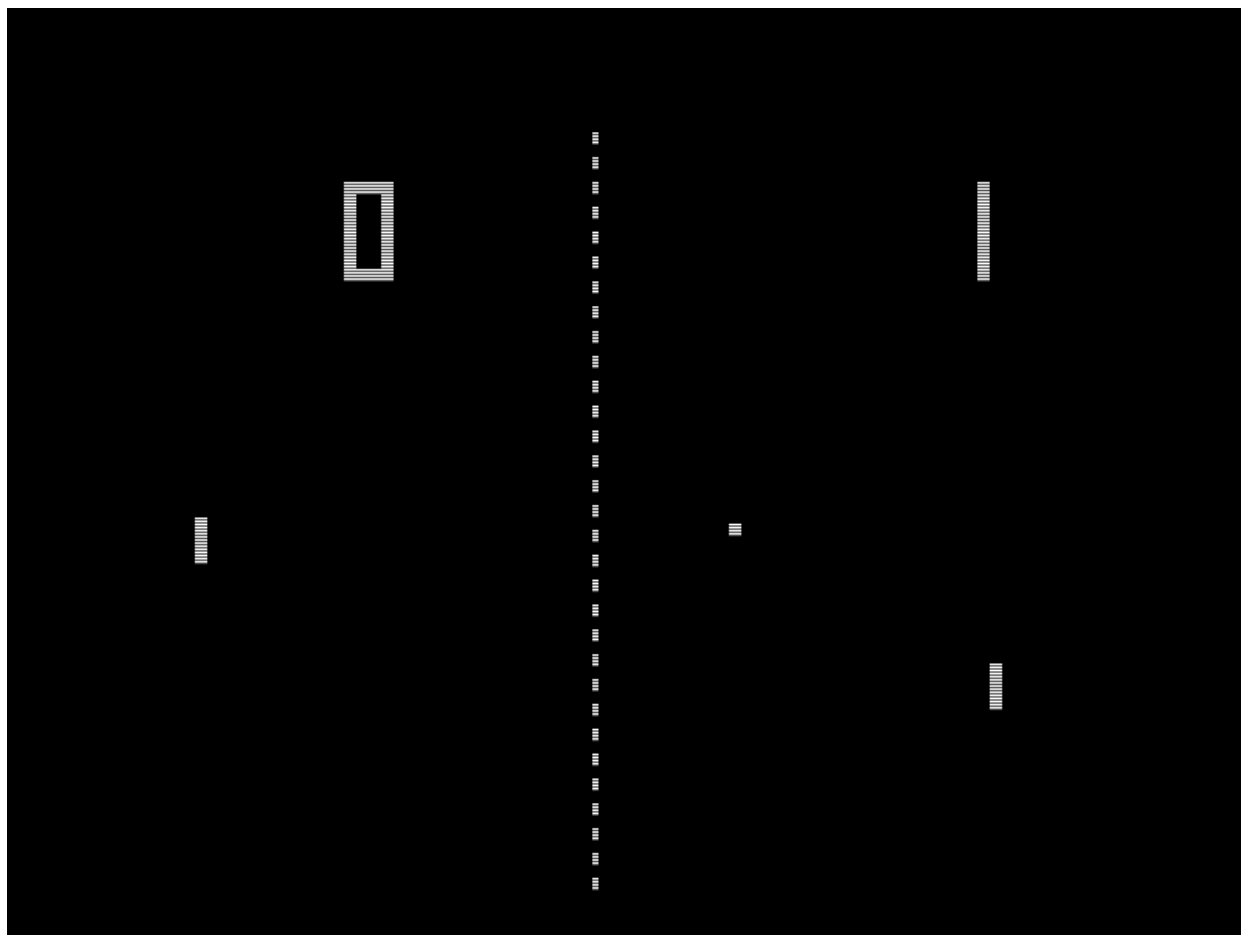


图5-2 最简单的物理游戏——Pong

2.面临的问题与解决方案

正如所前面解释的那样，物理同步并不是一种特殊的同步方式，而是在物理引擎和网络同步技术共同发展的条件下而诞生的一种综合性解决方案，其核心手段还然是我们熟悉的帧同步或者状态同步。使用帧同步技术我们需要每帧把玩家的Input信息发送出去，然后让另一端的物理引擎根据输入去模拟结果。如果使用状态同步我们则需要本地模拟好数据并把物理位置、旋转等关键信息发送到其他的客户端，然后其他客户端可以根据情况决定是否再执行本地的物理模拟（如果是快照同步，由于拿到的就是最终的结果，那么就不需要本地再进行模拟了）。

这样看来，物理同步好像与常规的同步也没什么本质上的区别，那么为什么他却是一个难题呢？我认为原因有以下两点：

- 物理引擎的不确定性
- 在物理引擎参与模拟的条件下，网络同步的微小误差很容易被迅速放大

a.物理引擎的确定性问题

首先，我们谈谈**物理引擎的确定性问题**。很不幸，目前所有的物理引擎严格来说都不是确定性的，因为想保证不同平台、编译器、操作系统、编译版本的指令顺序以及浮点数精度完全一致几乎是不可能的。关于物理确定性的讨论有很多[50]，核心问题大致可以归类为以下几点：

- 1.编译器优化后的指令顺序
- 2.约束计算的顺序
- 3.不同版本、不同平台浮点数精度问题[51][52]

这里摘选一段PhysX物理引擎的描述[53]：

The PhysX SDK can be described as offering limited determinism（注：提供了有限程度的确定性）。Results can vary between platforms due to differences in hardware maths precision and differences in how the compiler reorders instructions during optimization. This means that behavior can be different between different platforms, different compilers operating on the same platform or between optimized and unoptimized builds using the same compiler on the same platform（注：不同平台、编译器、优化版本都会影响确定性）。However, on a given platform, given the exact same sequence of events operating on the exact scene using a consistent time-stepping scheme, PhysX is expected to produce deterministic results. In order to achieve this determinism, the application must recreate the scene in the exact same order each time and insert the actors into a newly-created PxScene. There are several other factors that can affect determinism so if an inconsistent (e.g. variable) time-stepping scheme is used or if the application does not perform the same sequence of API calls on the same frames, the PhysX simulation can diverge.

如果游戏只是单个平台上发行，市面上常见的物理引擎（Havok，PhysX，Bullet）基本上都可以保证结果的一致性。因为我们可以通过使用同一个编译好的二进制文件、在完全相同的操作系统上运行来保证指令顺序并解决浮点数精度问题，同时打开引擎的确定性开关来保证约束的计算顺序（不过会影响性能），这也是很多测试者在使用Unity等商业引擎时发现物理同步可以完美进行的原因。当然，这并不是说我们就完全放弃了跨平台确定性的目标，比如Unity新推出的DOTS架构[54][55]正在尝试解决这个问题（虽然注释里面仍然鲜明的写着“Reserved for future”）。

Real-time physics engines [\[edit \]](#)

Open source

- [Advanced Simulation Library](#) - open source hardware accelerated multiphysics simulation software
- [Box2D](#)
- [Bullet](#)
- [Chipmunk physics engine](#) - 2D physics engine
- [Newton Game Dynamics](#)
- [Open Dynamics Engine](#)
- [PAL \(Physics Abstraction Layer\)](#) - A uniform API that supports multiple physics engines
- [PhysX](#)
- [Project Chrono](#) - An open source simulation engine for multi-physics applications.
- [Siconos Modeling and the simulation of mechanical systems with contact, impact and Coulomb's friction](#)
- [SOFA \(Simulation Open Framework Architecture\)](#)
- [Tokamak physics engine](#)

High precision physics engines [\[edit \]](#)

- [VisSim](#) - Visual Simulation engine for linear and nonlinear dynamics
- [Working Model](#) by Design Simulation Technologies

图5-3 常见物理引擎

考虑到物理引擎的确定性问题，我们可以得出一个初步的结论——完全使用帧同步做物理同步是不合适的（或者说做跨平台游戏是行不通的）。而对于状态同步，我们可以定时地去纠正位置信息来避免误差被放大。如果一定要使用帧同步去做跨平台同步，那么只能选择放弃物理引擎自己模拟或者用定点数来改造物理引擎，这可能是得不偿失的。

下面不妨先排除掉一致性的问题，来看看如何实现所谓的“物理同步”。实际上，无论是优化手段还是实现方式与前两篇提到的方案是几乎一致的，帧同步、快照同步、状态同步都可以采用，增量压缩、Inputbuffer等优化手段也一样可以用于物理同步的开发中。Network Next的创始人Glenn Fiedler在2014年撰写了一系列的物理同步相关的文章[56]，使用一个同步的Demo非常详细地阐述了同步技术是如何应用以及优化的。涉及到的技术点大致如下，涵盖了网络同步的大部分的知识细节：

- 如何确保物理引擎的确定性
- 如何实现物理帧同步
- Inputbuffer如何改善帧同步
- 为什么用UDP替代TCP
- 如何实现快照同步
- 怎样用插值解决网络抖动
- 如何通过快照压缩减少网络流量
- 如何实现增量压缩
- 如何实现状态同步

另外，在2018年的GDC上，Glenn也对物理同步进行一次演讲分享[57]，具体的细节建议大家移步到Glenn Fiedler的网站以及GitHub[58]去看。

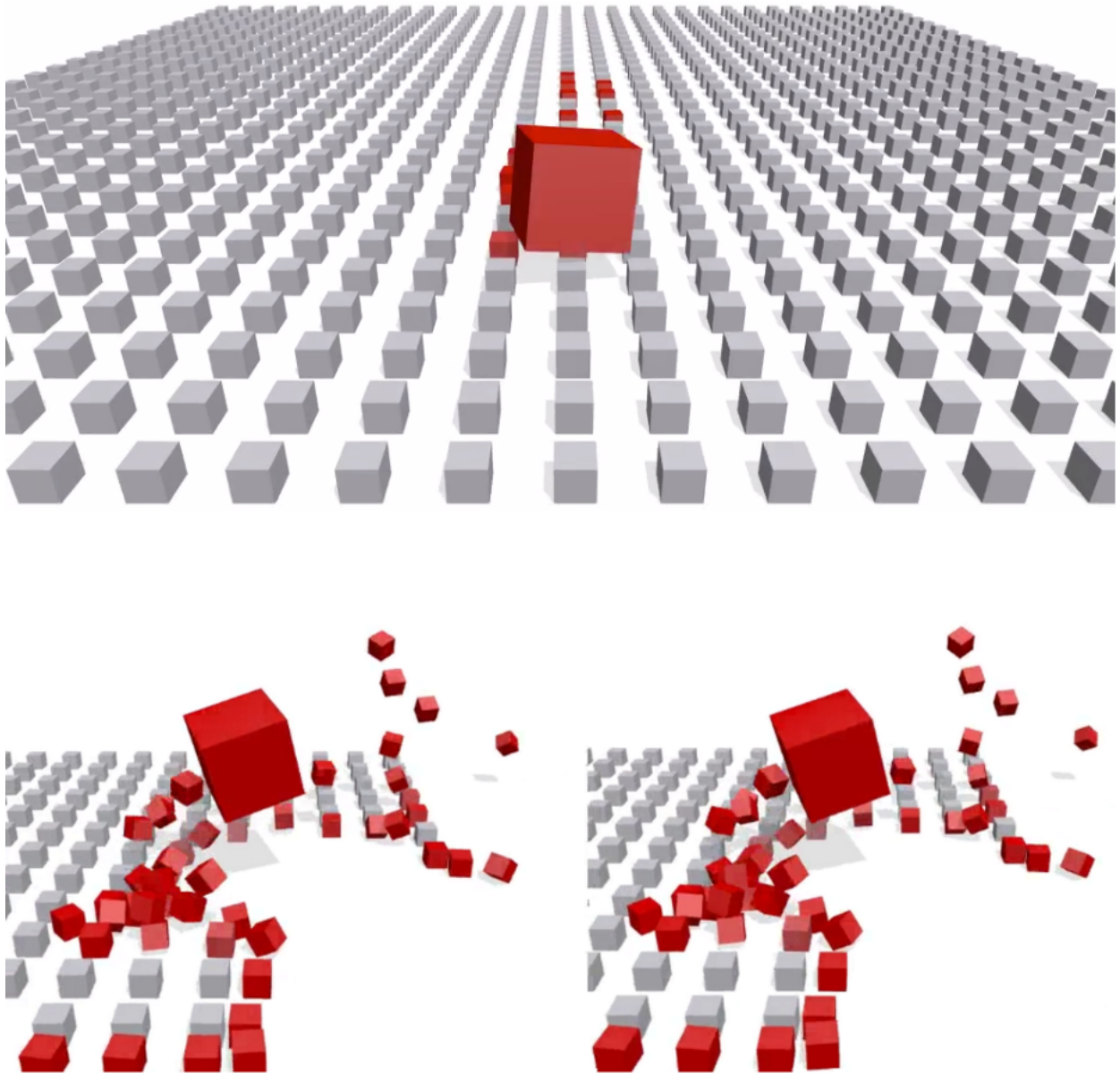


图5-4 物理同步效果展示

b.同步误差如何被物理引擎放大

接下来，我们再来谈谈第二个难点，即**网络同步的误差是如何被物理模拟迅速放大的（尤其在多人交互的游戏中）**。我们在前面的章节里也谈过，为了保证本地客户端的快速响应，通常会采取预测回滚的机制（Client prediction，即本地客户端立刻相应玩家操作，服务器后续校验决定是否合法）。这样我们就牺牲了事件顺序的严格一致来换取主控端玩家及时响应的体验，在一般角色的非物理移动同步时，预测以及回滚都是相对容易的，延迟比较小的情况位置的误差也可以几乎忽略。然而在物理模拟参与的时候，情况就会变得复杂起来。

主控（Autonomous/Master）即当前角色是由本地玩家控制的，模拟（Simulate/Replica）即当前角色是由其他玩家控制的

假如在一个游戏中（带有预测，也就是你本地的对象一定快于远端）你和其他玩家分别控制一个物理模拟的小车朝向对方冲去，他们相互之间可能发生碰撞而彼此影响运动状态，就会面临下面的问题。

1.由于网络同步的误差无法避免，那么你客户端上的发生碰撞的位置一定与其他客户端的不同。

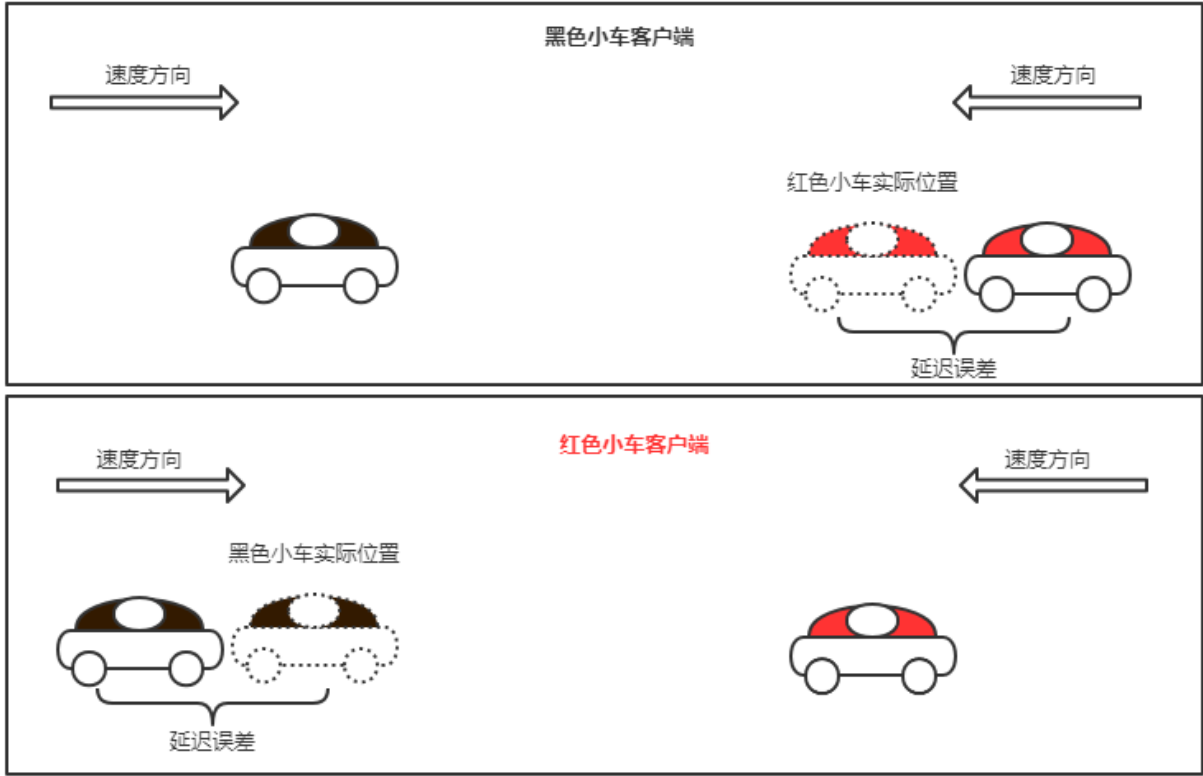


图5-5 不同客户端小车本地与远端位置不同

2.其次，对于本地上的其他模拟小车，要考虑是否在碰撞时完全开启物理模拟（Ragdoll）。如果不开启物理，那么模拟小车就会完全按照其主控端同步的位置进行移动，即使已经在本地发生了碰撞他可能还是会向前移动。如果开启碰撞，两个客户端的发生碰撞的位置会完全不同。无论是哪种情况，网络同步的误差都会在物理引擎的“加持”下迅速被放大进而导致两端的结果相差甚远。

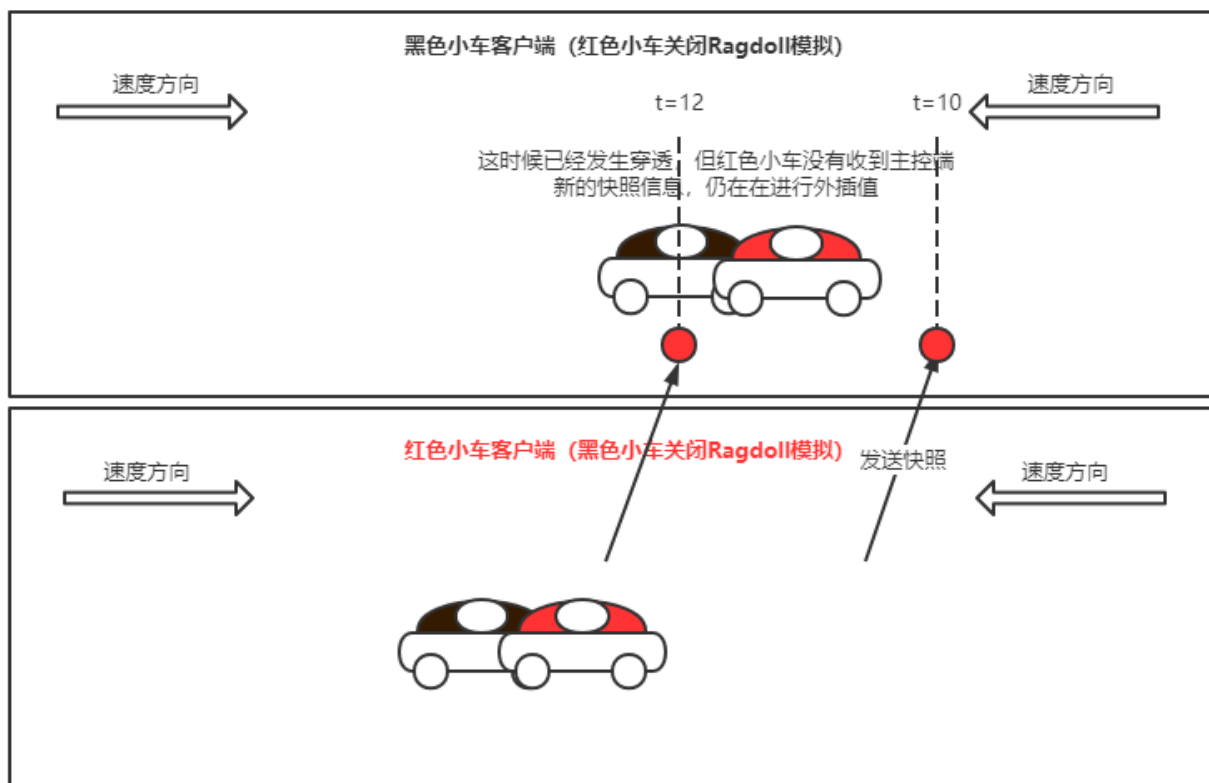


图5-6 关闭物理模拟时机影响碰撞的位置

其实对于一般角色的非物理移动同步，二者只要相撞就会迅速停止移动，即使发生穿透只要做简单的位置“回滚”即可。然而在物理模拟参与的时候，直接作位置回滚的效果会显得非常突兀并出现很强的拉扯感，因为我们几乎没办法在本地准确的预测一个对象的物理模拟路径。如果你仔细阅读了前面Glenn Fiedler的文章（或者上面总结的技术点），你会发现里面并没有提到常见的预测回滚技术，因为他只有一个主控端和一个用于观察结果的模拟端，并不需要回滚。

c. 《看门狗2》的载具同步

在2017年的GDC上，来自育碧的技术负责人Matt Delbosc就《看门狗2》中的载具同步进行了演讲[59]，详细的分析了多个主控端控制不同对象发生碰撞时应该如何处理。

《看门狗2》的网络模型是基于状态同步的P2P，主控角色预测先行而模拟对象会根据快照（snapshot，即模拟对象在其主控端的真实位置）使用Projective Velocity Blending做内插值，他们在制作时也面临和上面描述一样的问题。假如两个客户端各控制一个小车撞向对方，由于延迟问题，敌人在本地的位置一定是落后其主控端的。那么就可能发生你开车去撞他时，你本地撞到了他的车尾，而他的客户端什么都没有发生。

Too much interpolation

- Rendering objects that are further back in time

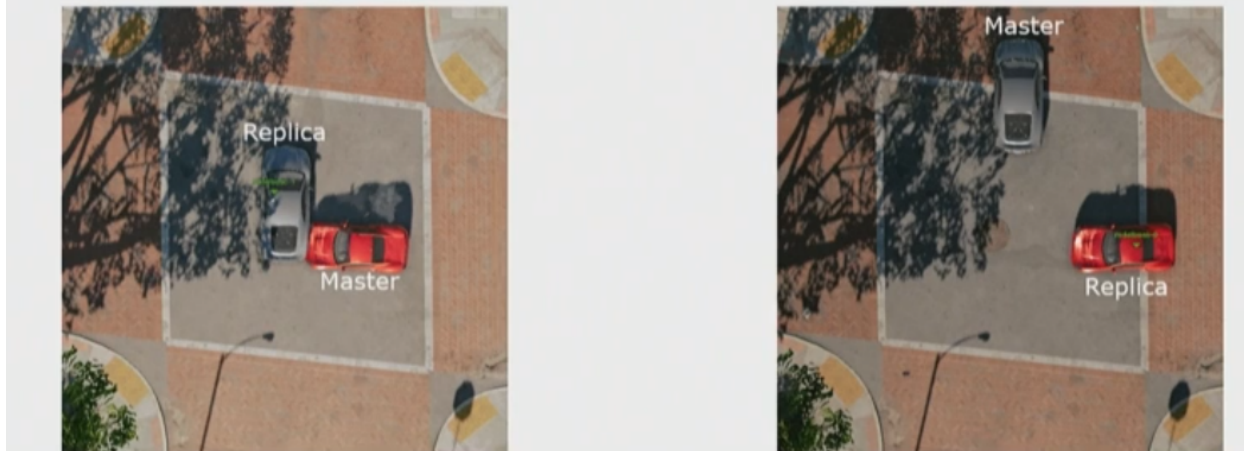


图5-7 延迟导致不同客户端效果不同

所以，首先要做的就是尽量减少不同客户端由于延迟造成的位置偏差，Matt Delbosc引入了一个TimeOffset的概念，根据当前时间与TimeOffset的差值来决定对模拟对象做内插值还是外插值，有了合适的外插值后本地的模拟对象就可以做到尽量靠近敌方的真实位置。

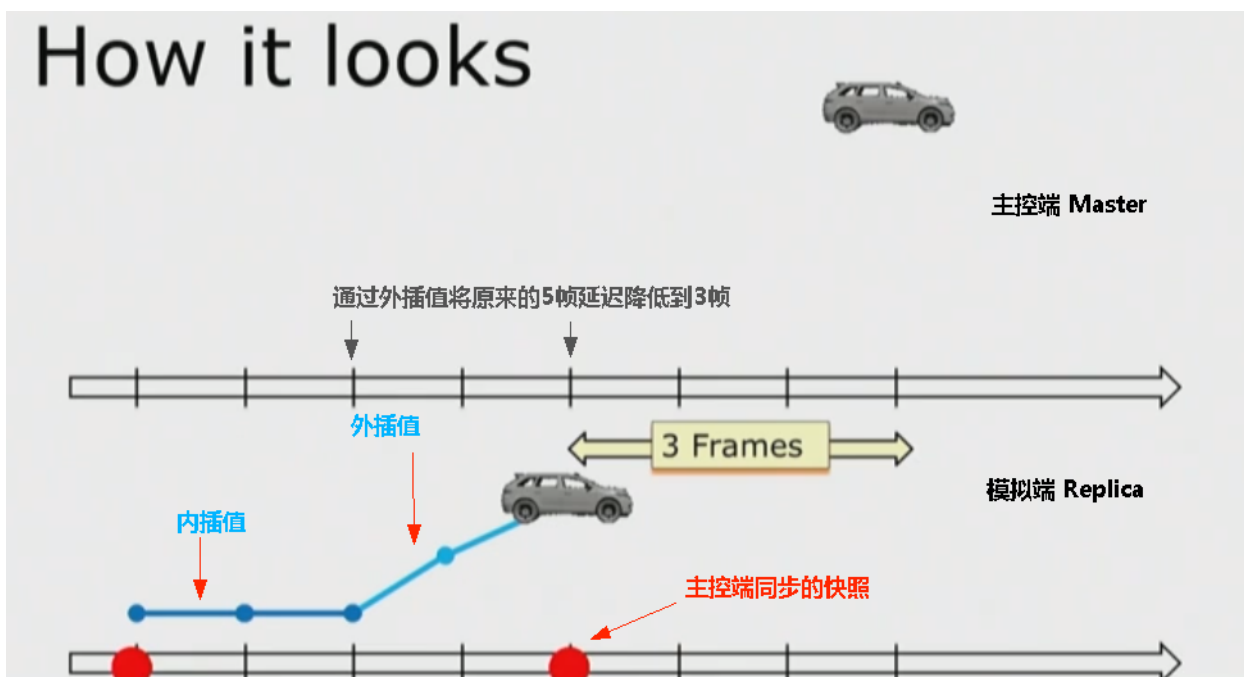


图5-8 外插值减少误差

而关于碰撞后位置的误差问题，他们采用了Physics Simulation Blending技术，即发生碰撞前开启模拟对象的RigidBody并设置位置权重为1（快照位置的权重为0），然后在碰撞发生后的一小段时间内，不断减小物理模拟的权重增大快照位置的权重使模拟对象的运动状态逐渐趋于与其主控端，最终消除不一致性，腾讯的吃鸡手游就采用了相似的解决方案[60]。

Blend Factor Tuning

- Play out lots of collisions and choose what looks good
 - Max blend factor to use while in a collision
 - How quickly to go back to full dead reckoning afterwards

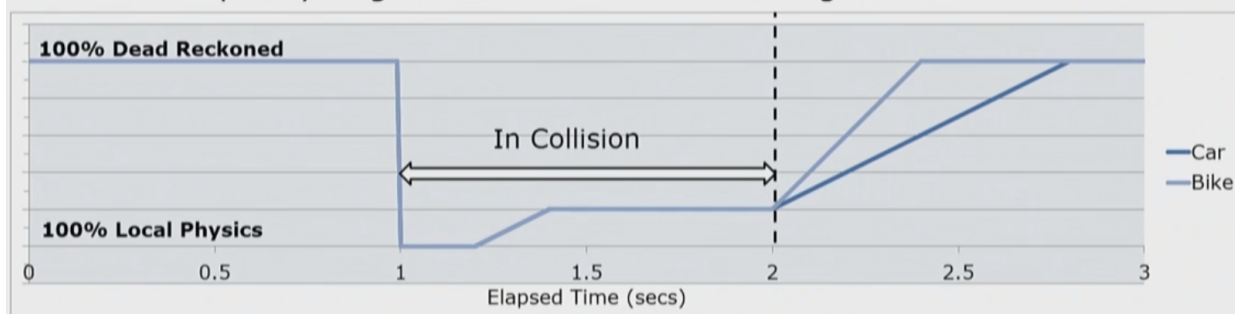


图5-9 Physics Simulation Blending逻辑展示

不过实际上，Matt团队遇到的问题远不止这些，还有诸如如何用插值解决旋转抖动问题，人物与载具相撞时不同步怎么办等等，知乎上有一篇译文可以参考[61]。

可能有些朋友会问，如果我不使用预测回滚技术是不是就没有这个问题呢？答案依然是否定的，假如你在运行一个车辆的中间突然变向，而这个操作被丢包或延迟，只要服务器不暂停整个游戏来等待你的消息，那么你本地的结果依然与其他客户端不同进而产生误差。也就是说除非你使用最原始的“完全帧同步”（即客户端每次行动都要等到其他客户端的消息全部就绪才行），否则由于网络同步的延迟无法避免，误差也必定会被物理模拟所放大。

d. 《火箭联盟》的物理同步

同样在2017年，另一款风靡全球的竞技游戏——《火箭联盟》悄然上线，可谓是将物理玩法发挥到了极致。次年，《火箭联盟》的开发者Jared Cone也来到了GDC，分享了他们团队是如何解决物理同步问题的[62]。

《火箭联盟》的核心玩法是“用车踢球”，每个玩家控制一个汽车，通过撞击足球来将其“踢”进敌方的球门。由于是多人竞技游戏，所以一定要有一个权威服务器来避免作弊，最终的结果必须由服务器来决定。相比于《看门狗》，他们遇到的情况明显更复杂，除了不同玩家控制不同的小车，还有一个完全由服务器操控的小球。按照常规的同步方式，本地的主控玩家预测先行，其他角色的数据由服务器同步下发做插值模拟。但是在这样一个延迟敏感且带有物理模拟的竞技游戏中，玩家的Input信息的丢失、本地对象与服务器的位置不统一都会频繁的带来表现不一致的问题，而且FPS中常见的延迟补偿策略并不适合当前的游戏类型（简单来说就是延迟大的玩家会影响其他玩家的体验，具体原因我们在上一篇延迟补偿的章节也有讨论）。

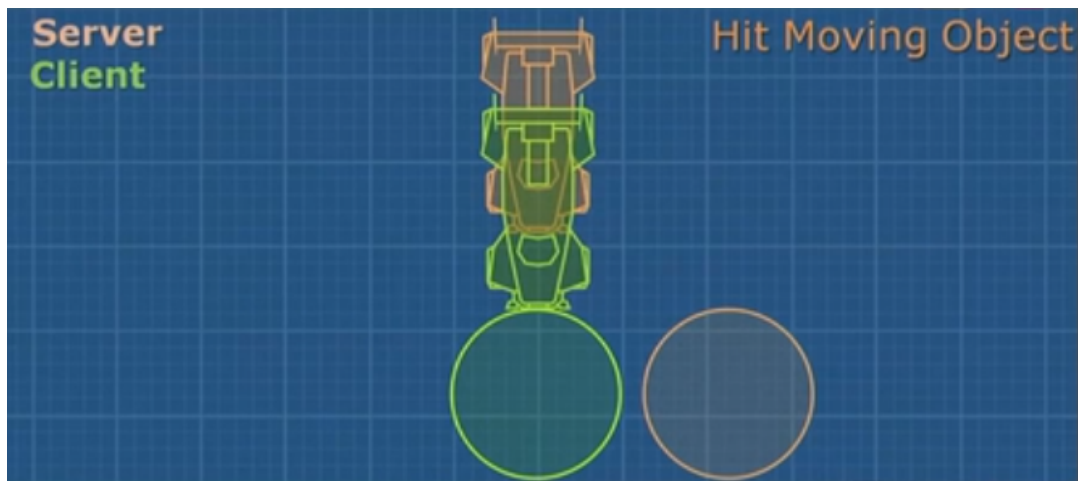


图5-10 客户端与服务器实际位置不同

为了解决这些问题，Jared Cone团队采用了“InputBuffer”以及“客户端全预测”两个核心方案。**InputBuffer**，即服务器缓存客户端的Input信息，然后定时的去buffer里面获取（buffer大小可以动态调整），这样可以减少网络延迟和抖动带来的卡顿问题。

Input Buffer

- Client sends input for every physics frame
- Client inputs do not arrive at constant rate
- Server buffers client input
- No need to pause for input
- Eliminates some cheats (speed, jitter)

图5-11 Input Buffer

客户端全预测，即客户端上所有可能产生移动的对象（不仅仅是主控对象）全部会在本地预测先行，这样本地在预测成功时所有对象的位置都是准确的，客户端与服务器的表现也会高度一致，当然预测失败的时候自然会也要处理位置回滚。



图5-12 通过客户端全预测保证客户端与服务器位置一致

仔细分析这两款游戏，你会发现他们采用都是“状态同步+插值+预测回滚”的基本框架，这也是目前业内上比较合适的物理同步方案。

除了同步问题，物理引擎本身对系统资源（CPU/GPU）的消耗也很大。比如在UE4引擎里面，玩家每一帧的移动都会触发物理引擎的射线检测来判断位置是否合法，一旦场景内的角色数量增多，物理引擎的计算量也会随之增大，进而改变Tick的步长，帧率降低。而帧率降低除了导致卡顿问题外，还会进一步影响到物理模拟，造成更严重的结果不一致、模型穿透等问题，所以我们需要尽量减少不必要的物理模拟并适当简化我们的计算模型。

六、TCP VS UDP

网络同步本质是数据的传输，当逻辑层面优化已经不能满足游戏的即时性要求时，我们就不得不考虑更深一层协议上的优化，而这件事开发者们从上世纪90年代就开始尝试了。

按照OSI模型（Open System Interconnection Model），我们可以将计算机网络分为七层。一般来说，我们在软件层面（游戏开发）最多能干涉的到协议就是传输层协议了，即选择TCP还是UDP。网上关于TCP和UDP的文章与讨论有很多[63]，这里会再帮大家梳理一下。

OSI model			
	Layer	Protocol data unit (PDU)	Function ^[19]
Host layers	7 Application	Data	High-level APIs, including resource sharing, remote file access
	6 Presentation		Translation of data between a networking service and an application; including character encoding, data compression and encryption/decryption
	5 Session		Managing communication sessions, i.e., continuous exchange of information in the form of multiple back-and-forth transmissions between two nodes
	4 Transport	Segment, Datagram	Reliable transmission of data segments between points on a network, including segmentation, acknowledgement and multiplexing
Media layers	3 Network	Packet	Structuring and managing a multi-node network, including addressing, routing and traffic control
	2 Data link	Frame	Reliable transmission of data frames between two nodes connected by a physical layer
	1 Physical	Symbol	Transmission and reception of raw bit streams over a physical medium

图6-1 OSI网络模型

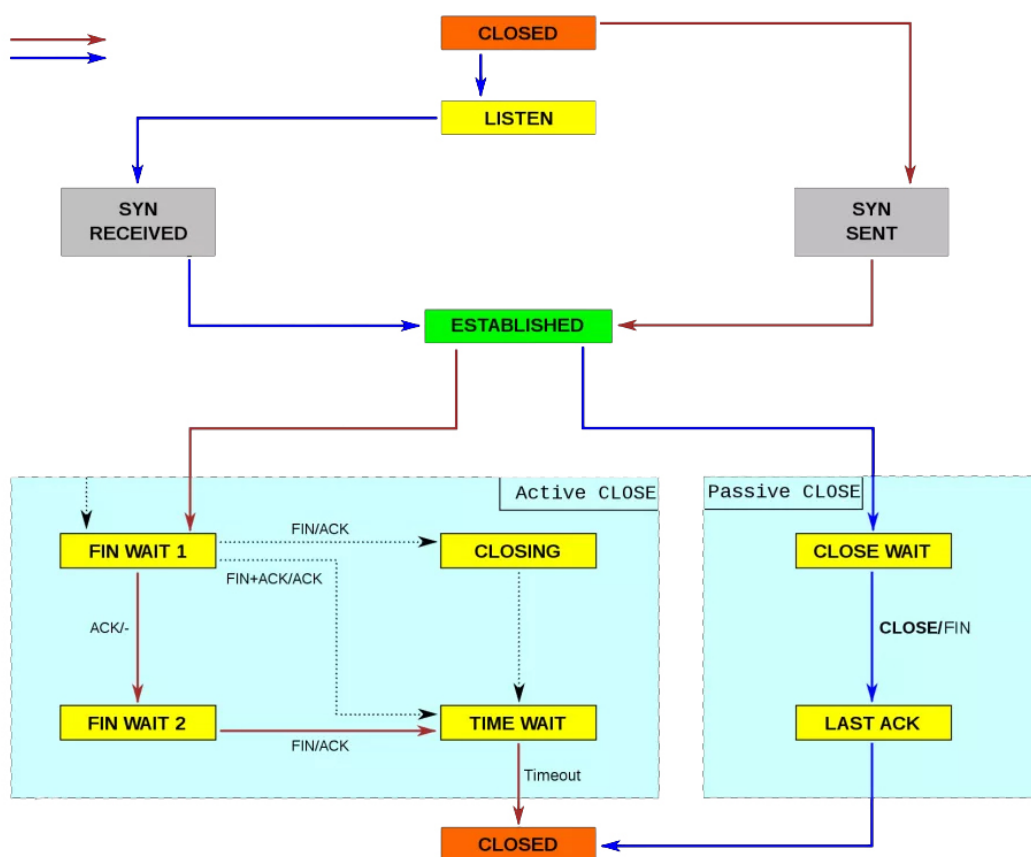


图6-2 TCP处理流程图

TCP (Transmission Control Protocol) ，即传输控制协议，是一种面向连接的、可靠的、基于字节流的传输层通信协议[64]。该协议早在1974年就被提出并被写进 **RFC (Request for Comments)** 中，自发布几十年来一直被不断优化和调整，如今已经是一个包含“可靠传输”，“拥塞控制”等多个功能的协议了 ([RFC 2581](#)中增加)。

在21世纪早期，我们因特网上的数据包有大约95%的包都使用了TCP协议，包括 HTTP/HTTPS, SMTP/POP3/IMAP、FTP等。当然，也包括一大部分网络游戏。我们如此偏爱于TCP就是因为他从协议层面上提供了许多非常重要的特性，如数据的可靠性、拥塞控制、流量控制等。这些可以让软件应用的开发者们无需担心数据丢失、重传等细节问题。

然而在游戏开发中，这些特性却可能是网络同步中的负担。在FPS游戏中，玩家每帧都在移动，我们期望这些数据在几毫秒内就能送达，否则就会对玩家产生干扰、影响游戏体验。因此对于FPS、RTS这种要求及时响应的游戏，TCP协议那些复杂的机制看起来确实有点华而不实。



图6-3 客户端位置要落后一些，与服务器有一定误差（守望先锋）

考虑到TCP协议非常复杂，这里只从几个关键的点来谈谈他的问题[65]。

1. 在TCP中，数据是通过字节流的方式发送的，但由于建立在IP协议上必须将字节流拆分成不同的包，默认情况下协议会将你的数据包缓冲，到达一定值才会发送。这样可能会出现游戏某个阶段你最后几个包明明已经执行了发送逻辑，但是由于缓冲机制被限制而无法到达。不过好在我们可以通过[TCP_NODELAY](#)来设置TCP立即刷新写入它的所有数据。
2. 其次，TCP的可靠数据传输响应并不及时。一旦数据包发生丢失或乱序，那么接收方就会一直等待这个数据的到来，其他新收到的数据只会被缓存在协议层，你在应用层根本获取不到任何数据也无法做任何处理。这个时候你可能要等超时重传机制响应后才能拿到重发的数据包，这时候可能已经过了几十毫秒。即使TCP拥有快速重传机制，仍然达不到理想的延迟效果。
3. 拥塞控制和流量控制不可控。TCP在网络环境比较差的条件下，会通过调整拥塞控制窗口大小来减少包的发送来降低吞吐量，这对于延迟敏感的游戏完全是无法接受的。同样，我们在应用层上面也无能为力。
4. 其他的还有一些的小问题，比如每个TCP的报头都需要包含序列号、确认号、窗口等数据结构，无形中增加了流量大小；TCP需要在端系统中维护连接状态，包括接收与发送缓存、拥塞控制参数等，在处理大量连接的消息时也更为繁琐和耗时。

TCP表头																																	
偏移	字节	0								1								2								3							
字节	比特	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	来源连接端口																目的连接端口															
4	32	序列号码																															
8	64	确认号码 (当ACK设置)																															
12	96	数据偏移				保留 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S S	S S Y	F I N	窗口大小															
16	128	校验和																紧急指针 (当URG设置)															
20	160	选项 (如果数据偏移 > 5。需要在结尾添加0。																															
...	...	...																															

图6-3 TCP报头

那么这些问题能解决么？也许能，但是从协议层面我们无能为力，因为TCP协议设计之初就不是为了及时响应，而另一个运输层协议UDP看起来比较符合我们的理念。

UDP(**U**ser **D**atagram **P**rotocol)，即用户数据包协议，是一个简单的面向数据报通信的协议[66]。该协议由David P. Reed在1980年设计并写入[RFC 768](#)中。顾名思义，UDP设计之初就是为了让用户可以自由的定义和传输数据，不需要建立链接、没有流量控制也没有拥塞控制，但是会尽可能快的将数据传输到目的IP和端口。

UDP报头																																	
偏移	字节	0								1								2								3							
字节	位	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	来源连接端口																目的连接端口															
4	32	报文长度																校验和															

图6-4 UDP报头

在上世纪90年代，Quake等游戏就开始使用UDP协议取代TCP进行数据同步，结果也很理想。除了游戏外，其他诸如视频、语音通信等领域也在广泛使用UDP，开发者们开始基于UDP创建自定义的Reliable UDP通信框架（QUIC、WbRTC、KCP、UDT等[67]），一些游戏引擎（如UE4）也将RUDP集成进来。随着网络带宽的提高，使用UDP代替TCP目测是一个趋势（参考Http3[68]）。

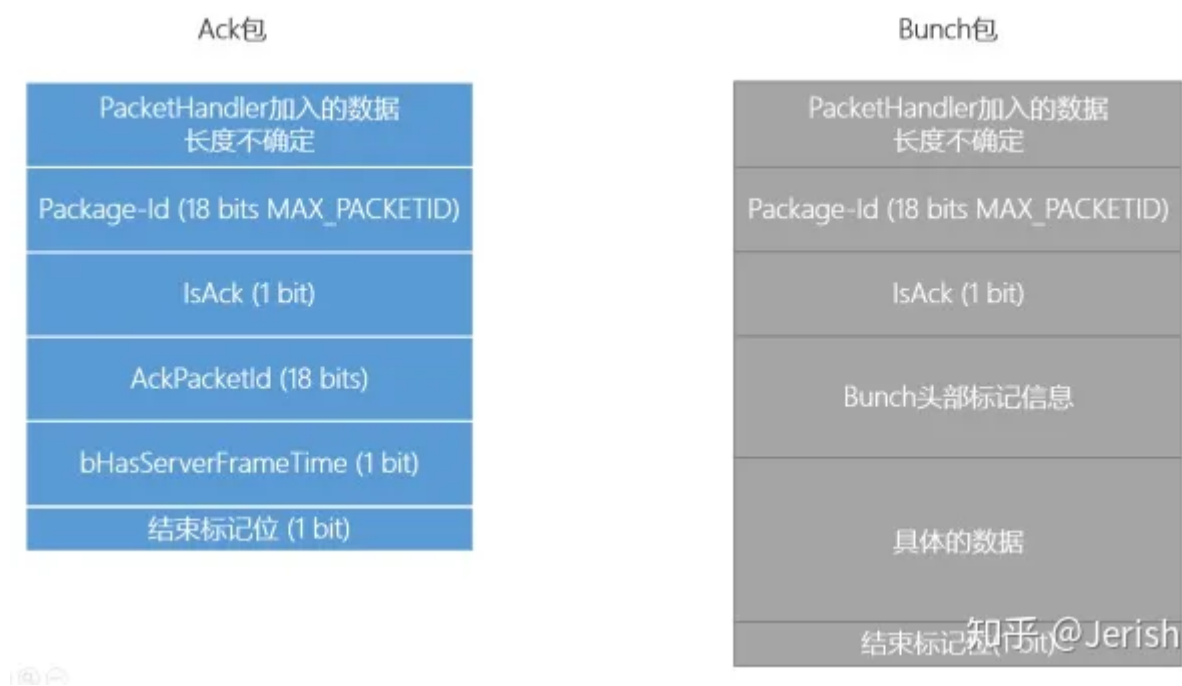


图6-5 虚幻引擎的UDP数据包，包含ACK标记位

虽然UDP很自由，但是需要开发者们自己写代码完善他。我们需要自己去写服务器客户端建立链接的流程，我们需要手动将数据分包，我们还需要自己实现应用层面的可靠数据传输机制。另外，UDP还有一个传输上的小劣势——当路由器上的队列已满时，路由器可以根据优先级决策在丢弃TCP数据包前先丢失UDP，因为他知道TCP数据丢失后仍然会进行重传。

总的来说，对于那些对延迟很敏感的游戏，UDP的传输模式更加适合而且弹性很大，同时他也可以胜任那些同步频率比较低的游戏，但是UDP的开发难度比较高，如果是自己从零开发确实有相当多的细节需要考虑，所以建议大家在已有的RUDP框架上进行优化。

七、常见同步优化技术

梳理完同步的发展历史，我们最后再来总结一下常见的网络同步优化技术。首先，提出一个问题，网络同步优化到底是在优化什么？

在单机游戏中，从我们按下按键到画面的响应中间经历了输入采样延迟、渲染流水线、刷新延迟、显示延迟等。而在一个网络游戏中，从我们按下按键到另一个机器收到指令，则会经历一个极为耗时的网络延迟（相比之下，单机的延迟可以忽略不计）。网络延迟其实也包括处理延迟、传输延迟（主要延迟）、排队延迟以及传播延迟，一般我们会将这些延迟统称为网络延迟，我们优化的目的就是想尽各种办法**降低或是抵消掉这个延迟**。

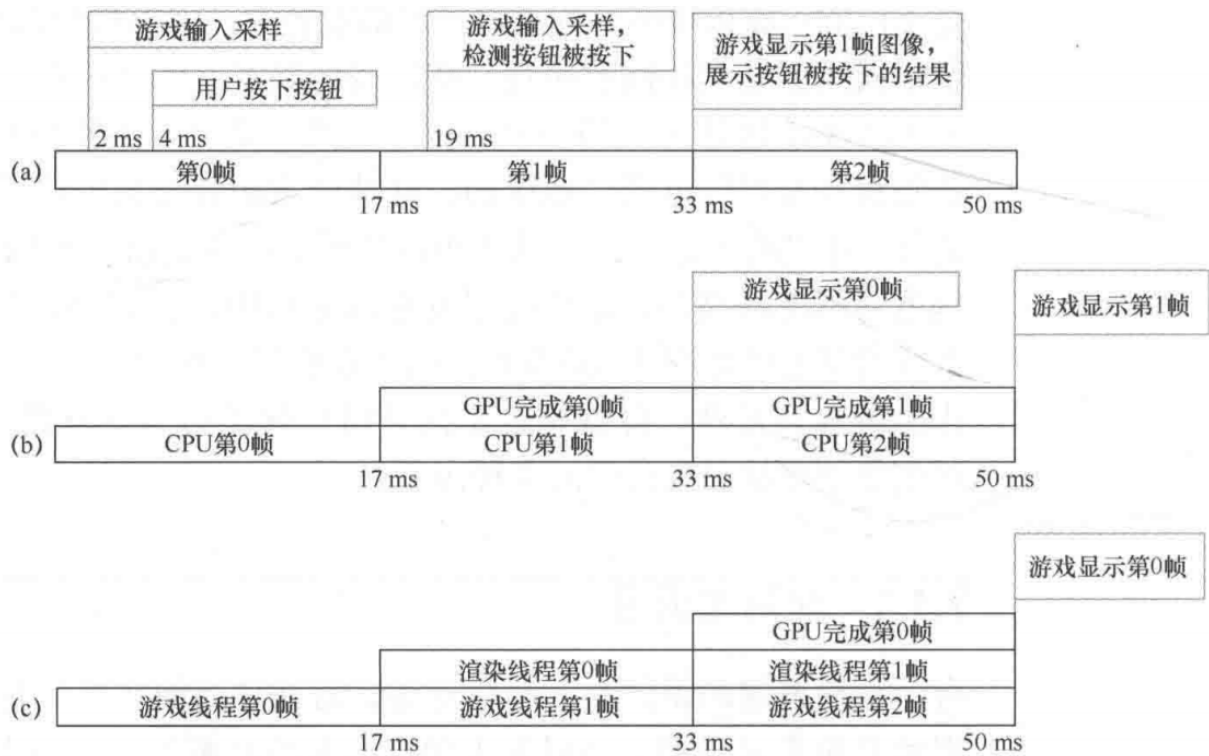


图7-1 单机游戏中的延迟

数据从客户端传输到服务器的一个来回称为一个RTT。在CS架构下，其实每个客户端的行为一直是领先于服务器1/2个RTT的，数据从客户端发送到服务器有一个1/2的RTT延迟，服务器处理后通知客户端又有一个1/2的RTT延迟。P2P架构下，由于没有权威服务器，我们可以省去1/2的RTT延迟，但是在目前的网络游戏中，为了对抗作弊行为以及容纳更多的玩家，我们不得不采用CS架构。

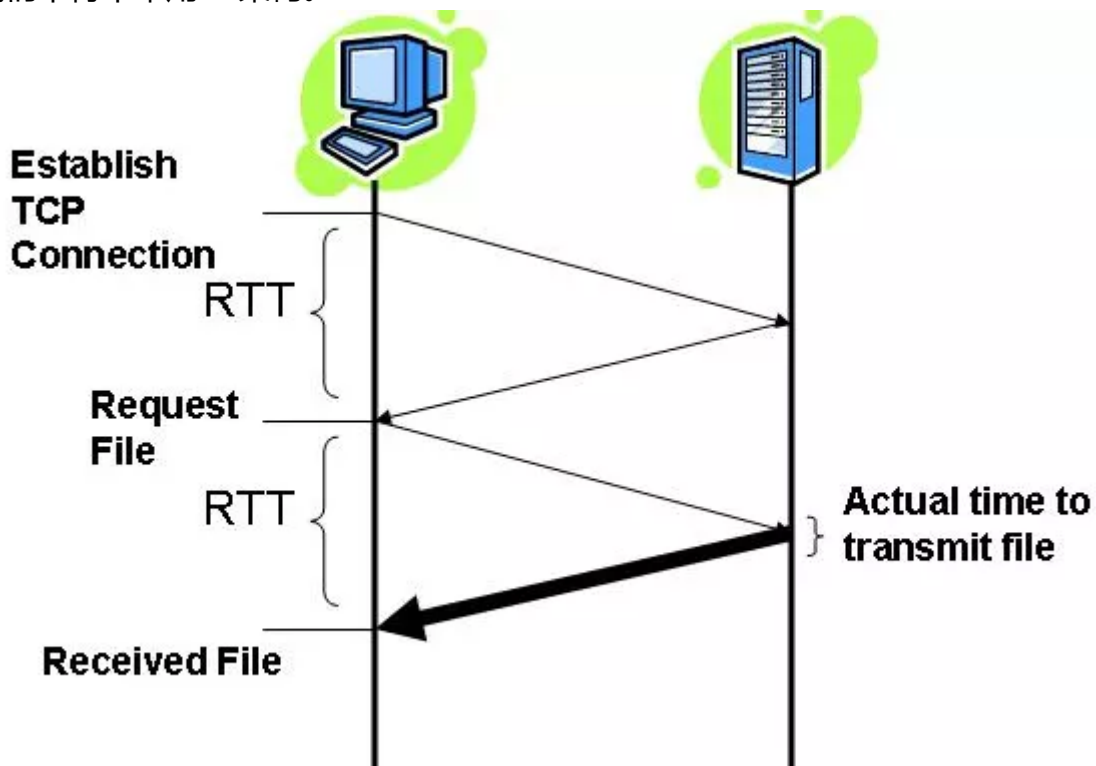


图7-2 网络延迟RTT示意

由于在网络游戏中，延迟是不可避免的，所以我们的优化手段就是如何减小这个延迟以及如何让玩家感受不到延迟。下面我会从**表现优化、延迟对抗、丢包对抗、带宽优化以及帧率优化**这几个方面来做一下总结，

1.表现优化（弱化玩家对延迟的感受）：

a.插值优化

通过内插值解决客户端信息离散更新的突变问题，通过外插值解决网络延迟过大收不到数据而卡顿的问题，两种方案并不冲突，可以同时采用。在具体应用时，我们可以使逻辑帧与渲染帧分离，这样在客户端没有收到数据的时候还可以继续更新渲染，也可以在二者不分离的情况只对渲染的模型进行插值，客户端收到权威数据后再进行整个对象的移动。

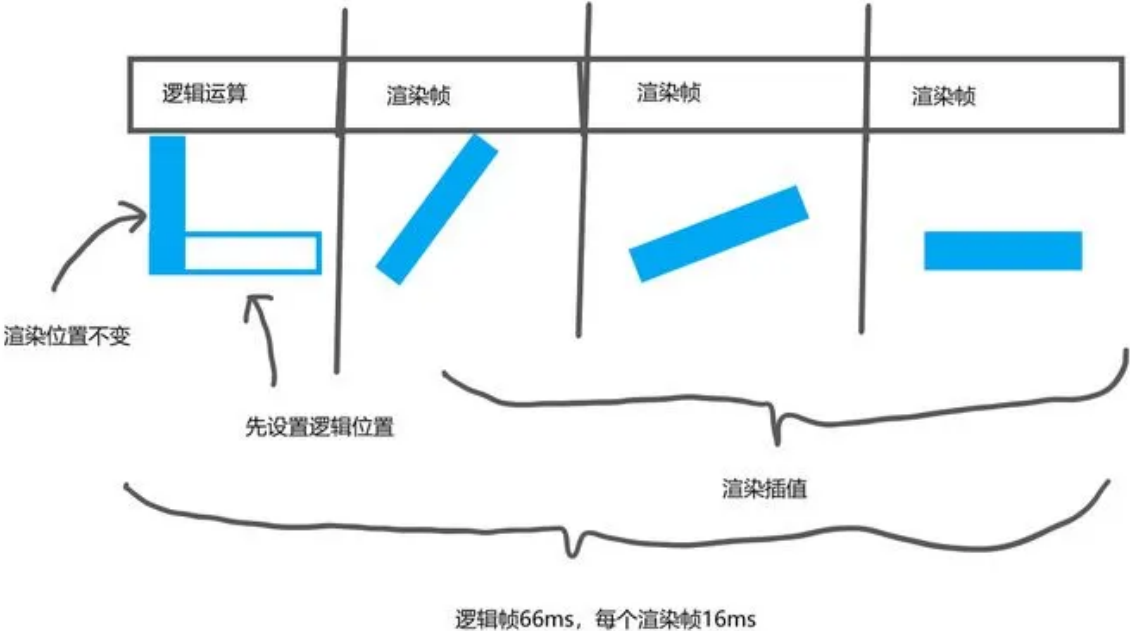


图7-3 逻辑与渲染分类

b.客户端预先执行+回滚

预测的目的是让玩家能在本地操作后立刻收到反馈，提升游戏体验，回滚是为了保证服务器的权威性。客户端预测包括位置预测以及行为预测两种，位置预测需要高频率的执行，因为移动在每帧都可能发生，而其他行为预测则相对低频一些，包括开枪、扔手雷、释放技能等。另外，对于延迟不太敏感的游戏（比如MMO），可以放宽校验条件（超过一定误差再纠正），这样即使降低服务器帧率客户端也不会有什么感觉。

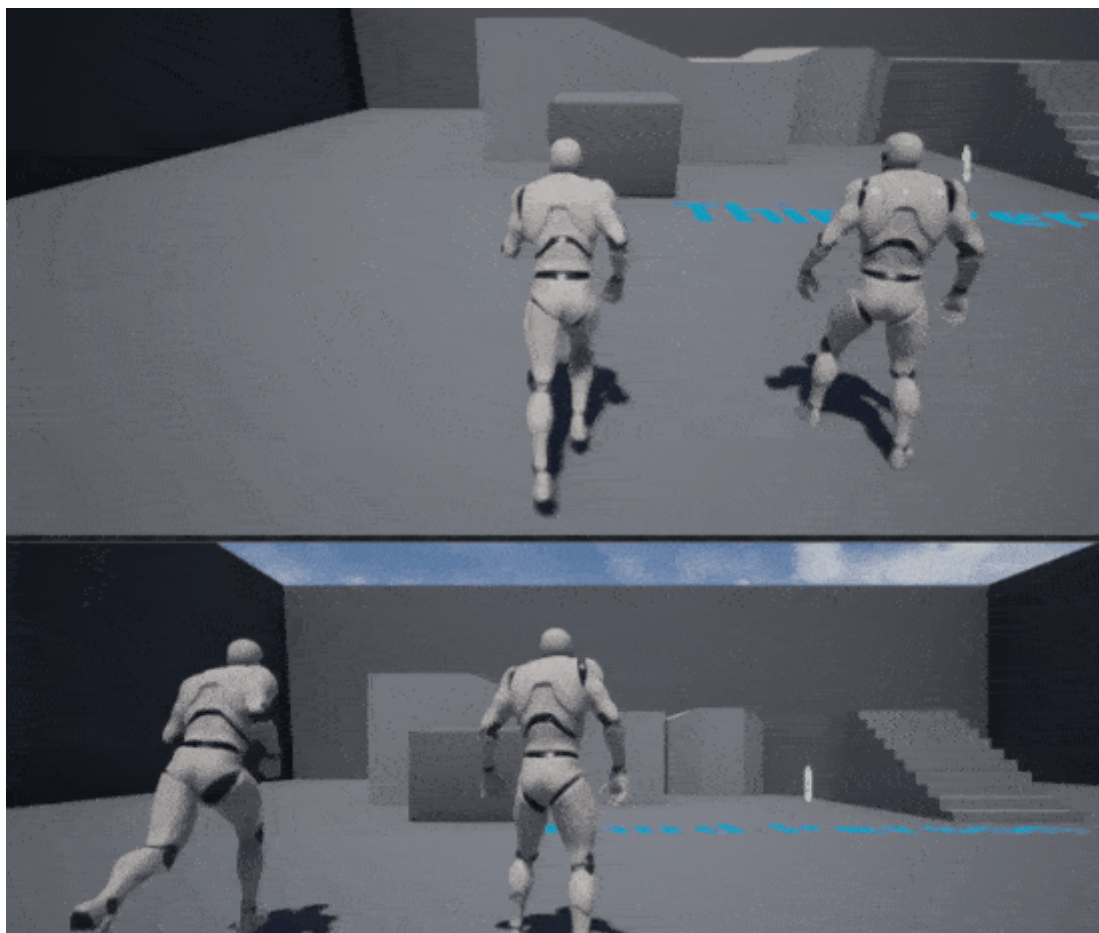


图7-4 仔细可以看出来本地的角色响应更快，同时利用插值解决位置突变问题

2.延迟对抗（弱化玩家对延迟的感受）：

a.延迟补偿

服务器（注意是服务器而不是客户端）记录一段时间内所有玩家的位置历史，在发生伤害计算时根据延迟对所有玩家角色进行位置的回滚与处理，可以尽量还原当时的场景。

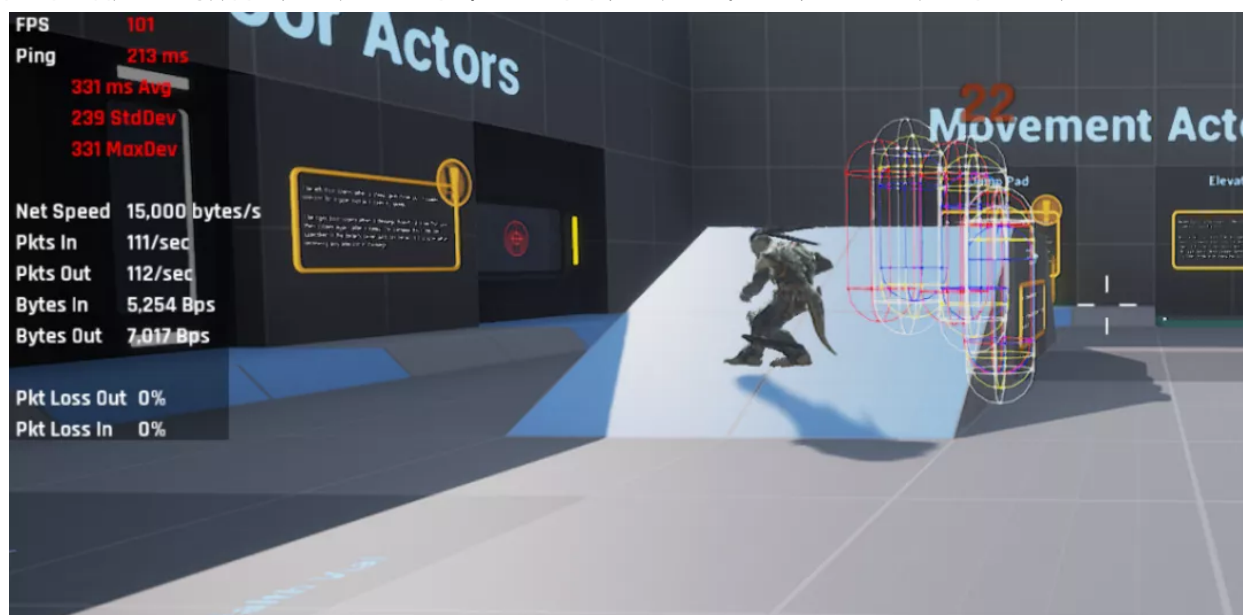


图7-5 角色身后的线表示延迟补偿回退的位置信息

b.命令缓冲区

把远端的数据缓存在一个buffer里面，然后按照固定频率从buffer里面取，可以解决客户端卡顿以及网络抖动问题。不过缓冲区与延迟是有冲突的，缓冲区越大，证明我们缓存的远端数据就越多，延迟就越大。

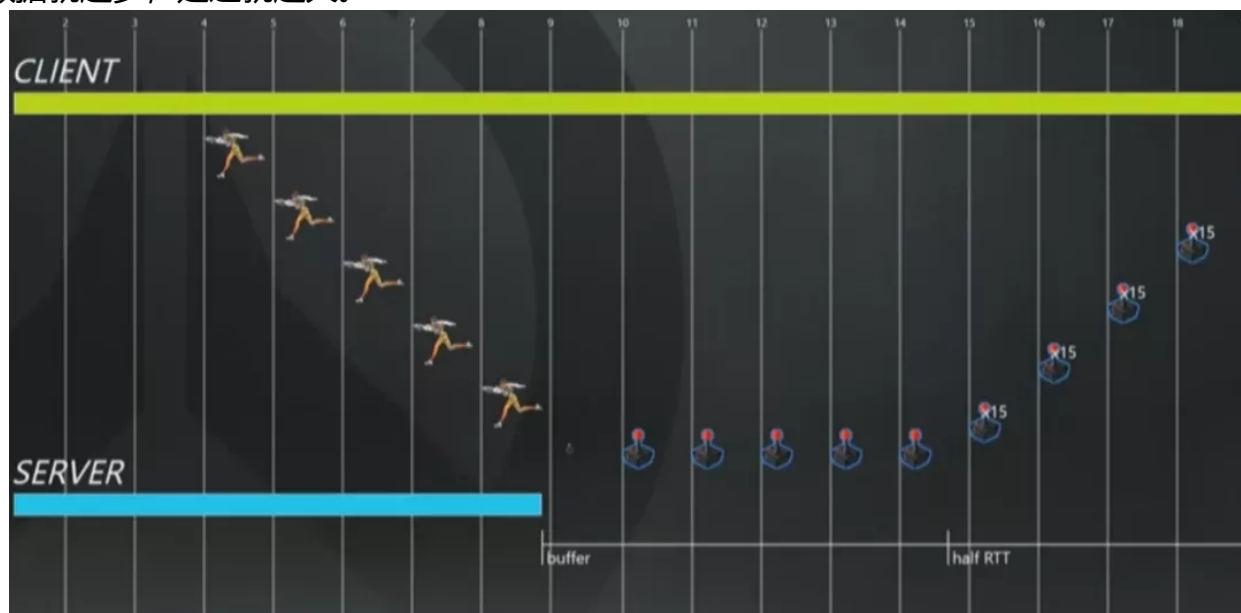


图7-6 守望先锋采用的Inputbuffer

c.从具体实现的技巧上对抗延迟

操作加一个前么时间，客户端释放技能等行为前有一个时间来抵消掉RTT的延迟。如无敌状态做一个过度动画，客户端播放动画后进入无敌，但是服务器可以在收到指令后直接进入无敌状态从而抵消延迟。在游戏Halo中，有很多类似的例子。比如在客户端玩家扔手雷的时候，我们可以在本地立刻播放扔手雷的动画并发送请求到服务器，然后服务器收到后不需要播放动画立刻生成手雷并同步，这样客户端真正扔出手雷的表现就是0延迟的。

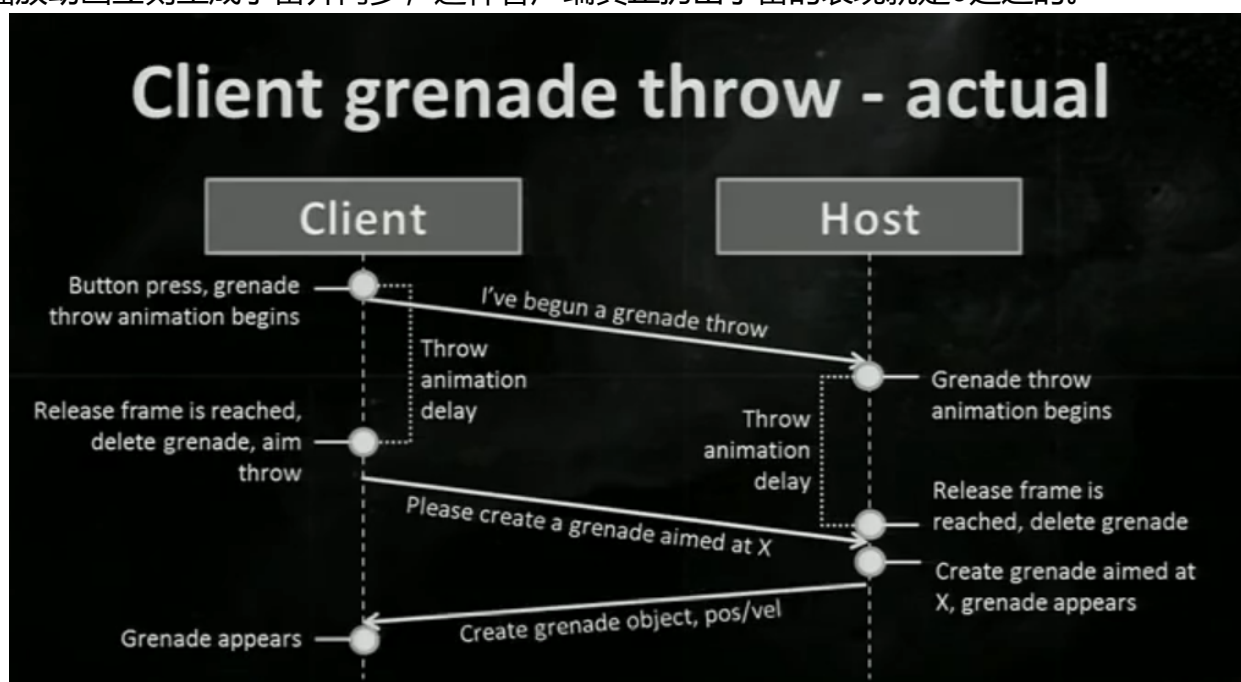


图7-7 光环中采用的扔手雷方案

3.丢包对抗（弱化玩家对延迟的感受）：

a.使用TCP而不是UDP

由于TCP不会丢包，对于延迟不敏感的游戏还是优先采取TCP来对抗丢包

b.冗余UDP数据包

一次性发送多个帧的数据来对抗丢包。UDP同步数据时经常容易丢包，我们虽然可以使用上层实现UDP的可靠性，但是像帧同步这种同步数据量比较小的游戏可以采用冗余UDP的方案，即后续的UDP包会冗余一定量的前面已发送的UDP包，这样即使丢失了部分包我们也能保证拿到完整的远端数据。

注：王者荣耀、守望先锋、火箭联盟等等游戏都使用类似的方案，该方案不仅仅适用于帧同步

4.带宽优化（减小延迟）：

带宽优化的目的是减小客户端以及服务器的同步压力，避免大量数据同时传输造成处理不过来，排队甚至是丢失。带宽优化是非常灵活且多变的，我们需要根据游戏的玩法来调整我们的优化行为。

a.同步对象裁剪

核心目的是根据相关性剔除那些不需要同步的对象（这里都是指在同一个服务器内），比如一个玩家距离我很远，我们的行为彼此不会互相影响，所以就不需要互相同步对方的数据。裁剪方式有非常多，常见的SOI（Spheres of Influence），静态区域（把场景划分为N个小区域，不在一个区域不同步），视锥裁剪（更多用于渲染），八叉树裁剪等。相关性还可能会涉及到声音等其他因素，需要根据自己项目来决定。

这里着重提一点AOI（Area Of Interest [69][70]），即根据玩家的位置维护一个动态的视野列表，视野外的对象会被完全忽略（能大幅的减少同步对象的遍历与比较）。其基本思想也是判断相关性，实现方式有很多，其中基于格子的空间划分算法是网络游戏中常见的实现方案。在虚幻引擎中，大世界同步框架ReplicationGraph[71]的核心思想也是如此。不过要注意的是，对于MMO这种可能有大量角色同时进行连续移动的游戏，视野列表频繁的增删查操作也可能对服务器造成一定的压力。

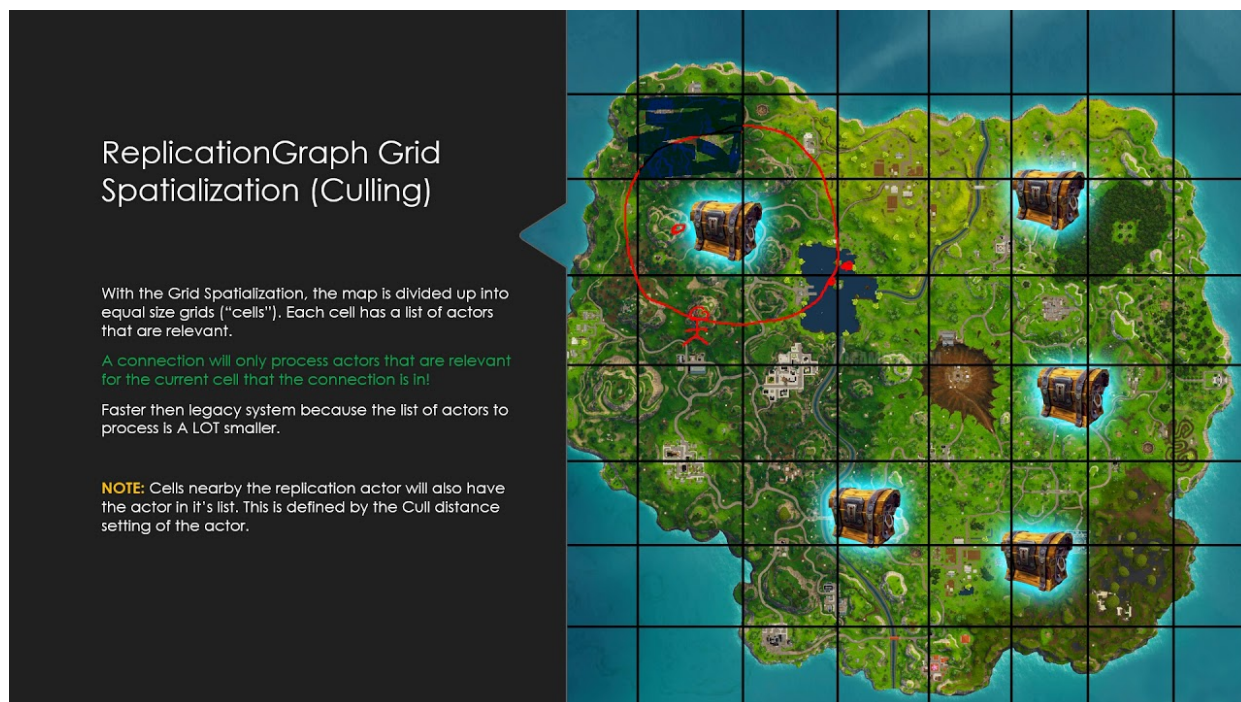


图7-8 ReplicationGraph示意

b.分区，分房间

对于大型MMO来说，这是常见的手段，将不同的玩家分散到不同的场景内（不同的服务器），这样减小服务器处理数据的压力，减小延迟。对于大世界游戏而言，不同服务器可能接管同一个地图不同区域的服务，其中的跨服数据同步比较复杂。

c.数据压缩与裁剪

坐标与旋转是我们常见的同步内容，但是很多数据其实是不需要同步的。比如对于大部分3D游戏角色的Pitch以及Roll是不会改变的，我们只要同步Yaw值即可。对于非第一人称游戏，我们可以接着把四个字节float类型的Yaw压缩到两个字节的uint16里面，玩家根本不会有什么体验上的差异。类似的方法可以应用到各种同步数据里面。

此外，在状态同步里面，我们可以采用增量发送来减少数据量，即第一次发送完整的数据信息后只发送哪些发生过变化的数据，这可以大大减少网络同步的流量。

参考：[Exploring in UE4 网络同步原理深入](#) 虚幻引擎的属性同步系统
[《守望先锋》回放技术-阵亡镜头、全场最佳和亮眼表现](#) 回放增量同步处理

d.减少遍历以及更细力度的优化

在Halo以及虚幻引擎里面都会对同步对象做优先级划分，发送频率调整等。在状态同步中，我们还需要合适的手段来快速定位发生变化的数据，如属性置脏、利用发射减少非同步属性的遍历等。进一步的，我们还可以根据客户端的类型以及信息作出更细致的同步信息过滤以及设置优先级，比如对同步属性进行优先级划分等（目前还没有见到过粒度如此细致的，但理论上是可行的）。

5.帧率优化（减小延迟）：

帧率优化是一个重要且复杂的难题，涉及到方方面面的技术细节，这里主要针对网络同步相关内容做一些分析。相比单机游戏，网游需要同时考虑客户端与服务器的帧率，这并不是单纯地提升帧率的问题，如何优化与平衡是一个很微妙的过程。

a.提升帧率

这个不用多说，帧率低就意味着卡顿，玩家的体验就会很差。不同游戏的性能瓶颈都可能不一样，包括内存问题（GC、频繁的申请与释放）、IO（资源加载、频繁的读写文件，网络包发送频率过大，数据库读取频繁）、逻辑问题（大量的遍历循环，无意义的Tick，频繁的创建删除对象，过多的加锁，高频率的Log）、AI（寻路耗时[72]）、物理问题（复杂模拟，碰撞次数过多）、语言特性（脚本语言比较费时）等，客户端相比服务器还有各种复杂的渲染问题（Drawcall太多，半透明，动态阴影等）。这些问题需要长期的测试与调试，每个问题涉及到的具体细节可能都有所不同，需要对症下药才行。

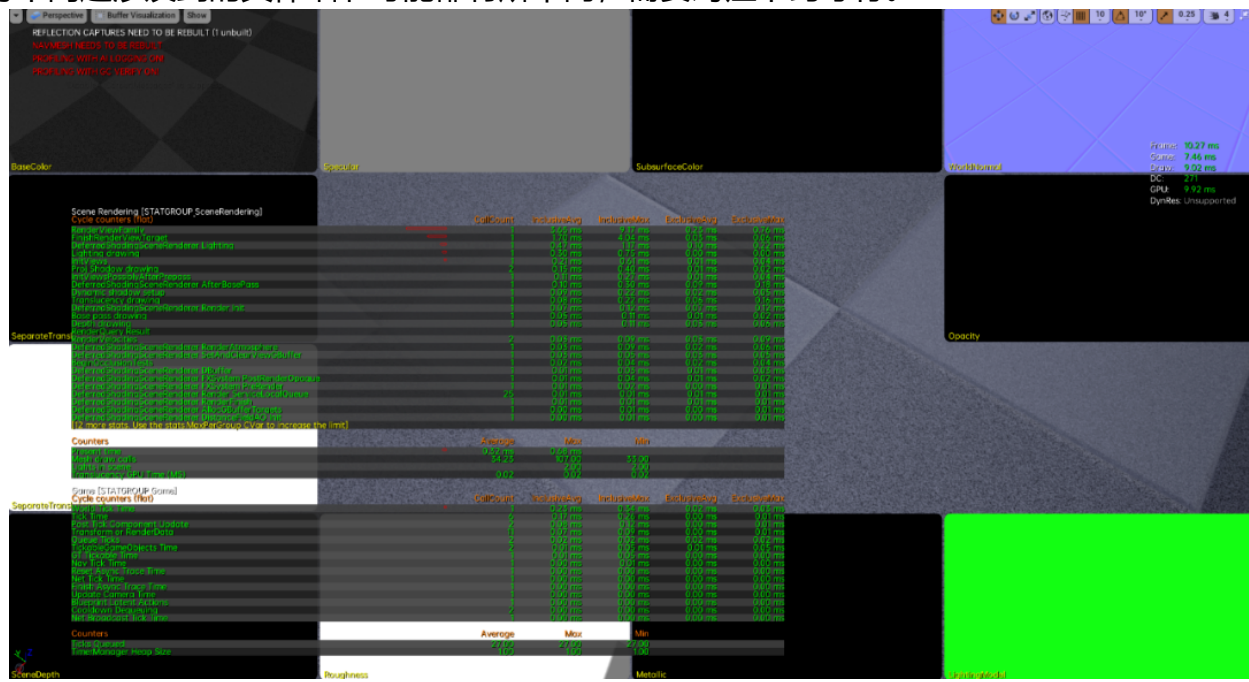


图7-9 虚幻引擎中的性能数据展示

b.保持帧率稳定与匹配

假如你的客户端与服务器帧率已经优化到极致，你也不能任其自由变化。首先，要尽量保持服务器的帧率稳定（减少甚至是消除玩家比赛时的所有潜在的卡顿问题），考虑一款对延迟比较敏感的射击游戏，如果你的客户端在开枪时遇到了服务器卡顿，那么就可能造成校验失败，导致客户端看到的行为与服务器行为不一致。其次，还要保持客户端与服务器的帧率匹配。对于延迟不敏感的游戏，考虑到玩家的体验以及服务器的压力，客户端的帧率可以高于服务器多倍，但是这个比例是需要通过实际的测试来调整。而对于延迟敏感的游戏，我们一般需要尽量让服务器的帧率接近客户端，这样服务器才能更及时的相应，减少延迟带来的误差。此外，我们也不能让客户端的帧率无限提高，对于某些同步算法，客户端与服务器过高

的帧率差异可能造成不断的拉回卡顿。所以，很多游戏会采取锁帧的方式来保证游戏的稳定性。

c.计算压力分担

对于MMO这种服务器压力比较大的游戏，我们通常会考虑把一部分计算资源转交给客户端去计算（甚至是计算后再返还给服务器），比如物理运算、自动寻路、AI逻辑计算等。其实将这种方式使用到极致的例子就是帧同步，服务器只做一些简单的校验即可。

总的来说，网络同步优化是一个长期的不断试错的过程，我们需要合理的利用计算机资源，把最重要的资源用在最重要的功能上面，减少重复的计算与流程，并需要配合一些经验和技巧来规避那些不好解决的问题。

八、总结

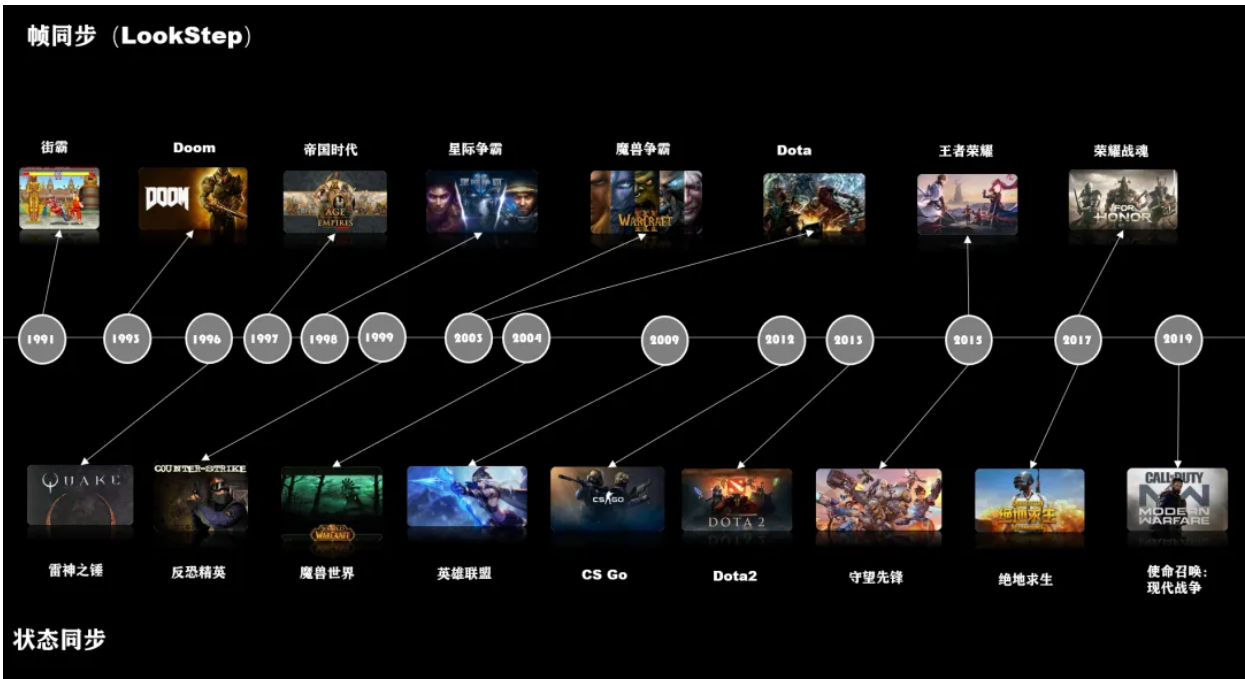


图8-1 历史上“帧同步”和“状态同步”的网络游戏对比

属性	帧同步 (LockStep)	状态同步
确定性	严格确定，每帧逻辑上完全一致	理论上也可以做到逻辑一致，但是一般允许微小的误差，会定时纠正不正确的数据
表现与响应速度	传统的确定性帧同步需要等待其他客户端的消息全部到达，所以响应会比较慢。不过采用乐观帧同步可以做到本地立刻响应，后面需要在与服务器不一致的时候进行回滚因此需要处理好序列化和反序列化的逻辑	一般会做预测处理，可以做到立刻响应。对于响应要求不高的游戏，也可以不做预测。这种情况下响应时间也是一个RTT，与确定性帧同步一样
带宽与流量	人数较少的情况带宽占用极低，只需要发送Input信息。但是人数多的情况每一帧都要发送所有人的Input信息（每帧，而且不能做数据的过滤和裁剪），不适合MMO	需要发送位置、行为等各种状态数据，占用带宽比较高。但是可以通过压缩、裁剪、增量等多种方式进行优化。人数较少的时候还不如帧同步节省流量。
网络延迟适应性	要求较低的延迟。如果延迟较高，所有玩家体验都会不好。即使采用乐观帧同步优化，在高延迟下也很容易产生卡顿等情况	较高，因为可以比较方便的做各种插值优化。当然高延迟下，也容易产生位置突变
开发难度	初期开发简单，框架实现容易，但是后期解决bug和完善系统很困难。也方便自测，但是由于浮点数、各种逻辑错误（随机数、执行顺序等）很容易写出计算结果不一致的代码，而且问题不会及时暴露，排查很困难	框架搭建很复杂，一般客户端与服务器采用同一套代码，每一个功能都需要客户端与服务器联调。属性同步的策略有很多值得斟酌和优化的空间。错误排查相对简单，但是也容易出现各种时序问题
玩家数量	适合少量的玩家，不适合MMO	对玩家数量没有限制，可多可少
跨平台	不适合跨平台，由于对结果的一致性要求严格，浮点数问题不容易解决。可以使用定点数，来将误差尽量减小到一个可以接受的范围。同时可以考虑定时的结果纠正	适合。由于服务器实时的传输权威数据，所以不太在意微小的误差
反外挂	P2P架构不好反作弊，因为数据全部在客户端上，但是只要引入服务器就比较容易。服务器跑战斗逻辑或者采取一些校验手段（比如各个客户端的结果）都可以比较理想的解决常见的外挂。但是透视、全图视野这种一般防不了	理论上可以做到较好的反外挂效果，与服务器加入的校验机制有关。但是一样防不了透视外挂
中途加入与断线重连	比较复杂。传统帧同步要求各个端从一帧开始就严格保持一致。不过目前优化的帧同步可以在断线的时候通过快速播放服务器同步的帧数据来快速追上	容易。由于实时记录了各个对象的状态信息，所以在重连的时候直接创建这些对象并同步这些信息即可
性能（客户端）	客户端需要跑完整的游戏逻辑，同时还要执行渲染逻辑，开销比较大。	可以灵活的优化。客户端可以跑较少的逻辑，不过现在为了及时响应以及使用客户端的物理引擎，很多游戏都会在游戏端也跑战斗逻辑（具体看游戏类型）。
性能（服务器）	如果不跑战斗逻辑那么服务器开销非常低，只需要做一些简单的校验即可。但是有的服务器为了更好地反外挂和校验，也会在服务器上跑逻辑。	可以灵活的优化。服务器要跑很多逻辑相对压力比较大，可以使用AOI等方式优化。
回放（离线）	由于本身就是通过收集所有玩家的Input信息进行逻辑推进，所以天然的支持回放。而且回放文件很小	可以支持回放，但是逻辑处理比较麻烦，需要不断记录状态信息同时回放的时候要读取合适的时间。回放文件较大
回放（实时）	比较复杂，客户端需要本地对全场状态进行序列化，才能回到目标时间。播完回放后还需要加速追上实时游戏状态	相对容易，可以方便的记录快照信息并按照录制内容随时播放
第三方库的引用	困难，需要保证第三方库的确定性	比较灵活自由

图8-2 “帧同步”和“状态同步”的使用场景与特点对比

我们从最开始的网络游戏架构谈起，按照时间线梳理了近几十年“帧同步”与“状态同步”的发展历程，并讲述了各种同步技术以及优化方案。虽然网络同步是游戏中的技术，但其本质还是计算机数据的同步。无论是Lockstep还是TimeWarp，最初都是用于计算机系统通信的技术，只不过应用场景从一台机器的内部通信转变为多台机器的通信，从传统的应用转移到网络游戏上面。

游戏的类型会影响到网络同步的解决方案，也会影响到项目的整体架构，所以我们在制作一款网络游戏前要事先做好需求分析并确定网络同步方案。同时也要意识到，网络同步延迟是不可消除的，除了算法层面的优化外还可以从实现技巧上来规避一些难题。

到此，历时半年多的网络同步系列终于迎来完结。不过网络技术还在进步，历史也还在前行，让我们一同关注同步技术的发展和变化，期待未来的游戏世界。

后记：笔者花费了近一年的时间才完成了《网络同步在历史上的发展变化》系列文章，期间查阅无数资料。目前该系列可以认为是整个网络上最为对游戏同步讲解最为全面的文章，如果发现文章有什么问题或者想进行技术交流，可以通过以下方式联系作者Jerish。

知乎连接：[Jerish](#)、微信公众号：《游戏开发那些事》、QQ群：875867499

微信公众号：

游戏开发那些事



游戏开发那些事(学…
QQ 群号：875867499



参考文献:

- 1 [1]WIKI "History of video games". Available:https://en.wikipedia.org/wiki/History_of_video_games[Accessed: 2020-03-24]
- 2 [2]T.A. Funkhouser."RING: A Client-Server System for Multi-User Virtual Environments", In Proc. 1995 Available:<https://dl.acm.org/doi/pdf/10.1145/199404.199418>[Accessed: 2020-03-24]
- 3 [3]Eric Cronin, Burton Filstrup Anthony R. Kurc, Sugih Jamin,"An Efficient Synchronization Mechanism for Mirrored Game Architectures", 2004. Available: <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.87.6043&rep=rep1&type=pdf>[Accessed:2020-03-24]
- 4 [4]WIKI "Lockstep (computing)". Available: [https://en.wikipedia.org/wiki/Lockstep_\(computing\)](https://en.wikipedia.org/wiki/Lockstep_(computing))[Accessed: 2020-03-24]
- 5 [5]JMP van Waveren, "The DOOM III Network Architecture" ,2006. Available:http://fabiansanglard.net/doom3_documentation/The-DOOM-III-Network-Architecture.pdf[Accessed: 2020-03-24]
- 6 [6]Jeff S. Steinrnan,"BREATHING TIME WARP" May 1993. Available:<https://dl.acm.org/doi/pdf/10.1145/174134.158473>[Accessed:2020-03-24]
- 7 [7]Christophe DIOT, Laurent GAUTIER, "A Distributed Architecture for Multiplayer Interactive Applications on the Internet", IEEE, 1999. Available: <https://www.cs.ubc.ca/~krasic/cpsc538a-2005/papers/diot99distributed.pdf>[Accessed: 2020-03-24]
- 8 [8]Nathaniel E. Baughman, Brian Neil Levine, "Cheat-Proof Payout for Centralized and Distributed Online Games", IEEE INFOCOM, 2001. Available: <http://forensics.umass.edu/pubs/baughman.infocom01.pdf>[Accessed: 2020-03-24]
- 9 [9]Mark Terrano,Paul Bettner "Network Programming in Age of Empires and Beyond" GDC 2001. Available:https://zoo.cs.yale.edu/classes/cs538/readings/papers/terrano_1500arch.pdf[Accessed: 2020-03-24]
- 10 [10]Ho Lee, Eric Kozlowski, Scott Lenker, Sugih Jamin, "Multiplayer Game Cheating Prevention With Pipelined Lockstep Protocol", 2002. Available: <http://www.ekozlowski.com/assets/multiplayer-game-cheating-prevention.pdf>[Accessed: 2020-03-24]
- 11 [11]Dacid Jefferson, Henry Sowizral "Fast concurrent simulation using the time wrap mechanism " 1982. Available: <https://www.rand.org/content/dam/rand/pubs/notes/2007/N1906.pdf>[Accessed: 2020-03-24]
- 12 [12]J. S. Steinman, J. W. Wallace, D. Davani, and D. Elizandro. "Scalable distributed military simulations using the SPEDES object-oriented simulation framework". In Proc. of Object-Oriented Simulation Conference (OOS'98), pages 3-23, 1998.
- 13 [13]云风 "lockstep网络游戏同步方案" Available:<https://blog.codingnow.com/2018/08/lockstep.html>
- 14 [14]Skywind "再谈网游同步技术" Available: <http://www.skywind.me/blog/archives/1343#more-1343>

- 15 [15]DonaldW "网络游戏同步技术概述"
Available:<https://zhuanlan.zhihu.com/p/56923109>
- 16 [16]Gordon "帧同步联机战斗(预测, 快照, 回滚)" Available: <https://zhuanlan.zhihu.com/p/38468615>
- 17 [17]zhepama "帧同步的相关问题"Available:<http://www.igiven.com/dotnet/lock-step/>
- 18 [18]kisence"关于帧同步和网游游戏开发的一些心得"Available:<https://www.kisence.com/2017/11/12/guan-yu-zheng-tong-bu-de-xie-xin-de/>
- 19 [19]Glenn Fiedler "Deterministic Lockstep" Available:https://gafferongames.com/post/deterministic_lockstep/
- 20 [20]Maksym Kurylovych "Lockstep protocol" Available:
<http://ds.cs.ut.ee/courses/course-files/Report%20-2.pdf>
- 21 [21] Glenn Fiedler, "What Every Programmer Needs To Know About Game Networking A short history of game networking techniques", 2010. Available: https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/
- 22 [22]"State Synchronization's Role in High Availability"
Available:[http://etutorials.org/Networking/Check+Point+FireWall/Chapter+13..+High+Availability/State+Synchronization+s+Role+in+High+Availability/\[Accessed:2020-07-17\]](http://etutorials.org/Networking/Check+Point+FireWall/Chapter+13..+High+Availability/State+Synchronization+s+Role+in+High+Availability/[Accessed:2020-07-17])
- 23 [23]WIKI, "Check Point VPN-1" Available: https://en.wikipedia.org/wiki/Check_Point_VPN-1[Accessed:2020-07-17]
- 24 [24]Check Point Documentation, "Synchronizing Connections in the Cluster"
Available:https://sc1.checkpoint.com/documents/R80.10/WebAdminGuides/EN/CP_R80.10_ClusterXL_AdminGuide/html_frameset.htm?topic=documents/R80.10/WebAdminGuides/EN/CP_R80.10_ClusterXL_AdminGuide/7288[Accessed:2020-07-17]
- 25 [25]id-Software, "GitHub Game Source Code" Available:<https://github.com/id-Software> [Accessed:2020-07-17]
- 26 [26]FABIEN SANGLARD, " FABIEN SANGLARD'S WEBSITE With GameSource Code Analysis " Available:<https://fabien DEGLARD.net/> [Accessed:2020-07-17]
- 27 [27] Mark Frohnmayer, Tim Gift, "The TRIBES Engine Networking Model or How to Make the Internet Rock for Multi player Games", 1998. Available: <http://www.gamedevs.org/uploads/tribes-networking-model.pdf>[Accessed:2020-07-17]
- 28 [28]WIKI, "Client-Side Prediction" Available:https://en.wikipedia.org/wiki/Client-side_prediction [Accessed:2020-07-17]
- 29 [29]id-Software, "The Quake 2 Networking Data Flow"
Available:http://www.gamers.org/dEngine/quake2/Q2DP/Q2DP_Network/Q2DP_Network.html#toc4 [Accessed:2020-07-17]
- 30 [30]WIKI, "QuakeWorld" Available:<https://en.wikipedia.org/wiki/QuakeWorld>[Accessed:2020-03-24]
- 31 [31]Gabriel Gambetta, " Client-Server Game Architecture"
Available:<https://www.gabrielgambetta.com/client-side-prediction-server-reconciliation.html>[Accessed:2020-07-17]
- 32 [32]Dacid Jefferson, Henry Sowizral "Fast concurrent simulation using the time wrap mechanism "

1982. Available: <https://www.rand.org/content/dam/rand/pubs/notes/2007/N1906df>[Accessed:2020-07-17]
- 33 [33]M. Damitio S. J. Turner, "Comparing the Breathing Time Buckets Algorithm and the Time Warp Operating System on a Transputer Architecture" January 1999. Available:https://www.researchgate.net/publication/2763616_Comparing_the_Breathing_Time_Buckets_Algorithm_and_the_Time_Warp_Operating_System_on_a_Transputer_Architecture[Accessed:2020-07-17]
- 34 [34]Jim Greer, Zack Booth Simpson, "Minimizing Latency in RealTime Strategy Games" Game Programming Gems 3 chapter 5.1, 2001.
- 35 [35]]WIKI, "Network Time Protocol" Available:https://en.wikipedia.org/wiki/Network_Time_Protocol[Accessed:2020-07-17]
- 36 [36]WIKI, "Interpolation" Available:<https://en.wikipedia.org/wiki/Interpolation>[Accessed:2020-07-17]
- 37 [37]WIKI, "Extrapolation" Available:<https://en.wikipedia.org/wiki/Extrapolation>[Accessed:2020-07-17]
- 38 [38]Jesse Aronson, "Dead Reckoning: Latency Hiding for Networked Games" September 19, 1997. Available:https://www.gamasutra.com/view/feature/131638/dead_reckoning_latency_hiding_for_.php[Accessed:2020-07-17]
- 39 [39]梁白鸥等, "Dead Reckoning技术在网络游戏中的应用" 2007. Available:<http://www.arocmag.com/getarticle/?aid=2f665567e92cf534>[Accessed:2020-07-17]
- 40 [40]Yahn W. Bernier, "Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization" 2001. Available:
41 https://developer.valvesoftware.com/wiki/Latency_Compensating_Methods_in_Client/Server_In-game_Protocol_Design_and_Optimization[Accessed:2020-07-17]
- 42 [41]Eric Cronin, Burton Filstrup Anthony R. Kurc, Sugih Jamin, "An Efficient Synchronization Mechanism for Mirrored Game Architectures", 2004. Available: <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.87.6043&rep=rep1&type=pdf>[Accessed:2020-07-17]
- 43 [42]David Aldridge, "I Shot You First: Networking the Gameplay of HALO: REACH", GDC, 2011. Available: <https://www.bilibili.com/video/BV1Vt4y127op>[Accessed:2020-07-17]
- 44 [43]Epic Games, " UnrealEngine: Networking and Multiplayer". Available: <https://docs.unrealengine.com/en-US/Gameplay/Networking/Overview/index.html>[Accessed:2020-07-17]
- 45 [44]Jerish, "使用虚幻引擎4年, 我想再谈谈他的网络架构". Available: <https://zhuanlan.zhihu.com/p/105040792>[Accessed:2020-07-17]
- 46 [45]Timothy Ford, " 'Overwatch' Gameplay Architecture and Netcode", GDC, 2018. Available: <https://www.bilibili.com/video/av44410490>[Accessed:2020-07-17] (翻译链接: <https://gameinstitute.qq.com/community/detail/114516>)
- 47 [46]Philip Orwig, " Replay Technology in 'Overwatch': Kill Cam, Gameplay, and Highlights", GDC, 2018. Available: <https://www.bilibili.com/video/BV1aA41147bY>[Accessed:2020-07-17] (翻译链接: <https://gameinstitute.qq.com/community/detail/115186>)
- 48 [47]WIKI, "Entity component system". Available: https://en.wikipedia.org/wiki/Entity_component_system[Accessed:2020-07-17]

49 [48] WIKI, "Pong", WIKI, 2020. Available: <https://en.wikipedia.org/wiki/Pong> [Accessed: 2020-12-12]

50 [49] Tony Wang, "游戏物理模拟简史", 知乎, 2020. Available: <https://zhuanlan.zhihu.com/p/106977617> [Accessed: 2020-12-12]

51 [50] Theraot, "How can I perform a deterministic physics simulation?", GameDev Stackexchange, 2019. <https://gamedev.stackexchange.com/questions/174320/how-can-i-perform-a-deterministic-physics-simulation> [Accessed: 2020-12-12]

52 [51] Yossi Kreinin, "Consistency: how to defeat the purpose of IEEE floating point", Personal Blog, 2008. Available: <http://yosefk.com/blog/consistency-how-to-defeat-the-purpose-of-ieee-floating-point.html> [Accessed: 2020-12-12]

53 [52] Glenn Fiedler, "Floating Point Determinism", Personal Blog, 2010. Available: https://gafferongames.com/post/floating_point_determinism/ [Accessed: 2020-12-12]

54 [53] NVIDIA, "NVIDIA PhysX SDK 3.4.0 Documentation Determinism", NVIDIA, 2020. Available: <https://docs.nvidia.com/gameworks/content/gameworkslibrary/physx/guide/Manual/BestPractices.html> [Accessed: 2020-12-12]

55 [54] MelvMay, "How much deterministic is Physics from Unity3d In 2019?", Unity Forum, 2020. Available:

56 <https://forum.unity.com/threads/how-much-deterministic-is-physics-from-unity3d-in-2019.711311/> [Accessed: 2020-12-12]

57 [55] Unity, "Burst User Guide", Unity Manual, 2020. Available: https://docs.unity3d.com/Packages/com.unity.burst@1.0/manual/index.html?_ga=2.60059693.1096806956.1607653832-2097754989.1600740353 [Accessed: 2020-12-12]

58 [56] Glenn Fiedler, "Introduction to Networked Physics", Personal Blog, 2014. Available: https://gafferongames.com/post/introduction_to_networked_physics/ [Accessed: 2020-12-12]

59 [57] Glenn Fiedler, "Physics for Game Programmers : Networking for Physics Programmers", 2018. Available: <https://www.gdcvault.com/play/1022195/Physics-for-Game-Programmers-Networking> [Accessed: 2020-12-12]

60 [58] Glenn Fiedler, "UnityDemo: Networked Physics in Virtual Reality: Networking a stack of cubes with Unity and PhysX", 2018. Available: <https://github.com/fbsamples/oculus-networked-physics-sample/> [Accessed: 2020-12-12]

61 [59] Matt Delbosc, "Replicating Chaos Vehicle Replication in Watch Dogs 2", GDC, 2017. Available: <https://www.bilibili.com/video/BV1KA41187jk> [Accessed: 2020-12-12]

62 [60] Ned, "手游中载具物理同步的实现方案", 腾讯游戏学院, 2018. Available: <https://gameinstitute.qq.com/knowledge/100044> [Accessed: 2020-12-12]

63 [61] Funny David, "看门狗2的载具同步(翻译)", 知乎, 2019. Available: <https://zhuanlan.zhihu.com/p/95560180> [Accessed: 2020-12-12]

64 [62] Jared Cone, "It IS Rocket Science! The Physics of 'Rocket League' Detailed", GDC, 2018. Available: <https://www.bilibili.com/video/av44416219> [Accessed: 2020-12-12]

65 [63] Glenn Fiedler, UDP vs. TCP https://gafferongames.com/post/udp_vs_tcp/ [Accessed: 2020-12-12]

- 66 [64]WIKI, "Transmission Control Protocol", WIKI, 2020. Available: https://en.wikipedia.org/wiki/Transmission_Control_Protocol [Accessed:2020-12-12]
- 67 [65]Draveness, "为什么 TCP 协议有性能问题", Personal Blog, 2020. Available: <https://draveness.me/whys-the-design-tcp-performance/> [Accessed:2020-12-12]
- 68 [66]WIKI, "User Datagram Protocol", WIKI, 2020. Available: https://en.wikipedia.org/wiki/User_Datagram_Protocol [Accessed:2020-12-12]
- 69 [67]小玩童, "Reliable UDP一览: 那些能替代TCP的RUDP方案", Personal Blog, 2020. Available: <https://juejin.cn/post/6844904089218711559> [Accessed:2020-12-12]
- 70 [68]WIKI, "HTTP/3", WIKI, 2020. Available: <https://en.wikipedia.org/wiki/HTTP/3> [Accessed:2020-12-12]
- 71 [69]哈库纳, "聊一聊游戏服务器架构设计—聊天功能的那些事", Personal Blog, 2016. Available: <https://my.oschina.net/ta8210/blog/709075> [Accessed:2020-12-12]
- 72 [70]云风, "AOI服务的设计与实现", Personal Blog, 2012. Available: https://blog.codingnow.com/2012/03/dev_note_13.html [Accessed:2020-12-12]
- 73 [71]Jerry, "大世界同步方案ReplicationGraph", 知乎, 2019. Available: <https://zhuanlan.zhihu.com/p/56922476> [Accessed:2020-12-12]
- 74 [72]王杰, "揭秘重度MMORPG手游后台性能优化方案", 知乎, 2018. Available: <https://zhuanlan.zhihu.com/p/49787350> [Accessed:2020-12-12]